FACULTY OF ENGINEERING AND NATURAL SCIENCES

CAPSTONE FINAL REPORT

SMART CAR AND DRIVER MONITORING APP FOR COMMERCIAL FLEET MANAGAMENT

Al- Attabi, Ashraf Atheer Ali – Mechatronics Engineering
Aleassa, Emad Alden Ahmad - Artificial Intelligence Engineering
Al- Tamimi, Rama Ayman Moh'd Ghazi - Artificial Intelligence
Barboti, Halit - Artificial Intelligence Engineering
Jasim, Zaid Khalid Jasim – Software Engineering
Mosleh, Mohammed Omar Mosleh – Software Engineering
Qushtom, Karam Zuheir Thaher – Mechatronics Engineering
Wardeh, Joud Luai Yousef - Artificial Intelligence

Advisors:
Prof. Fatih Kahraman – Artificial Intelligence
Prof. Binnur Kurt - Artificial Intelligence
Prof Senem Seven – Mechatronics
Sama Habibi - Software

June, 2026

STUDENT DECLARATION

By submitting this report, as partial fulfilment of the requirements of the Capstone Design Project course, each member of the project group certifies that:

they have given credit to and declared (by citation), any work that is not their own;

they have not received unpermitted aid for the project design, construction, report writing or presentation;

they have not falsely assigned credit for work to another student in the group and that each member's contribution is properly noted.

DECLARATION OF AI USAGE

This project report was developed with the assistance of generative AI tools (e.g., ChatGPT, Copilot, Gemini, Claude, etc.). The use of AI in this work was limited to the following activities:

- Brainstorming and idea generation
- Literature search and summarization
- Grammar and language editing
- Code generation and debugging assistance
- Data analysis support
- Other (please specify): _____

Validation and Authorship I certify that:

I have personally verified all facts, citations, and data points provided by the AI.

I have not simply copied and pasted AI outputs; all final text is my own writing or a significant rewrite of AI suggestions.

I accept full responsibility for the integrity and accuracy of this report.

Project Team

Department 1 : Artificial Intelligence

Member 1 : Rama Ayman Moh'd Ghazi Al-Tamimi

Member 2 : Joud Wardeh

Member 3 : Halit Barboti

Member 4 : Emad Alden Ahmad Hasan Aleassa

Advisor(s) : Fatih Kaharman, Binnur Kurt

Department 2 : Mechatronics Engineering

Member 1 : Karam Zuhier Thaher Qushtom

Member 2 : Ashraf Atheer Ali Al-Attabi

Advisor : Senem Seven, *Ph.D.*

Department 3 : Software Engineering

Member 1 : Zaid Khalid Jasim Jasim

Member 2 : Mohammed Omar Mosleh Mosleh

Advisor : Sama Habibi

ABSTRACT

Commercial fleet management requires reliable methods for monitoring driver safety, especially when driver fatigue, drowsiness, and distraction can increase accident risk. This report presents the Smart Car and Driver Monitoring App for Commercial Fleet Management, a multi-department capstone project that integrates Artificial Intelligence, Software Engineering, and Mechatronics contributions into a unified embedded, mobile, and backend architecture deployed on an NVIDIA Jetson Orin Nano on a 1:8 scale RC vehicle. The system generates a continuous Driver Reliability Score, starting from 100 and decreasing based on detected unsafe driving behaviours.

The in-cabin Artificial Intelligence subsystem focuses on detecting driver-related risk indicators using a driver-facing camera and includes four main modules: face detection, drowsiness detection, emotion classification, and head-pose estimation. The face detector identifies the driver's face region from live camera frames and provides the input region for the following modules. The drowsiness module combines facial landmark analysis with eye-state classification to detect fatigue-related behavior. Eye Aspect Ratio is used to estimate eye closure, while additional temporal logic is applied to reduce false positives caused by normal blinking. The emotion module uses an eight-class emotion recognition model, but the final system maps the output into three operational categories: sad, angry, and neutral, with all classes except sadness and anger treated as neutral to simplify scoring and reduce unnecessary emotional overinterpretation. The head-pose module estimates driver orientation and supports attention monitoring by detecting when the driver's face is not properly directed toward the driving task. The approximate measured accuracy of the drowsiness model is 92%, while the emotion model achieved approximately 85% accuracy.

The AI subsystem also includes road monitoring through YOLOPv2, which was first validated in the CARLA simulator to confirm lane detection and behavioural logic under controlled driving scenarios including varying weather and traffic conditions. This simulation phase enabled early verification of lane-deviation concepts before deployment on physical hardware. On the Jetson Orin Nano, YOLOPv2 was deployed using a TensorRT-optimised inference pipeline to achieve real-time performance. Due to limited precision of segmentation outputs on a custom yellow-tape track, a dedicated HSV-based lane-deviation estimator was also developed, processing raw camera frames using colour masking, region-of-interest filtering, and histogram-based lane extraction to compute a normalised deviation score in the range [0.0, 1.0].

The Mechatronics subsystem provides the physical platform, including the RC vehicle, IMX219 CSI camera mounted in a downward-facing configuration, and a controlled test track constructed from black cardboard with yellow lane markings, enabling consistent data acquisition for both simulation validation and real-world testing.

The Software Engineering subsystem manages system integration, data processing, and user interaction. It includes a Jetson-hosted FastAPI and WebSocket service, an Android application for live telemetry and scoring, and a backend system for session storage, analytics, and administrator review. All AI outputs are transferred as processed indicators rather than raw video, supporting privacy-aware integration with the scoring system through a lightweight bridge interface that enables structured communication of driver-state indicators without raw video transfer.

Overall, the project delivers a modular, edge-based driver monitoring framework combining AI perception, embedded deployment, simulation validation, mobile scoring, and scalable software infrastructure for real-time fleet safety assessment.

Key Words: Driver monitoring, Drowsiness detection, Emotion recognition, Head pose, Edge AI, CARLA, YOLOPv2

TABLE OF CONTENTS

LIST OF TABLES

LIST OF FIGURES

LIST OF ABBREVIATIONS

AI - Artificial Intelligence

API - Application Programming Interface

APK - Android Package Kit

BCrypt - Password-hashing algorithm used for credential protection

CRUD - Create, Read, Update, Delete

CSRF - Cross-Site Request Forgery

DAO - Data Access Object

DI - Dependency Injection

DTO - Data Transfer Object

HTTP - Hypertext Transfer Protocol

JPA - Java Persistence API

JSON - JavaScript Object Notation

KSP - Kotlin Symbol Processing

MVC - Model-View-Controller

MVVM - Model-View-ViewModel

REST - Representational State Transfer

Room - Android Jetpack persistence library over SQLite

SQL - Structured Query Language

SQLite - Structured Query Language embedded database engine

UI - User Interface

UX - User Experience

ONNX – Open Neural Network Exchange

1. Introduction

1.1 Background

Driver monitoring systems are becoming an important part of intelligent transportation and commercial fleet safety. Many traffic incidents are influenced by human factors such as fatigue, drowsiness, distraction, and reduced attention. For fleet operators, these risks are more critical because drivers may spend long periods on the road, and unsafe behavior can lead to accidents, vehicle damage, financial loss, and reduced operational reliability.

Halit Barboti and Emad Alden Aleassa were responsible for the main in-cabin AI work. Their part includes face detection, drowsiness detection, emotion classification, and head-pose estimation. Rama Ayman Moh'd Ghazi Al-Tamimi and Joud Wardeh contributed to Road Monitoring AI subsystem which focused on the outside part of the vehicle.

The in-cabin Artificial Intelligence subsystem of this project focuses on monitoring the driver directly using a camera placed inside the vehicle which analyzes the driver's face, eyes, facial expression, and head orientation. The purpose is not to identify the driver personally, but to detect visual indicators that may suggest fatigue, drowsiness, emotional distress, or attention loss. The final generated output is not the final Driver Reliability Score. Instead, the subsystem generates processed driver-state indicators that are transferred to the software layer. These indicators include drowsiness status, emotion category, head-pose status, and face-detection status. The software team then uses these outputs as part of the scoring system and user-facing application.

The road monitoring artificial intelligence subsystem goes further than just observing the driver, instead they track the outside part and link it to the driver’s actions such as unintentional lane deviations without a blinker turned on. This signals that the driver could appear focused when in fact he’s completely dazed off, so this subsystem was added as an additional measure of security for when the system fails to pick up on the driver’s lack of focus. Such embedded AI systems on edge devices such as the NVIDIA Jetson platform make it feasible to deploy such monitoring directly on vehicles without reliance on cloud connectivity. This project sits at the intersection of computer vision, embedded systems, and multi-team systems integration. Not only was it integrated on the jetson but it was also simulated on CARLA, open source driving simulator where the lane deviation logic was simulated along with, vehicle motion, and obstacle presence from a front-facing camera. Because testing such a system on real roads carries safety and cost risks, CARLA was used to provide a controlled, repeatable environment in which the perception pipeline could be exercised under varied traffic, lighting, and weather conditions.

These driver-state indicators feed into a broader software infrastructure designed to transform raw telemetry and risk signals into structured, explainable, and persistent driver-safety records. Commercial fleet operations require reliable software systems for monitoring driver behaviour, recording trip history, and supporting safety-related administrative decisions. In many fleet environments, driver performance is reviewed only after incidents occur or through isolated telemetry values that do not explain how behaviour changes over time. This creates a need for software that can transform raw telemetry and risk indicators into structured, explainable, and persistent driver-safety records.

This project falls within the areas of mobile software engineering, backend systems, fleet management, edge-connected telemetry processing, and safety-oriented data management. From the Software Engineering perspective, the project implements the software layer of a Driver Reliability and Safety Scoring System. The software subsystem connects structured telemetry from the vehicle and AI subsystems to an Android scoring application and a backend administration platform.

The software work is divided into two connected phases. Phase 1 is the Android/Kotlin client, which is responsible for local driver workflows, telemetry mapping, deterministic reliability scoring, session

handling, offline-first persistence, and client-side integration boundaries. Phase 2 is the Java Spring Boot backend-admin system, which is responsible for authenticated upload ingestion, persistent historical storage, secure administration, driver and vehicle management, session review, and fleet analytics.

The main technical contribution of the software subsystem is the complete path from local telemetry processing to backend historical administration. During a driving session, the Android client receives structured telemetry, converts it into domain values, applies the Driver Reliability Score logic, detects events and escalation patterns, and stores the completed session locally. After the session ends, the backend receives the completed session upload, validates it, stores the uploaded score and supporting artifacts, and exposes the results to administrators through protected web pages and analytics views.

A key differentiating aspect of the project is the separation between live score production and historical administration. The Android client owns real-time scoring because it must continue operating locally during a session. The backend owns authenticated storage, review, and analytics after the session is completed. This design keeps the scoring process explainable, avoids silent backend recomputation of historical results, and supports a clearer audit trail for driver-safety records.

Karam Zuheir Thaher Qushtom and Ashraf Atheer Ali Al-Attabi were in charge of the Mechatronics subsystem. They built the parts that the in-cabin AI and the road-monitoring AI and the software scoring subsystems all need to work. This includes the RC vehicle chassis and the real-time ESP32 control firmware and the Xbox-controller human interface and the vehicle telemetry stream that the Jetson Orin Nano uses and then the Android scoring engine uses.

The Mechatronics subsystem is not supposed to figure out what the driver is doing or calculate the risk. That is what the AI and Software departments do. The Mechatronics subsystem gives the signals that the AI and Software departments need to make sense of things. These signals are things, like speed and acceleration and steering angle and turn-signal state and ignition state. Without a working vehicle platform that can send out telemetry data the AI cannot understand what is happening. The Driver Reliability Score does not mean anything.

The Mechatronics subsystem is where embedded real-time control and sensor integration and human-machine interfacing all meet. The ESP32 microcontroller does the tasks that need to happen on time. Like making the actuators work and sampling the IMU and counting the Hall-effect pulses. The telemetry data is sent to the Jetson Orin Nano over a USB link. This makes the Mechatronics subsystem the base layer that the entire Driver Reliability Score model is built on. The Mechatronics subsystem is the foundation that the Driver Reliability Score model needs to work. The Mechatronics subsystem provides the data that the Driver Reliability Score model uses.

1.2 Problem Statement

The main problem addressed by the AI system is the lack of a reliable, low-cost, and real-time method for detecting unsafe driver states using only a standard camera and edge hardware. Driver drowsiness and reduced attention may appear gradually, and they are difficult to detect from a single frame or a single visual cue. For example, a normal blink should not be interpreted as drowsiness, and a temporary head movement should not immediately be interpreted as distraction.

The system therefore needs to distinguish between normal driver behavior and sustained risky behavior. This requires combining multiple AI and computer vision techniques, including face detection, eye analysis, emotion recognition, and head-pose estimation. The subsystem must also operate efficiently on the NVIDIA Jetson Orin Nano, where computational resources are limited compared with a desktop workstation.

Another important problem is privacy. Since the camera is placed inside the vehicle, raw video should not be unnecessarily stored or transmitted. The AI subsystem must process the video locally and provide only structured outputs to the rest of the system. This makes the system more suitable for fleet applications, where drivers may be concerned about continuous video surveillance.

However, producing structured outputs at the edge is only part of the challenge. Once those outputs leave the AI subsystem, a second problem emerges: driver and vehicle telemetry are not useful unless they are converted into consistent, explainable, persistent, and reviewable records. Raw values such as drowsiness score, lane deviation, speed, braking behaviour, emotional-risk indicators, obstacle distance, and session timing do not automatically provide meaningful fleet-safety insight. They must be normalized, scored, stored, and presented through reliable software workflows.

The affected users are drivers, fleet administrators, and system maintainers. Drivers need a system that can process session behaviour and produce a meaningful reliability score. Fleet administrators need protected access to driver records, completed sessions, event histories, score trends, and aggregate analytics. System maintainers need a clean architecture where scoring logic, persistence, networking, validation, authentication, and user-interface logic are separated and testable.

Another thing to note is that there is no lightweight pipeline existing for detecting lane deviation on a small-scale RC car using a downward-facing CSI camera and feeding that data into a unified telemetry system shared across multiple engineering sub-teams. Applying YOLOPv2 on a fake built road and detecting lane departures can be conceptually difficult due to the difference between the real road images YOLOPv2 was trained on as opposed to the fake track built by the road monitoring AI subsystem. Additionally, road monitoring AI modules need to be validated against a wide range of driving scenarios, including different weather, lighting, and traffic conditions, before they can be trusted on real vehicles. Collecting this range of scenarios in the real world is expensive, slow, and sometimes unsafe (e.g., heavy rain, night driving, or dense traffic with pedestrians). This part of the project addresses that gap by building a CARLA-based simulation that reproduces a front-camera driving view and feeds it into a lane-deviation, speed/acceleration, and obstacle-detection pipeline, so the Road Monitoring AI can be exercised safely and repeatably.

The core software problem is therefore to preserve the meaning of telemetry across the full software pipeline. The Android client must support local scoring, local storage, event detection, escalation detection, and upload preparation. The backend must accept only authenticated completed-session uploads, validate the uploaded structure, persist the session summary and child records, and provide administrator access to historical data.

This problem is important because driver reliability data can influence administrative interpretation of driver behaviour. If the score is calculated inconsistently, if historical records are rewritten, if events are lost, or if administrative data is exposed without protection, the system becomes unreliable as a safety-support tool. The software design therefore prioritizes deterministic scoring, offline-first operation, explicit data contracts, authenticated backend ingestion, protected administrator access, and preservation of historical session truth.

The Mechatronics subsystem must answer a specific engineering problem: how to produce a physically accurate, real-time stream of vehicle-state telemetry from a small-scale RC platform, using low-cost, non-automotive-grade hardware, in a form that is directly consumable by the software scoring engine without further interpretation.

This breaks down into several concrete sub-problems. First, the platform must be drivable by a human operator with both forward propulsion and a controllable, instantaneous braking response, using hardware (a single-click forward/reverse ESC) that does not natively provide a true brake mode. Second, the platform must derive wheel speed without a purpose-built automotive speed sensor, using a low-cost Hall-effect sensor and a small number of magnets, while remaining accurate enough to support the software's speed-derived deceleration and harsh-braking detection logic described in

Section 3.2. Third, acceleration must be measured from an IMU mounted on a non-rigid, vibration-prone 3D-printed chassis, with gravity-axis contamination and resting-offset bias corrected in firmware rather than left for the consuming software to interpret. Fourth, the resulting telemetry must be packaged into a schema-compatible JSON contract and transmitted reliably enough that the Jetson-side parser does not silently drop fields.

The affected stakeholders are the AI and Software departments, who depend on this telemetry being both timely (20 Hz) and physically truthful (correctly signed, correctly scaled, and zero-biased), and the end demonstration itself, which requires the vehicle to be safely controllable and to fail safe if the link to the Jetson is lost.

1.3 Project Objectives

The main objective of the in-cabin AI subsystem is to detect driver-state indicators from live camera input and provide structured outputs that support the Driver Reliability Score.

The specific objectives are:

1. To implement a face-detection module that identifies the driver's face region from live in-cabin camera frames.
2. To implement a drowsiness-detection module capable of identifying fatigue-related eye behavior with approximately 92% accuracy.
3. To use Eye Aspect Ratio and temporal filtering to reduce false drowsiness alerts caused by normal blinking.
4. To implement an emotion-recognition module with approximately 85% accuracy, using three operational categories: sad, angry, and neutral.
5. To simplify the eight-class emotion model output by mapping all non-sad and non-angry classes to neutral.
6. To implement head-pose estimation for detecting driver orientation and supporting attention monitoring.
7. To prepare AI outputs in a structured format suitable for transfer to the Jetson-hosted API and WebSocket system implemented by the software team.
8. To ensure that the AI subsystem provides processed indicators rather than raw video, supporting privacy-aware integration.
9. Implement a robust HSV-based yellow tape lane deviation estimator producing a normalised score ranging between [0.0, 1.0].
10. Optimize inference speed using TensorRT FP16 to achieve real-time performance (>15 FPS).
11. Deploy YOLOPv2 for multi-task road perception including lane line segmentation.
12. Ensure automatic startup upon the Jetson Orin Nano’s boot for continuous operation during vehicle deployment.
13. Develop a CARLA-based driving simulation environment with manual ego-vehicle control, front-camera streaming, and testing yolopv2 under different traffic, weather, and lighting conditions.
14. Implement real-time road monitoring and vehicle analysis, including YOLOPv2 lane segmentation, lane-deviation estimation, vehicle speed and acceleration calculation, and obstacle detection.

These AI outputs serve as the foundation for the software subsystem, whose main objective is to build a complete software foundation for driver reliability monitoring and fleet administration. The final system must support local Android scoring during driving sessions, completed-session persistence, backend upload, secure administration, and historical analytics.

The final software objectives are:

1. Develop a buildable Android/Kotlin thick-client application for driver-facing and admin-facing workflows.
2. Implement a deterministic Driver Reliability Score model based on continuous risk penalties, discrete safety-event penalties, and escalation-pattern penalties.
3. Map incoming telemetry into normalized domain values, including timestamp, speed, acceleration, obstacle distance, drowsiness score, emotional-risk value, lane deviation, and turn-signal state.
4. Store drivers, sessions, events, escalations, score-history points, vehicles, and synchronization metadata locally using Room/SQLite.
5. Support offline-first session handling so that scoring and local persistence can continue even when backend connectivity is unavailable.
6. Implement client-side boundaries for telemetry streaming, Jetson/vehicle control, backend authentication, completed-session upload, and admin-data access.
7. Develop a Java Spring Boot backend-admin system with secure administrator authentication and protected routes.
8. Implement authenticated API-client upload ingestion for completed driving sessions.
9. Validate and persist uploaded session summaries, safety events, escalation records, score-history points, and ingestion issue flags.
10. Provide backend management workflows for drivers and vehicles.
11. Provide administrator review workflows for sessions, driver details, driver analytics, global dashboard metrics, and fleet analytics.
12. Preserve uploaded final scores, penalty totals, validity status, upload-processing status, event records, escalation records, and score-history points without silently recomputing the completed session result on the backend.

The original proposal mainly emphasized a thick-client Android application and a driver reliability scoring model. During implementation, the software scope became a two-phase system. The Android client remained responsible for live local scoring and session production, while the backend-admin system became the authority for authenticated upload ingestion, persistent historical review, and fleet administration. This modification made the software system more complete because a fleet-management solution requires not only local scoring but also secure historical access and administrative oversight.

The objectives of the Mechatronics subsystem are:

- To implement real-time ESP32 firmware controlling a brushless motor (via 60 A ESC) and a steering servo from a Bluetooth Xbox Series X controller.
- To implement a safe-start gate that prevents motor activation until both control triggers are confirmed at neutral.
- To implement proportional, instantaneous reverse/brake control distinct from ramped forward acceleration, matching the single-click forward/reverse behavior of the ESC.
- To integrate an MPU6050 IMU over I2C, including gyroscope offset calibration and accelerometer resting-offset correction, to produce a zero-biased, correctly signed fore/aft acceleration signal.
- To integrate a US1881 Hall-effect sensor for interrupt-driven wheel-speed measurement, converting pulse counts into a linear speed estimate in km/h.
- To package vehicle telemetry (speed, steering angle, acceleration, obstacle distance placeholder, turn-signal state, ignition state, stationary state) into a JSON schema matching the Jetson FastAPI server's ESP32 contract, transmitted at 20 Hz over USB serial.
- To implement a controller-disconnect fail-safe that returns the vehicle to neutral if the Xbox controller connection is lost beyond a defined timeout.
- To design and fabricate a custom 3D-printed chassis with vibration-dampened sensor mounts, housing the Jetson Orin Nano, ESP32, IMU, Hall sensor, motor/ESC, and battery.

- To implement turn-signal LED outputs (left/right) mapped to controller bumper buttons, reflected in the telemetry's turnSignal field.
- To develop a browser-based telemetry dashboard that receives and visualizes live vehicle data — including speed, steering angle, acceleration, and system states — over a WebSocket connection to the Jetson FastAPI server, providing real-time monitoring of the platform during testing and demonstration.

The mechatronics subsystem forms the physical and embedded foundation of the ADAS testing platform, bridging high-level perception algorithms running on the Jetson Orin Nano with real-world actuation and sensing at the vehicle level. The objectives above collectively define a system that is safe, responsive, and instrumented: safe through the fail-safe and safe-start gate mechanisms; responsive through low-latency Bluetooth control and non-blocking firmware architecture; and instrumented through the IMU, Hall-effect speed sensor, and structured telemetry output. The browser-based dashboard extends this instrumentation to the operator, enabling real-time visibility into vehicle state during track testing without requiring a dedicated monitor or serial connection. Together, these objectives ensure that the platform can serve as a reliable and observable testbed for validating ADAS algorithms under controlled, repeatable conditions.

1.4 Scope and Limitations

The scope of Halit and Emad's work is limited to the in-cabin camera AI subsystem. This includes face detection, drowsiness detection, emotion recognition, and head-pose estimation. The subsystem analyzes the driver-facing camera feed and produces processed driver-state outputs for use by the software scoring layer.

The subsystem does not calculate the final Driver Reliability Score by itself. The score calculation, database handling, WebSocket/API hosting, and application-level integration were handled by the software team. The AI subsystem provides the detection results that support the scoring process.

The subsystem also does not perform identity recognition, face recognition, or medical diagnosis. It does not determine whether a person has a medical condition. Instead, it detects observable visual patterns related to drowsiness, emotion category, and head orientation.

The main technical limitations include camera quality, lighting variation, face angle, occlusion, glasses, and computational constraints on the Jetson Orin Nano. Poor lighting or incorrect camera placement may reduce the reliability of face detection, landmark extraction, emotion recognition, and head-pose estimation.

From an ethical and privacy perspective, the subsystem is designed to avoid unnecessary raw video transfer. The AI output is represented as processed values and event indicators. This reduces the amount of sensitive visual data shared across the system.

As for the CARLA simulation environment, the scope includes the camera and YOLOPv2-based lane-deviation pipeline, the speed/acceleration and obstacle telemetry, and weather/traffic test scripts. It does not cover the design of the Road Monitoring AI's downstream decision logic, nor any physical/hardware testing. Limitations include: testing was done with a single ego vehicle (Dodge Charger) on one map (Town10HD_Opt); driving was manual via keyboard rather than autonomous; YOLOPv2 inference runs every 8 frames rather than every frame to keep the simulation smooth; and the static-obstacle fallback can only reliably classify obstacles as generic "static_obstacle" rather than identifying their exact type, since many parked cars in CARLA are static map meshes rather than actors.

The scope of the Road Monitoring subsystem is limited to the external camera pipeline running on the Jetson Orin Nano. This includes IMX219 CSI camera integration, YOLOPv2 inference deployment, HSV-based, lane deviation estimation, TensorRT engine conversion, per-frame deviation score output, MKV video recording, and autostart configuration. The subsystem produces a single normalised deviation score per frame and writes it to a shared temporary file. It does not calculate a final driver reliability score, perform driver identity recognition, process the driver-facing camera feed, handle vehicle speed or steering data, or manage network communication.

The physical track environment is a controlled indoor setting constructed from black cardboard sheets with yellow adhesive tape lane edges approximately 50cm apart. Each cardboard sheet is 50x70 cm. The pipeline does noy include GPS-based positioning, multi-lane tracking. Technical constraints include the Jetson Orin Nano's 8GB shared GPU and CPU memory pool, which limits concurrent process load during TensorRT engine building and inference.

These structured outputs are then consumed by the software subsystem, whose scope includes the Android client, local scoring engine, telemetry mapping, local database persistence, ViewModel and UI workflows, Jetson communication boundaries, backend upload DTOs, synchronization readiness, Spring Boot backend APIs, upload validation, MySQL persistence, administrator authentication, driver management, vehicle management, session review, dashboard pages, and analytics services.

The software subsystem does not include AI model training, camera inference internals, mechanical chassis design, motor-control firmware, physical sensor mounting, power-system design, or real vehicle actuation. These are handled by the AI and Mechatronics subsystems. From the software perspective, those subsystems provide structured telemetry and vehicle-state inputs through defined interfaces. The software consumes those inputs but does not depend on their internal implementation.

The Android client is limited by the quality and consistency of the telemetry it receives. If telemetry is delayed, missing, incorrectly normalized, or noisy, the scoring output may become less reliable. The client also requires real-device validation with live Jetson telemetry to confirm end-to-end behaviour under realistic operating conditions. Repository-level build success and unit tests provide important evidence, but they do not fully prove field performance.

The backend is limited by its role as a completed-session ingestion and administration system. It does not perform live scoring, raw sensor processing, AI inference, or physical telemetry reconstruction. Its responsibility is to authenticate uploads, validate uploaded records, preserve historical session truth, and provide administrative review and analytics.

The main technical constraints are real-time telemetry handling, deterministic scoring, local persistence reliability, synchronization correctness, backend validation, database consistency, and access control. The main realistic constraints are prototype-level deployment configuration, limited production hardening, limited end-to-end hardware validation, and the need for stronger operational procedures before real fleet use. Before production deployment, secrets should be moved out of source code, HTTPS should be enforced, database migrations should be managed, backups should be defined, monitoring should be strengthened, and deployment recovery should be tested.

Ethically, the Driver Reliability Score should be treated as a decision-support indicator rather than an absolute judgment of a driver. The software stores driver-related safety records, so access control, privacy, and explainability are necessary. The backend should not store raw facial video or unnecessary biometric data. The software should operate on structured telemetry, scores, events, escalations, and session summaries only.

The scope of the Mechatronics subsystem is limited to the physical vehicle platform: mechanical design and fabrication, real-time embedded firmware, sensor acquisition, actuation, and telemetry transmission to the Jetson. It explicitly excludes the AI perception models and the

application/scoring software, which consume this subsystem's telemetry at the defined JSON interface but are owned by the other departments.

Limitations are primarily economic and physical. The platform uses low-cost, off-the-shelf, non-automotive-grade components and 3D-printed parts, introducing a maintenance "time penalty" — a structural failure requires reprinting rather than a part swap. The 3D-printed chassis lacks the rigidity of a production vehicle, so vibration can introduce IMU noise, mitigated by vibration-dampening features at sensor mounts. The Jetson is a non-real-time host while the ESP32 is real-time, introducing timing variability at the USB serial boundary. High-current LiPo batteries require strict fire-safety and charging supervision. Component sourcing within Turkey at the required specification and budget proved to be a recurring constraint.

The ESC used is single-click forward/reverse, not forward/brake — there is no true active-brake mode in this design; deceleration is achieved by commanding reverse against forward motion, not by an ESC-internal brake circuit. Because the ESC's low-band pulse range serves as both reverse drive and the braking mechanism, braking authority is tied directly to the depth of the reverse command: the firmware maps trigger pressure proportionally to pulse depth so that a light press produces gentle deceleration and a full press produces maximum opposing torque, but the physical stopping force is ultimately bounded by the ESC's reverse current limit rather than a dedicated brake circuit.

Steering control is subject to an electrical noise limitation arising from the shared ground topology between the servo, ESC, and ESP32. Under the current wiring, the servo's inrush current during hard steering inputs produces a ground-reference disturbance on the ESC's signal line, which the ESC can momentarily interpret as a below-neutral (reverse) command. This was characterized during testing and partially mitigated in firmware by raising the idle pulse to true neutral and applying a proportional brake response; the complete hardware fix — relocating the servo ground return to the ESC's BEC ground rather than the ESP32 header — was identified and documented but not yet implemented in this iteration due to time constraints.

The MPU6050 IMU provides reliable, drift-free roll and pitch (tilt) angles derived from gravity-referenced accelerometer data, and a gyroscope-calibrated fore/aft acceleration signal corrected for resting offset. Yaw (heading) is derived solely from gyroscope integration and accumulates drift continuously in the absence of a magnetometer; it is suitable only for short-term relative rotation measurement and is not a stable compass heading. The Hall-effect sensor provides speed magnitude only — it cannot distinguish forward from reverse motion, so reverse driving registers as a positive speed in telemetry. The reported steeringAngleDeg field is a commanded value derived from the servo pulse, not a physically measured angle — there is no dedicated steering-angle sensor on the platform. The obstacleDistanceMeters field is a constant placeholder, since no ranging sensor is fitted to this iteration of the platform.

The Bluetooth link between the Xbox Series X controller and the ESP32 introduces a small and variable latency inherent to the BLE protocol. Under normal operating conditions this latency is imperceptible, but a controller dropout — caused by range, obstruction, or interference — triggers the firmware's disconnect fail-safe, which commands neutral after a defined timeout. This means the vehicle coasts briefly before stopping if the link is lost, rather than stopping instantaneously; this behavior is a known and accepted limitation of a wireless control architecture without a hardware kill switch.

Power architecture on the platform separates the high-current motor/ESC circuit from the logic supply, with the Jetson powered through a dedicated buck converter and the ESP32 powered from USB during development. The BEC integrated into the ESC supplies the steering servo. This

separation was chosen to prevent motor switching noise from coupling into the logic rails, though full decoupling between the servo ground return and the ESP32 signal reference was not achieved in this iteration, as noted above.

The browser-based telemetry dashboard visualizes live vehicle data over a WebSocket connection and is therefore dependent on network availability between the host machine and the Jetson. It is intended as a development and demonstration tool and is not a safety-critical interface; loss of the dashboard connection has no effect on vehicle operation.

1.5 Report Organization

This report is organized according to the Capstone Final Report template and covers the comprehensive contributions of the AI, Mechatronics, and Software departments. Section 1 introduces the project background, problem statements, specific objectives, scopes, and limitations across the in-cabin AI pipeline, the software application, the road monitoring lane deviation pipeline, and the mechatronics vehicle platform — including its embedded firmware architecture, sensor integration, actuation design, power architecture, and telemetry interface. Section 2 reviews related work spanning driver monitoring, drowsiness detection, emotion recognition, face detection, head-pose estimation, lane detection, multi-task road perception models, edge AI deployment on NVIDIA Jetson hardware, TensorRT model optimization, HSV-based color segmentation, offline-first mobile architecture, edge-connected telemetry processing, explainable scoring, backend administration systems, and RC-scale ADAS testing platforms including embedded motor control, BLE wireless control, and inertial sensing for small-scale vehicle dynamics. Section 3 presents the methodology and technical architecture of all pipelines and subsystems, including the in-cabin AI processing pipeline, the Android thick-client design, the backend-admin infrastructure, the four-stage road frame processing architecture, iterative development approaches, physical track construction and calibration, integration data flows, the mathematical basis of the Driver Reliability Score and backend analytics, and the mechatronics subsystem architecture covering the ESP32 firmware design, ESC and servo control loops, IMU and Hall-effect sensor integration, the JSON telemetry schema, the browser-based telemetry dashboard, and the power and wiring architecture of the vehicle platform. Section 4 provides a detailed implementation account covering the face detector, drowsiness model, emotion model, and head-pose module, alongside camera integration with GStreamer, YOLOPv2 TorchScript inference, the TensorRT conversion process with ONNX graph surgery, and the HSV lane deviation estimator. It also details the Android client and Spring Boot backend implementation — covering scoring, persistence, networking, upload validation, security, administrator pages, analytics services, output interfaces, and autostart configurations — the mechatronics firmware implementation covering safe-start gating, proportional reverse/brake control, non-blocking IMU acquisition with gyroscope offset calibration, interrupt-driven wheel-speed measurement, 20 Hz JSON telemetry transmission, controller-disconnect fail-safe logic, and turn-signal LED output — and structures all major technical challenges and their respective solutions across all departments. Section 5 presents the testing, validation, and benchmarking approach, including reported approximate accuracies of the AI models, FPS measurements, lane deviation accuracy across known positions, single-edge fallback validation, TensorRT performance benchmarking, Android build verification, local unit tests, backend workflow and upload ingestion validation, mechatronics bench validation of ESC arming, ramp behavior, brake response, IMU axis mapping, Hall-effect pulse counting, and telemetry schema conformance, and remaining end-to-end integration gaps. Section 6 discusses the final results and their significance, analyzes the strengths and limitations of the integrated implementations across all

three departments, and evaluates the achieved system outcomes against the original project objectives including the mechatronics platform's role as the physical testbed for the AI and software pipelines. Section 7 summarizes project management, work distribution, and collaboration across the respective subsystems and sub-teams, including the internal split of mechatronics responsibilities between chassis and mechanical design, firmware and electronics, and Jetson bring-up. Section 8 addresses ethical, safety, privacy, security, and sustainability considerations relevant to both user-facing software and embedded, real-time edge vehicle monitoring systems, including hardware fail-safe design, LiPo battery fire-safety protocols, and the physical safety implications of an untethered autonomous-capable vehicle platform. Section 9 concludes the report with the main achievements, lessons learned, and strategic recommendations for future work across all three departments.

2. Literature Review / State of the Art

The rapid evolution of autonomous and semi-autonomous vehicle technologies has fundamentally reshaped the landscape of commercial fleet management. Traditionally, fleet safety relied on reactive measures, such as reviewing accident reports or analyzing telematics data long after an incident occurred. However, modern industry demands proactive, real-time intervention to mitigate risks associated with human error, which remains a primary cause of road accidents. Consequently, current research focuses on integrating three distinct domains: intelligent perception algorithms for behavioral analysis, robust mechatronic systems for physical control, and resilient software architectures for decision-making.

Recent developments in computer vision techniques enabled the transition from passive observation to active safety engagement. The authors Madane et al. [1] used Deep Learning techniques (CNNs) for yawn recognition and eye closure recognition in real time with an accuracy rate of 97%. However, while their work primarily focused on achieving a high accuracy rate, it lacked an integrated approach to assess the overall level of the driver's risk. Our project aims to address that shortcoming by using the internal data with the environment data to provide the fleet managers with an assessment of the "Driver Reliability Score." Similarly, another study conducted by Florez-Zela et al. [2] verified the effectiveness of simplistic CNN models accessible in edge hardware, including NVIDIA Jetson orin Nano, which was capable of reaching an accuracy of 96% at 14 frames per second using conventional RGB cameras. We extended this project further by ensuring cost-effectiveness by using an algorithm called "Temporal Smoothing" for higher resistance to the sensors' noise, where fps (frames per second) can be associated with false alarms due to frame-by-frame observation.

As for the external, environment perception, a single camera or a single sensor often face issues with adverse lighting conditions. This implies that fusing LiDAR data with camera images is capable of increasing the accuracy rate by more than 12% according to Wang et al. [3] Although the online project applies data fusion for the purpose of object boundary detection, our research project applies it for the purpose of risk evaluation comparing camera-based lane drift notifications with LiDAR-based obstacle distances. For the purpose of monitoring the driver's actions, Lee and Chang [4] conducted a study that categorized the driver's actions based on the levels of risk associated with the trip. Their methodology works around a post-trip analysis however our project provides real time alerts on the application to the driver so the drivers could make corrective actions during the trip.

The project also needs robust hardware architecture solutions in computing driver monitoring tasks. Sarvajcz et al. [5] proposed a new architecture based on dividing the computation between a high-level processor in AI and a microcontroller, as dividing computational tasks can prevent system "freeze" problems. Our project extends this design by using the microcontroller as a sort of data logger, where even if the high level AI processor freezes or reboots, the low level controller has

continued to record "Ground Truth" telemetry (Speed, Steering Angle) and the integrity of the driving session log is never called into question.

As for validating the scoring logic without any physical risk, García Daza et al. [6] pioneered "Sim-to-Real" transfer the methods using CARLA simulator. We adapt this methodology for "Behavioral Simulation." Instead of conducting emergency stop tests, we conduct simulations of concrete dangerous driving behaviors (e.g., drifting, microsleeping) in a virtual environment to ensure that the scoring engine is correctly computing the penalties and degradation rates before it is used with human drivers.

It is also imperative to validate the physical platform. In this regard, a performance evaluation of NVIDIA Jetson Orin Nano carried out by Mishra and Sehgal [7] validated its capability to support heavy tasks such as real-time object detection (>30 FPS) and power under 15W, thus justifying its choice for safety prototype designs powered by batteries. On the mechanical platform, already-existing research carried out by Baharuddin et al. [8] elaborated on modifications necessary to integrate sensors in RC platforms. Our ongoing project thus expands upon this, with emphasis on vibration reduction, and our designed 3D-printed mechanical parts being tailored to attenuate vibrations which would otherwise compromise noise in the IMU readings, so that the “Harsh Braking” and “Aggressive Turning” reliability component in the reliability index would be well grounded in reality.

Effective perception needs to integrate multiple modalities to identify context**.** Abdulmaksoud and Ahmed [9] performed an extensive literature study on the use of the Transformer for sensor fusion, showing that it is vital to integrate LiDAR point clouds and camera inputs to effectively mitigate the weaknesses that each modality has in poor lighting conditions. Our proposal further generalizes this fusion paradigm for "Contextual Profiling." Instead of fusion for navigation, we apply fusion to confirm the Driver Reliability Score. A "Drowsy" warning from the camera and a "Steering Jitter" signal from the IMU sensor are checked simultaneously; for a high-priority intervention to occur, a positive confirmation is required from both sensors to avoid frustrating a human driver through false positive actions.

Another effective perception demands the integration of information across various senses for contextual understanding. Shankar [10], in evaluating the integrated AI designs, reported that while there are high-performance Systems-on-Chip (SoCs) in the likes of the Jetson, “always-on” sensing leads to non-linearly increasing power consumption that can quickly drain battery life in small vehicles”. Although our work builds on existing research by incorporating a “Hybrid Wake-Up” Strategy. Rather than having the power-hogging Jetson system running all the time for every application, we employ the ESP32 system for observing the IMU threshold levels. The resource-heavy AI perception framework is initialized only when the vehicle is in motion or in cases of “Pre-Trigger” activation.

A critical challenge in modern driver monitoring systems is the architectural decision between cloud-centric and edge-centric processing. Historically, commercial telematics solutions operated as "Thin Clients" that streamed raw sensor data to remote servers for analysis. However, a comparative study by Abdullah and Adelusi [11] indicates that while cloud models offer scalability, they introduce unpredictable latency spikes that frequently exceed the 200 ms safety threshold required for accident prevention. In contrast, edge computing, which processes data locally on the device, has been proven to significantly reduce response times. Recent findings by Pallikonda et al. [12] demonstrate that decentralized edge architectures can lower decision-making latency by up to 65% compared to traditional cloud loops, making them the only viable option for real-time alerts.

Furthermore, the centralization of sensitive biometric data in the cloud has raised substantial privacy concerns. A study on “Privacy-Preserving Driver Monitoring” by Bai et al. [13]emphasizes that transmitting facial video feeds to external servers creates significant security vulnerabilities and regulatory compliance issues. To address this, researchers propose "Privacy-Preserving" frameworks where all sensor data is aggregated and processed strictly within the vehicle or on the local edge device. This "Local-First" approach is further supported by Kim et al. [14], which argues that embedded monitoring systems utilizing edge AI modules effectively block unauthorized access by eliminating the need for constant server communication.

In addition to performance and privacy, the interpretability of automated decisions is paramount for administrative trust. While deep learning models excel at detection, they often function as opaque "Black Boxes" that fail to explain why a specific safety score was assigned. A comprehensive review of Explainable AI (XAI) in autonomous driving by Kuznietsov et al. [15], argues that safety-critical systems must employ "Interpretable Design" paradigms to ensure transparency. Moreover, a comparative analysis suggests that for regulated environments, rule-based or hybrid scoring engines are superior to pure data-driven models because they provide deterministic, human-readable justifications for every alert or penalty [16].

Finally, the reliability of the mobile application itself depends heavily on the underlying software architecture. Ensuring that a safety app functions correctly during network blackouts requires a robust "Offline-First" design. According to a guide on mobile resilience, adopting architectural patterns such as MVVM (Model-View-ViewModel) combined with local persistence libraries allows applications to maintain seamless functionality regardless of connectivity status [17]. This effectively decouples critical safety logic from the availability of cellular networks, ensuring continuous driver monitoring.

In conclusion, the synthesis of recent academic literature highlights a clear shift away from passive, cloud-dependent telematics towards active, edge-based safety systems. The integration of real-time AI perception with mechatronic actuation provides the necessary physical context for safety, while the adoption of "Thick Client" software architecture ensures that these systems remain fast, private, and explainable. By combining these advanced methodologies, the proposed project addresses the significant gaps identified in current commercial solutions, offering a comprehensive platform that not only detects risk but actively empowers fleet managers to understand and mitigate it.

3. Methodology and Technical Approach

3.1 Overall Approach

The in-cabin AI subsystem follows a modular computer vision and deep learning approach. The subsystem receives video frames from the driver-facing camera and processes each frame through four main AI modules: face detection, drowsiness detection, emotion recognition, and head-pose estimation. These modules work together to produce driver-state indicators that can be used by the software subsystem in the Driver Reliability Score.

The system is designed as an edge-AI pipeline for the NVIDIA Jetson Orin Nano. The reason for using edge processing is that driver-facing video contains sensitive information. Instead of sending raw video to an external server, the AI subsystem processes the frame locally and outputs only processed values such as drowsiness state, emotion category, face presence, and head-pose status.

The overall pipeline begins with camera frame acquisition. Each frame is passed to the face detector. If a face is detected, the face region is used for emotion recognition and facial landmark extraction. The

landmarks are then used for drowsiness detection and head-pose estimation. If no face is detected, the subsystem does not force driver-state predictions, because this could create unreliable outputs.

The AI methodology combines rule-based computer vision with deep learning. The drowsiness module uses Eye Aspect Ratio, Mouth Aspect Ratio, temporal frame counting, and a CNN-based eye-state classifier. The emotion module uses a MobileNetV2-based eight-class emotion model, but the operational output is simplified into three categories: sad, angry, and neutral. The head-pose module uses facial landmarks and a solvePnP-based geometric method to estimate yaw, pitch, and roll.

The software team handled the WebSocket and API hosting on the Jetson Orin Nano. Therefore, the AI department's responsibility was to prepare processed AI outputs that could be transferred to the Jetson-hosted communication layer, rather than implementing the full software-side data transfer system.

Once these structured outputs are made available through the Jetson-hosted communication layer, the software subsystem takes over. The software subsystem was developed using a layered, iterative, and integration-driven methodology. The project problem required more than a single application screen or isolated scoring function; it required a complete path from telemetry intake to local scoring, local persistence, backend upload, historical storage, and administrator review. For this reason, the software work was divided into two connected phases: the Android thick-client application and the Spring Boot backend-admin system.

Before beginning with the real implementation of the lane deviation logic, the team approached simulation. The steps were as followed: build a minimal CARLA connection, add the camera and YOLOPv2 inference, then incrementally layer on lane-deviation estimation, vehicle telemetry, and obstacle detection, testing manually after each addition. Each subsystem (lane deviation, obstacle detection, traffic, weather) was developed and tuned as a separate script or module so that working logic, especially the lane-deviation estimator, could be isolated and left unchanged once it produced correct behavior.

The Road Monitoring AI subsystem followed an iterative and incremental development methodology. The pipeline was first established in its simplest functional state, displaying raw frames from the IMX219 camera and was progressively extended through validated phases: YOLOPv2 TorchScript inference was added and confirmed running, then the custom HSV deviation estimator was designed and integrated, then the software output interface was implemented in order to ensure the lane deviation score could be sent to the software team accurately and swiftly, then TensorRT acceleration was applied, and finally calibration and sensitivity tuning were completed. Each phase was validated independently before proceeding to the next, which allowed hardware and software bugs to be isolated quickly without compounding failures across the system.

A custom dataset of 200 images was collected by photographing the physical cardboard track under various camera angles and positions. Each image was manually annotated with lane edge bounding regions. An attempt was made to fine-tune YOLOPv2's lane segmentation head on this dataset; however, the model weights were found to be frozen in the TorchScript export format, preventing gradient updates from being applied to the network parameters. TorchScript serialisation locks model weights at export time and does not expose them for further training without access to the original PyTorch model definition and training code, which were not available in the YOLOPv2 repository in a form compatible with the existing TorchScript checkpoint. As a result, fine-tuning could not be completed. A classical computer vision approach was therefore adopted for the deviation estimator: HSV colour segmentation combined with histogram peak detection. It is worth noting that YOLOPv2's original segmentation output does function correctly on the track and produces visible lane overlays, the custom HSV estimator was developed not because the model failed entirely, but to provide a more precise, calibrated, and computationally controllable deviation score than what could be extracted from the model's raw segmentation mask. The classical approach produces deterministic,

interpretable outputs with no dependency on additional model weights or training data and allows exact calibration to the specific track geometry.

The TensorRT optimisation phase was treated as a separate engineering task performed after the deviation logic was fully validated at PyTorch baseline speed. This ordering was deliberate: optimising a pipeline before its correctness is confirmed risks masking logical errors behind performance changes.

The overall software approach was based on separation of responsibilities. Phase 1, the Android client, was designed as the live-session and local-scoring component. It receives structured telemetry from external subsystems, maps the received data into domain-level telemetry packets, applies deterministic Driver Reliability Score logic, detects safety events and escalation patterns, stores session artifacts locally, and prepares completed-session records for backend synchronization. Phase 2, the backend-admin system, was designed as the authenticated ingestion, persistence, administration, and analytics component. It receives completed-session uploads, validates the uploaded structure, persists the uploaded historical records, and exposes the records to administrators through protected web pages and analytics views.

The Android development methodology followed a local-first architecture. This decision was made because a driver-monitoring application cannot depend on continuous internet access during an active session. The scoring engine, event detection, escalation detection, and local persistence were therefore implemented on the client side. The backend is contacted after a completed session is available for upload. This approach improves resilience because a driving session can still be scored and stored locally even if the backend connection is temporarily unavailable.

The Android codebase was structured into domain, data, network, orchestration, and UI layers. The domain layer contains the scoring model, session state, event detection, escalation detection, and core business rules. The data layer contains Room database entities, DAOs, repositories, and local persistence logic. The network layer contains DTOs, mappers, WebSocket telemetry handling, Jetson-control boundaries, and backend API contracts. The orchestration layer connects live telemetry, scoring, persistence, and synchronization workflows. The UI layer exposes driver and administrator workflows through ViewModels and Compose screens. This layered structure was selected to prevent scoring logic, database logic, network transport, and user-interface code from becoming mixed together.

The backend development methodology followed a layered Spring Boot architecture. The backend separates domain models, persistence entities, repositories, mappers, validation services, command services, query services, security components, REST controllers, page controllers, and view models. Mutating workflows, such as session upload, driver management, vehicle management, and password changes, are handled by command-side services. Read workflows, such as dashboards, session details, driver analytics, and fleet analytics, are handled by query-side services. This separation makes the backend easier to test, maintain, and extend.

A key engineering decision was that the backend should not recompute the session score produced by the Android client. The Android client owns live scoring because it has access to the active session state and telemetry sequence during the drive. The backend owns completed-session validation, persistence, and review. Therefore, the backend preserves uploaded final scores, penalty totals, events, escalations, score-history points, validity status, and upload-processing status as historical truth. This prevents the backend from producing a different interpretation of a completed session after upload.

The Driver Reliability Score methodology uses an explainable rule-based model rather than an opaque machine-learning score. Continuous penalties represent ongoing risk signals such as fatigue, lane deviation, and braking aggression. Event penalties represent discrete unsafe behaviours such as high fatigue, lane departure, harsh braking, and emotional distress. Escalation penalties represent repeated

or combined risk patterns, such as repeated harsh braking within a short time window or fatigue occurring together with lane deviation. This approach was selected because administrators should be able to understand why a score changed instead of receiving only a black-box result.

The integration methodology was contract-based. The AI and Mechatronics subsystems are treated as external providers of structured telemetry and vehicle-state values. The software subsystem does not depend on the internal implementation of camera inference, lane detection, motor control, or physical sensor acquisition. Instead, it consumes agreed fields such as timestamp, speed, acceleration, obstacle distance, drowsiness score, emotional-risk value, lane deviation, and turn-signal state. This keeps the software design modular and allows the external subsystems to evolve without requiring the scoring and backend logic to be rewritten.

Testing and validation were also performed iteratively. The Android side was validated through build execution and local unit tests for scoring, telemetry mapping, synchronization, credential behaviour, and related logic. The backend side was validated through repository-level tests and workflow checks for administrator authentication, protected routes, driver and vehicle management, upload ingestion, persistence of child records, session review, and analytics output. This validation approach supports capstone-level software confidence, while full real-device, live Jetson, deployed-backend, and physical-system testing remains a required future validation step.

Overall, the methodology combines local-first mobile design, layered software architecture, deterministic scoring, authenticated backend ingestion, contract-based integration, and repository-supported validation. This approach directly supports the project goal of converting telemetry and behavioural indicators into explainable, persistent, and administratively reviewable driver-reliability records.

The Mechatronics subsystem followed a bench-first, assemble-second methodology, adapted from the project-wide simulate-then-validate philosophy used by the AI departments. Because chassis 3D printing sat on the critical path and took longer than expected, the electronics and firmware workstream was deliberately decoupled from mechanical fabrication: every firmware module and sensor was individually verified on the bench before any component was committed to the printed chassis. This kept electronics development unblocked while printing continued in parallel, and means that once the chassis is available, final assembly proceeds against components that are already individually proven.

Hardware selection itself was subject to an iterative constraint-driven process. Component sourcing within Turkey at the required specification and budget proved to be a recurring challenge throughout development, requiring substitutions and workarounds at several points — most notably in the power protection circuit, where preferred TVS diodes were unavailable locally and were replaced with back-to-back Zener diode pairs, and in the buck converter selection, where the XL4016 was chosen as a locally available alternative to the preferred LTC3780. Each substitution was validated on the bench before acceptance, preserving the bench-first principle even when the original design had to change.

Firmware development itself proceeded as a layered sequence of working states, each fully bench-validated before the next was added: (1) core actuation — ESC throttle/brake/reverse and servo steering from the Xbox controller over Bluetooth; (2) IMU integration — orientation and motion sensing, first as a standalone test sketch to confirm axis mapping and calibration, then merged into the control loop as a non-blocking 50 ms sampled task; (3) wheel-speed sensing — US1881 Hall-effect interrupt-driven pulse counting converted to a km/h speed estimate; (4) Jetson telemetry — structured JSON output over USB serial at 20 Hz matching the schema shared with the AI and Software departments.

Each firmware layer introduced its own validation criteria before the next layer was started. Core actuation was validated by confirming ESC arming at neutral, safe-start gate operation, ramp behavior under trigger input, proportional brake response, and controller-disconnect fail-safe return to neutral

— all observed on the Serial Monitor with wheels off the ground. IMU integration was validated by a dedicated orientation test sketch that printed plain-language movement descriptions (nose up, lean left, turning right) against raw axis values, confirming correct sign convention and axis mapping for the specific physical mounting of the sensor on this chassis before any angle data entered the control loop. Hall-effect sensing was validated by spinning the wheel by hand and confirming interrupt-driven pulse counts converted correctly to a speed reading before the telemetry schema was finalized. This per-layer validation discipline meant that when a bug appeared, its source was already bounded to the most recently added layer, significantly reducing debugging time compared to a big-bang integration approach.

The control architecture itself reflects a deliberate real-time design philosophy. The ESP32 firmware loop runs without any blocking delays after setup, using non-blocking millis()-based timers to multiplex the throttle ramp (15 ms), IMU read (50 ms), and telemetry transmission (50 ms) independently. This ensures that controller input latency and steering response are never degraded by sensor reads or serial output, which was a specific design requirement given that Bluetooth input latency is already an irreducible baseline. The Jetson, running a non-real-time Linux host, consumes the telemetry stream at the USB serial boundary and is explicitly not in the control loop — actuation decisions are made entirely on the ESP32, and the Jetson receives observational telemetry only. This separation was a deliberate architectural choice to keep vehicle safety functions immune to any latency or scheduling jitter on the Jetson side.

The power architecture followed the same bench-first philosophy. The full Branch 2 protection circuit covering reverse-polarity protection via a P-MOSFET, EMI filtering, and regulated 10 V output to the Jetson via the XL4016 buck converter was simulated in LTspice and validated across four test scenarios (reverse polarity, spike injection, EMI attenuation, and motor-surge dropout) before any physical build. The LiPo voltage monitoring circuit, using a resistor divider on the ESP32's ADC, was similarly bench-verified against known voltages before integration.

The browser-based telemetry dashboard was developed in parallel with the firmware telemetry layer, using the agreed JSON schema as the contract between the two. The dashboard was tested against a simulated serial stream before the full hardware loop was available, ensuring that the visualization layer was independently verified and ready to connect the moment the Jetson's FastAPI WebSocket relay was operational.

This iterative, bench-verified approach mirrors the Road Monitoring AI subsystem's own incremental methodology (Section 3.1) — each subsystem treated correctness validation at each stage as a precondition for proceeding to the next, rather than attempting a single end-to-end integration pass. The result is a mechatronics platform where every individual layer has been confirmed to function correctly in isolation, and integration risk is limited to the interfaces between layers rather than the layers themselves.

3.2 System Design / Architecture

The final in-cabin AI subsystem is designed as a sequential perception pipeline that receives live frames from the driver-facing camera and converts them into processed driver-state indicators. The pipeline starts with camera frame input, followed by face detection, landmark extraction, drowsiness detection, emotion detection, and head-pose estimation. The outputs from these AI modules are then prepared for transfer to the Jetson-hosted API and WebSocket layer implemented by the software team. This architecture was selected because it separates perception from scoring: the AI subsystem detects driver-state signals, while the software subsystem uses these signals as inputs to the Driver Reliability Score.

The face detector is the first major processing stage because all other modules depend on the correct localization of the driver's face. The implementation uses OpenCV Haar cascade detection with grayscale conversion and histogram equalization before detection. Histogram equalization improves contrast and helps the detector work under different lighting conditions inside the vehicle. If multiple faces are detected, the system selects the largest detected face because the driver is expected to be the closest and most relevant person in the camera frame. The output of the face detector is a bounding box represented as ((x, y, w, h)), where (x) and (y) represent the top-left coordinate of the detected face region, and (w) and (h) represent its width and height.

After the face is detected, facial landmarks are extracted from the face region. These landmarks provide the geometric points required for eye analysis, mouth analysis, and head-pose estimation. The drowsiness module uses landmarks around the eyes and mouth, while the head-pose module uses landmarks such as the nose tip, chin, eye corners, and mouth corners. This design makes the system more interpretable because important decisions such as eye closure and face orientation are based on measurable geometric relationships rather than only on black-box model outputs.

The drowsiness detection module combines geometric analysis and learning-based classification. Eye Aspect Ratio is used to estimate eye openness from six eye landmarks. When the eyes are open, the vertical distances between eyelid landmarks are larger. When the eyes are closed, these vertical distances decrease and the EAR value becomes smaller. The EAR equation used in the system is:

$$EAR = \frac{\|p_2 - p_6\| + \|p_3 - p_5\|}{2\|p_1 - p_4\|}$$

The Mouth Aspect Ratio is used as a supporting indicator for yawning. It measures the relationship between vertical and horizontal mouth landmark distances. A high MAR value over multiple consecutive frames can indicate yawning behavior. The MAR equation used in the system is:

$$MAR = \frac{\|p_2 - p_8\| + \|p_3 - p_7\| + \|p_4 - p_6\|}{3\|p_1 - p_5\|}$$

The drowsiness module does not classify the driver as drowsy from a single low EAR frame because normal blinking can temporarily reduce the EAR value. Instead, the system uses temporal persistence, where eye closure must continue across multiple consecutive frames before a drowsiness event is produced. The module also uses a CNN-based eye-state classifier to support the geometric EAR signal, providing a second independent confirmation of eye closure before a drowsiness state is raised.

The emotion classification module uses a MobileNetV2-based eight-class FER+ emotion model. The eight raw output classes are: Neutral, Happiness, Surprise, Sadness, Anger, Disgust, Fear, and Contempt. For operational use within the Driver Reliability System, the output is simplified to three categories: Sad, Angry, and Neutral. Classes that do not map to Sad or Angry are collapsed to Neutral. This simplification was chosen because only emotional states associated with reduced driver control or elevated stress are considered relevant to the driver-safety monitoring objective.

The head-pose estimation module uses the solvePnP algorithm from OpenCV with a predefined set of 3D facial reference points and their 2D landmark correspondences. The module estimates yaw (left/right head turn), pitch (looking up or down), and roll (head tilt). Yaw and pitch are used to determine whether the driver is looking away from the forward direction. Head-pose is treated as a distraction indicator and contributes to the driver-state output, but it is not a standalone scoring input. The software team uses this signal together with the drowsiness and emotion outputs as part of the Driver Reliability Score.

The road monitoring AI sybsystem was responsible for simulating the work on CARLA. The pipeline begins with the CARLA simulator operating on the Town10HD_Opt map, where an ego vehicle

(vehicle.dodge.charger_2020) is equipped with a forward-facing RGB camera configured with a resolution of 640 × 384 pixels, a 90° field of view, and a mounting position of x = 0.6 m, z = 1.35 m, and pitch = −12°. The camera stream is processed by the YOLOPv2 model, loaded from the trained weights file (yolopv2.pt), which generates lane-line segmentation masks at regular intervals. These masks are subsequently analyzed by the LaneDeviationEstimator module to calculate a quantitative lane-deviation value representing the vehicle's lateral position relative to the detected lane markings.

Obstacle detection is performed using a hybrid approach that combines direct CARLA actor queries for dynamic objects, such as vehicles and pedestrians, with a filtered obstacle sensor used as a fallback mechanism for detecting static obstacles, including parked vehicles. Simultaneously, vehicle telemetry data, including timestamp, lane deviation, speed (km/h), acceleration (m/s^2), obstacle detection status, obstacle distance (m), and obstacle type, are collected and recorded in JSONL format at each simulation step. The same telemetry records can also be transmitted to a Jetson device through an optional WebSocket connection for real-time processing.

To ensure consistent evaluation across multiple scenarios, two auxiliary scripts, spawn_city_life.py and change_carla_weather.py, are used to independently configure traffic density, pedestrian activity, and weather conditions. This design allows the driving environment to be varied without requiring any modifications to the core AI processing pipeline.

The road monitoring AI subsystem was also responsible for the implementation part of integrating the lane deviation work on the jetson orin nano which consisted of five sequential processing stages applied to each camera frame. In the first stage, the IMX219 CSI camera captures a 1280x720 BGR frame via the GStreamer nvarguscamerasrc pipeline, which is delivered to the Python script through an OpenCV VideoCapture interface. In the second stage, the captured frame is preprocessed, letterboxed and resized to 384x640, converted to a float16 tensor, and passed to the TensorRT inference engine. The engine executes a forward pass of YOLOPv2 and returns segmentation masks for the drivable area and lane lines, along with detection head outputs. These segmentation masks are decoded and overlaid on the display frame for visual inspection. In the third stage, the raw 1280x720 BGR frame, not the resized inference input, is passed to the LaneDeviationEstimator, which applies HSV masking, histogram analysis, and peak detection to identify the yellow tape lane edges and compute a normalised lateral deviation score. The decision to run the estimator on the full-resolution raw frame rather than the resized inference input is deliberate: it preserves the maximum available pixel detail for colour analysis and ensures that the HSV calibration constants remain consistent at the native camera resolution regardless of what input dimensions the inference engine requires. In the fourth and final stage, the computed deviation score is written to a yolo_log and a recorded mkv video is saved with the FPS counter, yolopv2 overlays and the lane deviation score. Finally, the fifth stage consisted of building an ai interface module in order to correctly set up the AI data and send it to the software's server.

An important architectural property of this design is that the TensorRT engine and the lane deviation estimator are fully independent of each other. The engine could be replaced, retrained, or removed without affecting the deviation computation, and the deviation estimator could be retuned or replaced without touching the inference pipeline. The only shared resource between the two is the original camera frame, which is consumed by both in parallel within the same processing loop iteration.

The structured outputs produced by the AI pipeline are made available to the software subsystem through the Jetson-hosted WebSocket and REST API layer. The software subsystem then applies a comprehensive scoring architecture built around a clear separation between live scoring, local persistence, backend ingestion, and administrative analytics. The Android client is responsible for processing telemetry during the active driving session, because it has direct access to the ordered packet stream and must continue operating even when backend connectivity is unavailable. The

backend is responsible for receiving completed sessions, validating the uploaded structure, preserving the uploaded historical records, and presenting the results through protected administrator pages.

The data flow begins when structured telemetry is received from the external vehicle and AI subsystems. The Android client maps this input into domain-level telemetry values such as timestamp, speed, acceleration, lane deviation, drowsiness score, emotional-risk value, obstacle distance, and turn-signal state. These values are processed by the local scoring engine. The scoring engine updates the current Driver Reliability Score, records continuous penalties, detects safety events, detects escalation patterns, and stores score-history points. When the driving session ends, the completed session summary and its related artifacts are prepared for backend upload.

The Driver Reliability Score was designed as an explainable rule-based model rather than an opaque machine-learning score. This decision was made because the system is intended for driver-safety review, where administrators must be able to understand why a score changed. The scoring model therefore separates risk into three categories: continuous penalties, event penalties, and escalation penalties. Continuous penalties represent sustained risk signals over time, event penalties represent discrete unsafe behaviours, and escalation penalties represent repeated or combined risk patterns that are more serious than isolated events.

The following mathematical model summarizes the core scoring and analytics logic used by the software subsystem. The Android formulas describe local live-session scoring and session-summary generation. The backend formulas describe how Phase 2 aggregates uploaded historical data for administrator dashboards and analytics. In production, the backend does not recalculate the Android session score; it stores and analyzes the completed values supplied by the upload payload.

Driver Reliability Score

The driver reliability score is bounded between 0 and 100 and is computed after each telemetry packet (i) as:

$$S_i = min\big(100, max(0, 100 - C_i - E_i - G_i)\big)$$

S_i = driver reliability score after telemetry packet i

C_i = cumulative continuous penalty after packet i

E_i = cumulative event penalty after packet i

G_i = cumulative escalation penalty after packet i

The raw time difference between two telemetry packets is:

$$\delta_i = \frac{t_i - t_{i-1}}{1000}$$

where (t_i) and (t_{i-1}) are timestamps in milliseconds and (δ_i) is measured in seconds.

To prevent invalid or overly large time gaps from creating unfair penalties, the continuous-penalty time step is bounded as:

$$\Delta t_i = \begin{cases} 0, & if\ t\_\{i-1\} \le 0\ or\ t\_i \le t\_\{i-1\} \\ min(\delta\_i, 1), & if\ t\_i > t\_\{i-1\} \end{cases}$$

The continuous penalty for packet (i) is:

$$c_i = (F_i + L_i + B_i)\Delta t_i$$

and the cumulative continuous penalty is:

$$C_i = C_{i-1} + c_i$$

Equivalently:

$$C_i = \sum_{k=1}^{i} c_k$$

Temporal Smoothing

For any risk signal (x_i), exponential moving average smoothing is applied as:

$$\bar{x}_i = \bar{x}_{i-1} + \alpha(x_i - \bar{x}_{i-1})$$

$\alpha = 0.30$

Therefore:

$$\bar{x}_i = 0.70\bar{x}_{i-1} + 0.30x_i$$

Fatigue Continuous Penalty

The fatigue penalty uses the smoothed drowsiness score:

$$F_i = \begin{cases} 0.40\bar{d}_i, & if \bar{d}_i \geq 0.70 \\ 0, & if \bar{d}_i < 0.70 \end{cases}$$

$\bar{d}_i$ = smoothed drowsiness score

Lane Deviation Continuous Penalty

The absolute lane-deviation signal is:

$$l_i = |laneDeviation_i|$$

The smoothed lane-deviation signal is:

$$\bar{l}_i = 0.70\bar{l}_{i-1} + 0.30l_i$$

The lane deviation continuous penalty is:

$$L_i = \begin{cases} 0.35\bar{l}_i, & if \bar{l}_i \geq 0.40 \\ 0, & if \bar{l}_i < 0.40 \end{cases}$$

If the lane signal is treated as a normalized AI output, then ($l_i \in [0,1]$). If the implementation treats it as a physical lane-deviation measurement, then the threshold (0.40) must be interpreted in meters.

Speed Conversion

If speed is received in kilometers per hour, it is converted to meters per second as:

$$v_i = \frac{v^{km/h}_i}{3.6}$$

Speed-Based Deceleration

Because IMU acceleration can be noisy on the prototype vehicle, braking risk is calculated using speed-derived deceleration:

$$D_i = \begin{cases} max\left(\frac{v_{i-1} - v_i}{\delta_i}, 0\right), & if\ \delta\_i > 0 \\ 0, & if\ \delta\_i \leq 0 \end{cases}$$

D_i = estimated deceleration in m/s^2

v_i = vehicle speed at packet i in m/s

v_{i-1} = vehicle speed at packet i−1 in m/s

Braking Aggression

The harsh-braking threshold is:

$$\theta\_b = 3.5$$

The normalized braking aggression value is:

$$A_i = \begin{cases} \frac{D_i - \theta\_b}{\theta\_b}, & if\ D\_i > \theta\_b \\ 0, & if\ D\_i \leq \theta\_b \end{cases}$$

Substituting (θ_b = 3.5):

$$A_i = \begin{cases} \frac{D_i - 3.5}{3.5}, & if\ D\_i > 3.5 \\ 0, & if\ D\_i \leq 3.5 \end{cases}$$

Equivalent compact form:

$$A_i = max\left(\frac{D_i - 3.5}{3.5}, 0\right)$$

The smoothed braking aggression is:

$$\bar{A}_i = 0.70\bar{A}_{i-1} + 0.30A_i$$

The braking continuous penalty is:

$$B_i = 0.25\bar{A}_i$$

Full Continuous Penalty Formula

The full continuous penalty per packet is:

$$c_i = (F_i + L_i + B_i)\Delta t_i$$

Event Penalty

The cumulative event penalty is:

$$E = 5.0N_f + 4.0N_l + 3.5N_b + 4.5N_e$$

N^f = number of high-fatigue episodes

N_l = number of lane-departure events

N^b = number of harsh-braking events

N_e = number of emotional-distress episodes

A high-fatigue event is detected when $d_i \geq 0.70$ for at least $T^f \geq 5$ consecutive frames, with severity $\sigma^f = d_i$.

A lane-departure event is detected when $l_i \geq 0.40$ for at least $T_l \geq 2$ consecutive frames, with severity $\sigma_l = l_i$.

A harsh-braking event is detected when $D_i \geq 3.5$, with severity $\sigma^b = D_i$.

An emotional-distress event is detected when $e_i \geq 0.70$ for at least $T_e \geq 4$ consecutive frames, with severity $\sigma_e = e_i$.

Escalation Penalty

The cumulative escalation penalty is:

$$G = 7.0M_r + 8.5M_{fl} + 6.5M_{ld} + 9.0M_m$$

M_r = number of repeated harsh-braking escalations

M^f_l = number of fatigue-with-lane-deviation escalations

M_l^d = number of prolonged lane-drift escalations

M_m = number of multiple-events-short-window escalations

A repeated harsh-braking escalation is detected when $N^b(15s) \geq 3$, with stored multiplier $m_r = 1.5$.

A fatigue-with-lane-deviation escalation is detected when $A^f = 1 \wedge A_l = 1$, where A^f is the fatigue active flag and A_l is the lane-deviation active flag, with stored multiplier $m^f_l = 2.0$.

A prolonged lane-drift escalation is detected when $A_l = 1$, $T_l \geq 2$, and $l_i \geq 0.40$, with stored multiplier $m_l^d = 1.75$.

A multiple-events-short-window escalation is detected when $N_{all}(15s) \geq 3$, with stored multiplier $m_m = 2.5$.

Distance Calculation

The distance increment for telemetry packet (i) is:

$$\Delta D_i = \frac{v_i \delta_i}{1000}$$

The total session distance is:

$$D_{total} = \sum_{i=1}^{n} \Delta D_i$$

where (ΔD_i) is measured in kilometers.

$$T_{session} = \frac{max(t_{end} - t_{start}, 0)}{1000}$$

where (t_{start}) and (t_{en}^d) are measured in milliseconds and ($T_{session}$) is measured in seconds.

Long-Term Driver Reliability

For a driver with (n) completed sessions, the intended long-term reliability score is:

$$R_{long} = \begin{cases} \frac{1}{n}\sum_{j=1}^{n} S_j, & if\ n > 0 \\ 0, & if\ n = 0 \end{cases}$$

S_j = final score of completed session j

The total driver distance is:

$$D_{driver} = \sum_{j=1}^{n} D_j$$

where (D_j) is the total distance of session (j).

Backend Average Final Score

For backend analytics, the average final score is:

$$\bar{S} = \begin{cases} \frac{1}{n}\sum_{j=1}^{n} S_j, & if\ n > 0 \\ 0, & if\ n = 0 \end{cases}$$

where (S_j) is the stored final score of session (j). Phase 2 does not recompute (S_j); it reads the uploaded final score stored for each completed session.

The backend total distance is:

$$D_{fleet} = \sum_{j=1}^{n} D_j$$

where (D_j) is the stored uploaded distance of session (j).

Daily Score Trend

For a given day (q), the daily average score is:

$$\bar{S}_q = \begin{cases} \frac{1}{n_q}\sum_{j=1}^{n_q} S_{j,q}, & if\ n_q > 0 \\ 0, & if\ n_q = 0 \end{cases}$$

where (n_q) is the number of sessions grouped into day (q).

Count-Based Analytics

For a selected category (k), the count is:

$$N_k = \sum_{j=1}^{n} 1(category\_j = k)$$

$$1(condition) = \begin{cases} 1, & if\ the\ condition\ is\ true \\ 0, & otherwise \end{cases}$$

This applies to driver status counts, vehicle status counts, session status counts, session validity counts, event-type counts, and escalation-type counts.

Dashboard Percentage Rates

For any dashboard rate, the general form is:

$$Rate = \begin{cases} \frac{N_{selected} \times 100}{N_{total}}, & if\ N_{total} > 0 \\ 0, & if\ N_{total} = 0 \end{cases}$$

Therefore:

$$invalidSessionRate = \begin{cases} \frac{invalidSessionCount \times 100}{totalSessions}, & if\ totalSessions > 0 \\ 0, & if\ totalSessions = 0 \end{cases}$$

$$warningSessionRate = \begin{cases} \frac{processedWithWarningsSessionCount \times 100}{totalSessions}, & if\ totalSessions > 0 \\ 0, & if\ totalSessions = 0 \end{cases}$$

$$completedSessionRate = \begin{cases} \frac{completedSessionCount \times 100}{totalSessions}, & if\ totalSessions > 0 \\ 0, & if\ totalSessions = 0 \end{cases}$$

$$activeDriverShare = \begin{cases} \frac{activeDriverCount \times 100}{totalDrivers}, & if\ totalDrivers > 0 \\ 0, & if\ totalDrivers = 0 \end{cases}$$

$$busyVehicleShare = \begin{cases} \frac{busyVehicleCount \times 100}{totalVehicles}, & if\ totalVehicles > 0 \\ 0, & if\ totalVehicles = 0 \end{cases}$$

Backend Driver Aggregate Update

After an accepted upload, the backend should update driver aggregate fields as:

$$totalSessions_{new} = totalSessions_{old} + 1$$

$$totalDistanceKm_{new} = totalDistanceKm_{old} + D_{uploaded}$$

The intended long-term reliability update should be:

$$R_{long,new} = \frac{R_{long,old} \cdot totalSessions_{old} + S_{uploaded}}{totalSessions_{old} + 1}$$

$S_{tploa}{}^{d}{}_{e}{}^{d}$ = uploaded final session score

$D_{tploa}{}^{d}{}_{e}{}^{d}$ = uploaded session distance

The backend must not recompute the uploaded session final score, penalty totals, duration, distance, events, escalations, or score history during ingestion. Those values are produced by the Android scoring pipeline and preserved by the backend as historical truth.

These formulas define the relationship between telemetry, risk classification, session scoring, and backend analytics. The continuous-penalty terms make the score sensitive to sustained unsafe behaviour without overreacting to a single noisy packet. The time-step cap prevents abnormal timestamp gaps from producing unfairly large penalties. The exponential moving average reduces sensitivity to short-term telemetry noise, which is important because AI and vehicle signals can fluctuate from frame to frame.

During implementation, the scoring weights, thresholds, and configuration values were not treated as fixed theoretical constants. They were adjusted through experimentation because the system was tested on a scaled RC car rather than a full-sized commercial vehicle. The physical prototype produced different speed, acceleration, braking, steering, and lane-deviation ranges from those expected in a real vehicle. As a result, applying the original configuration values directly would have made the Driver Reliability Score either too harsh or too insensitive. The final configuration was therefore tuned experimentally so that the scoring system behaved as intended on the prototype: normal RC-car movement did not cause excessive penalties, while clear unsafe behaviours such as harsh braking, sustained lane deviation, fatigue indicators, and repeated risky events still produced meaningful score

reductions. This calibration changed the numerical sensitivity of the scoring model, but it did not change the deterministic structure of the scoring equations or the separation between continuous penalties, event penalties, escalation penalties, and backend analytics.

The event-penalty and escalation-penalty formulas provide a second layer of explanation beyond continuous risk. Events identify specific unsafe behaviours, while escalations identify repeated or combined behaviours that require stronger penalties. This makes the score more interpretable because a final score can be explained through accumulated continuous risk, detected events, and escalation patterns rather than a single unexplained value.

The backend analytics formulas serve a different purpose from the Android scoring formulas. They do not generate the original session score. Instead, they summarize stored completed sessions through averages, totals, category counts, daily trends, and dashboard rates. This preserves the responsibility boundary between the two software phases: the Android client produces completed scored sessions, while the backend validates, stores, aggregates, and presents those sessions for administrator review.

This architecture supports traceability because the final driver reliability record is not only a number. It is supported by penalty totals, event records, escalation records, score-history points, session validity, upload-processing status, and backend analytics. As a result, the software subsystem can provide both immediate scoring during a session and long-term historical review after upload.

The Mechatronics subsystem implements a multi-layered, dual-controller architecture that separates real-time, time-critical control from high-level computation. A real-time ESP32 microcontroller owns everything that must happen deterministically motor and servo PWM generation, IMU sampling, Hall-effect pulse counting, telemetry packaging, and a hardware fail-safe — while the non-real-time Jetson Orin Nano hosts the AI pipeline, logs ground truth, and consumes telemetry. The two communicate over a USB serial link (USB-C cable, 115200 baud), which is the primary integration surface with the AI and Software departments.

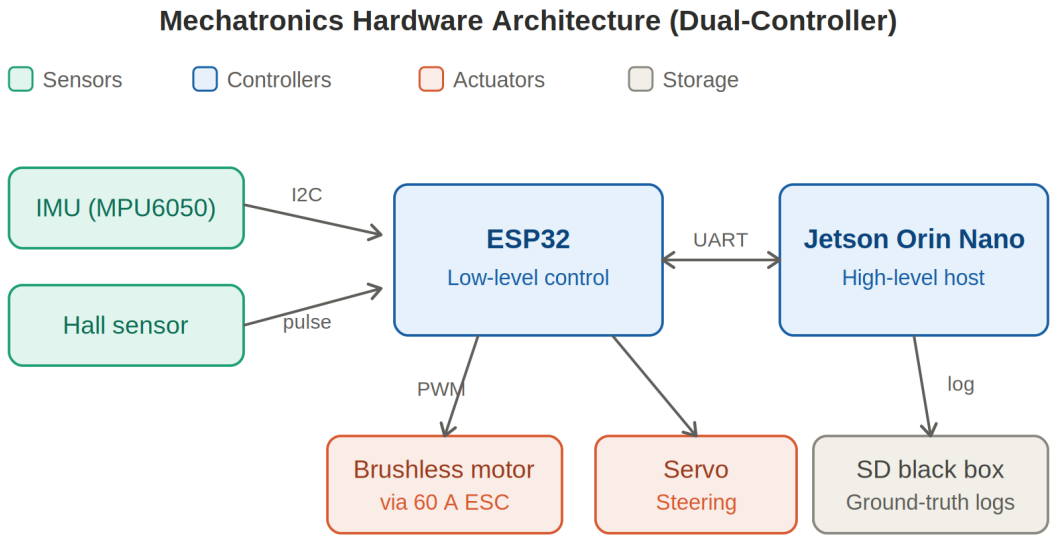

Figure 1Mechatronics hardware architecture (dual-controller)

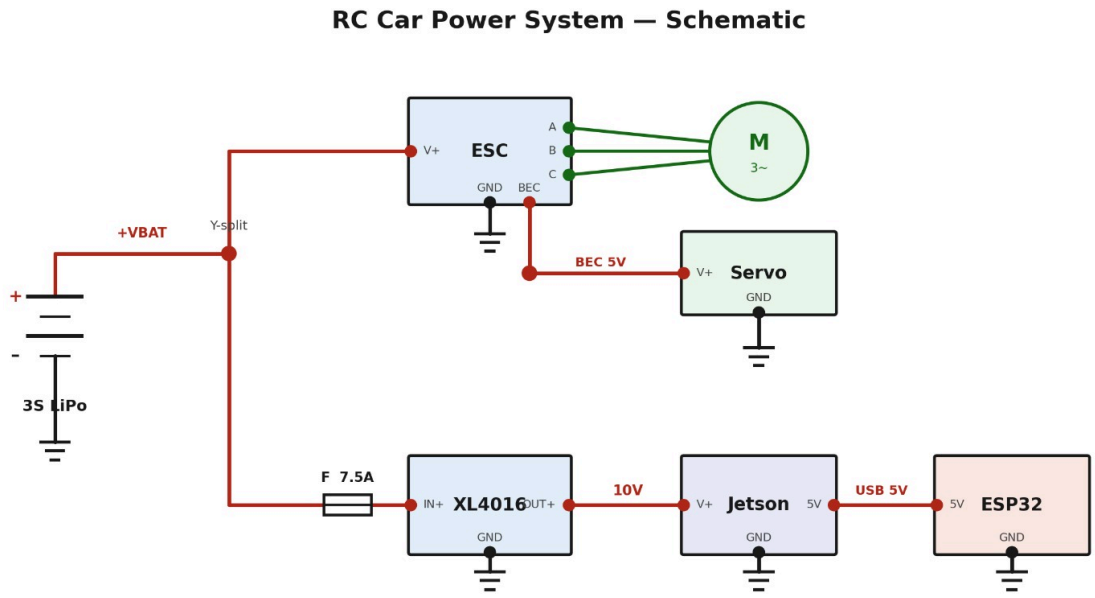

Figure 2 Car power System

The ESP32 DOIT DevKit V1 (ESP32-D0WD-V3) pin assignments are fixed and share no conflicts with the BLE stack, I^2C peripheral, or LEDC PWM channels:

PIN Name	PIN NO	
SERVO_PIN	15	MG995 steering servo (PWM, 50 Hz LEDC)
ESC_PIN	12	60 A ESC signal (PWM, 50 Hz LEDC)
VPIN	34	Battery voltage sense (ADC input, 12-bit)
SDA_PIN	21	I2C data — MPU6050
SCL_PIN	22	I2C clock — MPU6050
HALL_PIN	23	US1881 Hall-effect interrupt input (INPUT_PULLUP, CHANGE mode)

GPIO 34 is configured as input-only (no internal pull-up). GPIO 23 is configured INPUT_PULLUP and attached to the external interrupt handler in CHANGE mode to capture all four magnet transitions per wheel revolution.

Both the steering servo and the ESC receive 50 Hz PWM signals generated by the ESP32's LEDC peripheral at 16-bit resolution. A 50 Hz period corresponds to a 20,000 µs frame, giving the following duty-cycle conversion:

$$duty = (pulse\_width\_us / 20000) \times MAX\_DUTY$$

where MAX_DUTY = $2^{16} - 1 = 65535$, giving sub-microsecond pulse resolution. The same formula applies to both servo and ESC channels.

Code implementation:

```
// 16-bit duty-cycle from pulse-width in microseconds
uint32_t usToDuty(int us) {
  return (uint32_t)((float)us / 20000.0f * MAX_DUTY);
}
void writeServoUs(int us) {
  lastServoUs = us;
  ledcWrite(SERVO_PIN, usToDuty(us));
}
void writeEscUs(int us) {
  ledcWrite(ESC_PIN, usToDuty(us));
}
```

The ledcAttach() call binds each GPIO to its own LEDC timer channel at 50 Hz. Both channels are initialised before armESC() so that a clean neutral pulse is present on the ESC signal wire from the moment the peripheral is powered.

ESC Throttle, Brake, and Reverse

The 60 A WSDT-60A ESC operates in single-click forward/reverse mode. This was confirmed on the bench: a 1300 µs test command issued while the motor was spinning forward produced immediate reverse rotation rather than active braking. Consequently, "braking" in this design means commanding reverse against forward momentum rather than relying on an ESC-internal brake circuit.

The usable pulse bands are:

$$ESC\_NEUTRAL\_US = 1500\,\mu s \quad (idle\ /\ coast)$$
$$ESC\_FWD\_SPIN\_US = 1540\,\mu s \quad (forward\ break - loose\ point)$$
$$ESC\_FWD\_MAX\_US = 1600\,\mu s \quad (top\ forward\ speed)$$
$$ESC\_REV\_SPIN\_US = 1460\,\mu s \quad (reverse\ break - loose\ point)$$
$$ESC\_REV\_MAX\_US = 1000\,\mu s \quad (maximum\ reverse\ /\ hardest\ brake)$$

Ramp time across the forward band (forward direction as example):

$$usable\_band = ESC\_FWD\_MAX\_US - ESC\_FWD\_SPIN\_US = 1600 - 1540 = 60\,\mu s$$
$$t\_target = (usable\_band\ /\ ACCEL\_STEP) \times RAMP\_INTERVAL = (60\ /\ 3) \times 15\,ms = 300\,ms$$

The throttle decision function implements four distinct cases:

- LT pressed (reverse/brake): trigger pressure maps proportionally to pulse depth, applied instantly by overwriting currentThrottle without entering the ramp gate.
- RT pressed, moving backward: ramps up at BRAKE_STEP until neutral, then continues to forward target.
- RT pressed, at or above neutral: ramps up at ACCEL_STEP; a dead-zone kick snaps currentThrottle to ESC_FWD_SPIN_US on the first forward-direction tick.
- Neither trigger: coasts to neutral at COAST_STEP.

The key design insight in Case 1 is that brake depth must not depend on currentThrottle. When the driver lifts RT and then presses LT, the ramp coasts currentThrottle back to neutral in ≈75 ms while the car is still physically rolling forward. If the brake branch tested currentThrottle > NEUTRAL, it would find the condition false and fall into the gentle coast path. By snapping to the reverse pulse directly and proportionally to trigger pressure, the brake responds with the same latency whether the command has already coasted or not.

Code implementation:

```
int decideThrottle(int rt, int lt, int *stepOut) {


  // CASE 1 — REVERSE / BRAKE (LT): proportional, applied instantly.
  // Brake depth scales with trigger pressure; no dependency on
  // currentThrottle so it bites even when the command has already
  // coasted to neutral while the car is still rolling forward.
  if (lt > TRIG_DEAD_ZONE) {
    int rev = map(lt, TRIG_DEAD_ZONE, TRIG_MAX,
                  ESC_REV_SPIN_US, ESC_REV_MAX_US);
    currentThrottle = rev;   // snap — bypass the ramp
    *stepOut = REV_ACCEL_STEP;
    return rev;
  }


  // CASE 2 — FORWARD (RT): smoothly ramped.
  if (rt > TRIG_DEAD_ZONE) {
```

```
    int target = map(rt, TRIG_DEAD_ZONE, TRIG_MAX,
                     ESC_FWD_SPIN_US, ESC_FWD_MAX_US);
    if (currentThrottle < ESC_NEUTRAL_US) {
      *stepOut = BRAKE_STEP;   // still rolling backward -> brake up
    } else {
      *stepOut = ACCEL_STEP;
      if (currentThrottle < ESC_FWD_SPIN_US)   // kick past dead zone
        currentThrottle = ESC_FWD_SPIN_US;
    }
    return target;
  }

  // CASE 3 — COAST: release to neutral at COAST_STEP.
  *stepOut = COAST_STEP;
  return ESC_NEUTRAL_US;
}
```

Steering Control

The left joystick horizontal axis (joyLHori, range 0–65535) is mapped to a servo pulse in the range [SERVO_MIN_US, SERVO_MAX_US] = [544, 2400] µs. A direction-inversion flag (STEER_REVERSED) was confirmed during bench testing and corrects for the physical mounting orientation of the servo linkage:

$$us = SERVO\_MAX\_US - [(joyLHori / 65535) \times (SERVO\_MAX\_US - SERVO\_MIN\_US)] \quad (STEER\_REVERSED = true)$$

Code implementation (direct mode):

```
void controlSteering(int rt, int lt) {
  // Direct joystick mode
  uint16_t raw = xboxController.xboxNotif.joyLHori;  // 0..65535
```

```
  int us = STEER_REVERSED
      ? map(raw, 0, 65535, SERVO_MAX_US, SERVO_MIN_US)
      : map(raw, 0, 65535, SERVO_MIN_US, SERVO_MAX_US);
  writeServoUs(us);
}
```

Closed-Loop Heading Assist (Proportional Controller)

A supplementary proportional heading-assist controller is implemented and toggled by the LB button during forward driving. It holds a target heading derived from IMU yaw, updated by the joystick at a commanded turn rate, and closes the loop via the servo:

$$\theta\_target, i \;=\; \theta\_target, i-1 \;+\; k\_cmd \;\cdot\; u\_stick \;\cdot\; \omega\_max \;\cdot\; \Delta t$$

$$e\_i \;=\; \theta\_target, i \;-\; \psi\_i$$

$$\Delta u\_i \;=\; K\_p \;\cdot\; e\_i$$

$$\Delta u\_i \;=\; clamp(\Delta u\_i, -\Delta u\_max, +\Delta u\_max)$$

$$PW\_i \;=\; PW\_center \;+\; s \;\cdot\; \Delta u\_i$$

where ψ_i is the current yaw from the IMU, K_p = 12.0, ω_max = 90 °/s, PW_center = STEER_CENTER_US = 1472 µs, Δu_max = STEER_MAX_DELTA = 350, and s = STEER_SIGN = +1. The controller is only active while RT is pressed and LT is released, so manual braking/reverse always exits assist mode cleanly.

Code implementation:

```
// Heading-assist P-controller (toggled by LB button)
void controlSteering(int rt, int lt) {
  if (millis() - lastCtrl < CTRL_INTERVAL) return;
  lastCtrl = millis();
  const float dt = CTRL_INTERVAL / 1000.0f;

  bool fwdDriving = (rt > TRIG_DEAD_ZONE) && (lt <= TRIG_DEAD_ZONE);
  bool useAssist  = assistOn && readyToDrive && fwdDriving;
```

```
    if (useAssist) {
      if (!headingActive) { targetHeading = carYaw; headingActive = true; }
      float stickNorm = ((int)xboxController.xboxNotif.joyLHori - 32768) / 32768.0f;
      if (fabs(stickNorm) < STICK_DEADBAND) stickNorm = 0;
      targetHeading += TURN_CMD_SIGN * stickNorm * MAX_TURN_RATE * dt;

      float err   = targetHeading - carYaw;
      float delta = KP_HEADING * err;
      delta = constrain(delta, -(float)STEER_MAX_DELTA, (float)STEER_MAX_DELTA);
      int us = STEER_CENTER_US + (int)(STEER_SIGN * delta);
      writeServoUs(clampi(us, SERVO_MIN_US, SERVO_MAX_US));
    } else {
      headingActive = false;
      uint16_t raw = xboxController.xboxNotif.joyLHori;
      int us = STEER_REVERSED
          ? map(raw, 0, 65535, SERVO_MAX_US, SERVO_MIN_US)
          : map(raw, 0, 65535, SERVO_MIN_US, SERVO_MAX_US);
      writeServoUs(us);
    }
}
```

IMU Acquisition (MPU6050)

The MPU6050 is read over I^2C at SDA = GPIO 21, SCL = GPIO 22, I^2C address 0x68 (AD0 tied to GND). The MPU6050_tockn library provides a built-in complementary filter that fuses accelerometer and gyroscope data into drift-free roll and pitch angles. Yaw is gyroscope-only and drifts in the absence of a magnetometer.

At startup, calcGyroOffsets(true) is called with the car held stationary, measuring the gyro's resting bias over approximately 3 seconds and subtracting it from all subsequent readings. A separate resting-offset correction is applied to the fore/aft acceleration channel to remove the gravity leakage component caused by the chassis not sitting perfectly level:

$$a\_fwd \;=\; k\_sign \;\cdot\; (a\_x,raw \;-\; a\_x,offset) \;\cdot\; g \quad where\ g \;=\; 9.81\ m/s^2$$

Axis mapping for this chassis mounting (confirmed by empirical tilt test):

$$pitch\ (nose\ up/down) \;=\; -AngleY \qquad (sign\ corrected\ so\ nose-up\ is\ positive)$$
$$roll\ \ (lean) \qquad = \;+AngleX$$
$$yaw\ \ \ (heading) \qquad = \;+AngleZ \qquad (gyro-only, drifts)$$
$$a\_fwd \qquad\quad = \;ACCEL\_SIGN \;\times\; getAccX() \;\times\; 9.81$$

The IMU read is non-blocking: updateIMU() is called every loop iteration. The MPU6050_tockn library's update() function handles its own internal timing and is safe to call at loop speed. No delay() is introduced into the main loop by the IMU read path.

Code implementation:

```
#define ACCEL_SIGN  (-1)   // flip to +1 if forward acceleration reads negative


void updateIMU() {
  mpu6050.update();
  carPitch    = -mpu6050.getAngleY();              // nose-up positive
  carRoll     =  mpu6050.getAngleX();
  carYaw      =  mpu6050.getAngleZ();              // gyro-only, drifts
  accelFwdMps2 =  ACCEL_SIGN * mpu6050.getAccX() * 9.81f; // resting offset
}


// Resting-offset capture (run once at startup, car held STILL)
// Called inside setup() after calcGyroOffsets():
//   float accXrest = 0;
//   for (int i=0; i<200; i++) { mpu6050.update(); accXrest += mpu6050.getAccX(); delay(5); }
//   accXrest /= 200.0f;   // stored as ACCEL_X_REST_OFFSET
```

Wheel-Speed Sensing (US1881 Hall-Effect Sensor)

Four magnets are mounted in alternating N-S-N-S polarity on a 60 mm diameter circle on the wheel hub. The wheel outer diameter is 130 mm. The US1881 is a latching Hall-effect sensor: it toggles its output state on each magnet transition. CHANGE-mode interrupts on GPIO 23 therefore capture all four transitions per revolution, giving twice the resolution of FALLING-only triggering:

$$C = \pi \times d\_wheel = \pi \times 0.130 \approx 0.4084\, m$$

Speed is computed in a 500 ms windowed calculation to smooth transient noise:

$$n\_rev = N\_pulses \,/\, N\_pulses\_per\_rev = N\_pulses \,/\, 4$$
$$d = n\_rev \times C$$
$$v = d \,/\, \Delta t$$
$$v\_km/h = v \times 3.6$$

The ISR is placed in IRAM (IRAM_ATTR) to guarantee execution even when the flash cache is busy. pulseCount is declared volatile and read with interrupts disabled to prevent a torn read mid-update. The Hall-effect sensor cannot distinguish forward from reverse rotation; during reverse driving, pulses continue to accumulate and speedKmh reports a positive value, which is a known limitation documented in the scope section.

Code implementation:

```
volatile unsigned long pulseCount = 0;


void IRAM_ATTR onHall() { pulseCount++; }   // ISR — runs on every magnet edge


void updateSpeed() {
  unsigned long now = millis();
  if (now - lastSpeedCalc < SPEED_INTERVAL) return;   // 500 ms window


  noInterrupts();
  unsigned long count = pulseCount;
```

```
  interrupts();

  unsigned long pulses = count - lastSpeedCount;
  float dt           = (now - lastSpeedCalc) / 1000.0f;
  lastSpeedCount     = count;
  lastSpeedCalc      = now;

  float revs   = (float)pulses / PULSES_PER_REV;       // 4 pulses/rev
  float meters = revs * WHEEL_CIRCUMFERENCE_M;         // pi * 0.130 m
  float mps    = meters / dt;
  speedKmh     = mps * 3.6f;
}
```

Jetson Telemetry — USB Serial JSON Output

The ESP32 transmits one JSON object per line over its USB-to-serial bridge at 115200 baud, 20 Hz (TELEM_INTERVAL = 50 ms). The Jetson reads the port as /dev/ttyUSB0 or /dev/ttyACM0. When DEBUG_HUMAN is false, the serial port carries only clean JSON so the FastAPI parser receives no human-readable startup messages that would cause a parse error.

The commanded steering angle is derived from the last servo pulse written, converting pulse width back to degrees:

$$\theta\_steer \;=\; [(PW\_last \;-\; PW\_center) \,/\, \Delta PW\_max] \;\times\; \theta\_full\ lock$$

where PW_center = STEER_CENTER_US = 1472 µs, ΔPW_max = STEER_MAX_DELTA = 350, and θ_full lock = STEER_FULL_DEG = 79.5°. The stationary flag is set true when speedKmh ≤ 0.5 km/h.

Code implementation:

```
void sendTelemetry() {
  unsigned long now = millis();
  if (now - lastTelem < TELEM_INTERVAL) return;   // 50 ms = 20 Hz
  lastTelem = now;
```

```
  float steerDeg = (float)(lastServoUs - STEER_CENTER_US)
                 / (float)STEER_MAX_DELTA * STEER_FULL_DEG;
  bool stationary = (fabs(speedKmh) <= 0.5f);

  Serial.print('{"speedKmh":');
  Serial.print(speedKmh, 2);
  Serial.print(',"steeringAngleDeg":');
  Serial.print(steerDeg, 1);
  Serial.print(',"accelerationMps2":');
  Serial.print(accelFwdMps2, 2);
  Serial.print(',"obstacleDistanceMeters":');
  Serial.print(NO_OBSTACLE_M, 1);
  Serial.print(',"turnSignal":false');
  Serial.print(',"ignitionOn":true');
  Serial.print(',"stationary":');
  Serial.print(stationary ? 'true' : 'false');
  Serial.println('}');
}
```

Example JSON packet transmitted to the Jetson:

```
{
  "speedKmh":         3.20,
  "steeringAngleDeg":  12.5,
  "accelerationMps2":  1.80,
  "obstacleDistanceMeters": 999.0,
```

```
  "turnSignal":      false,
  "ignitionOn":      true,
  "stationary":      false
}
```

This packet structure matches, field-for-field, the ESP32 JSON contract expected by the Jetson FastAPI server, allowing the telemetry to be consumed directly by the software scoring pipeline described in Section 3.2 of the Software department's methodology, without further field-name translation.

Safe-Start Gate and Controller Fail-Safe

Two independent safety mechanisms are implemented in firmware:

- Safe-start gate: after the Xbox controller connects over BLE, the readyToDrive flag remains false and the ESC is held at neutral until both RT and LT are confirmed below TRIG_DEAD_ZONE. This prevents a motor launch if either trigger is held during the pairing handshake. The gate re-arms automatically on every reconnection.
- Disconnect fail-safe: if xboxController.isConnected() returns false for more than DISCONNECT_LIMIT = 200 consecutive loop iterations (≈75 ms at loop speed), currentThrottle and the ESC output are forced to ESC_NEUTRAL_US. The readyToDrive flag is also cleared so the safe-start gate must be satisfied again on reconnection. The neutral target ensures a dropout commands coast rather than a locked reverse command.

The ESC arming sequence also contributes to safety: writeEscUs(ESC_NEUTRAL_US) is called at the start of armESC() and held for 3000 ms before any other pulse is generated. This satisfies the ESC's power-on neutral-detection requirement and ensures that reverse pulses, which would confuse the arming routine, are structurally impossible during the arming window.

The firmware loop runs without any blocking delays after setup(). Five concurrent tasks are multiplexed using independent millis()-based timers:

- xboxController.onLoop() — called every iteration for minimum BLE input latency.
- updateIMU() — MPU6050 update called every iteration (library manages its own timing).
- updateSpeed() — 500 ms windowed pulse-count calculation.
- sendTelemetry() — 50 ms (20 Hz) JSON serial output.
- Throttle ramp — 15 ms timer; ramps currentThrottle toward the target decided by decideThrottle(), then writes the clamped result to the ESC.

Steering is updated every loop iteration (not timer-gated) to achieve the lowest possible latency between joystick movement and servo response. The clampi() guard ensures currentThrottle can never exceed [ESC_REV_MAX_US, ESC_FWD_MAX_US] regardless of which path set it, providing a belt-and-suspenders bound against runaway commands.

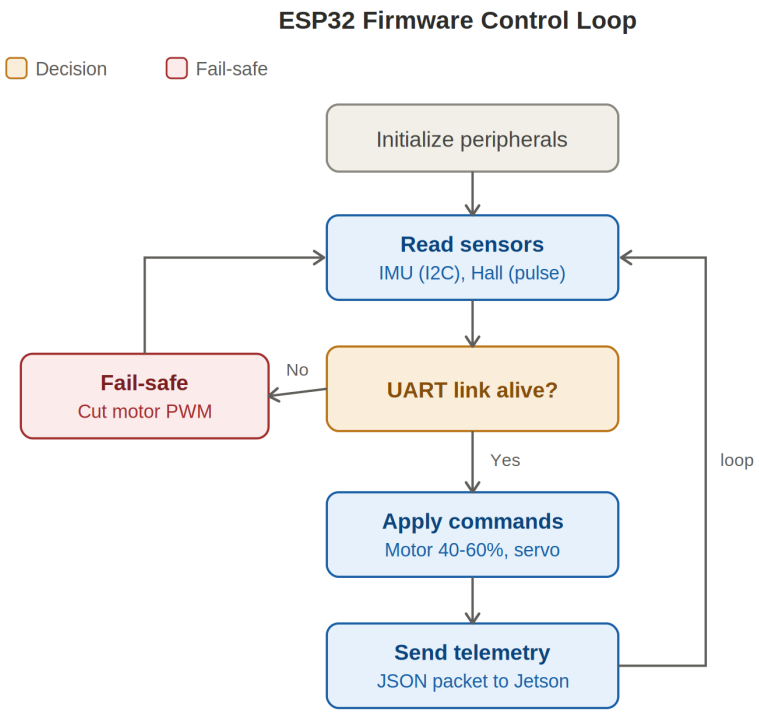

Figure 3 ESP32 firmware control loop

3.3 Tools, Technologies, and Standards

TheAI system was implemented using Python because Python provides strong support for computer vision, deep learning inference, and rapid prototype development.

The main tools and technologies used in the AI system are:

Python: Main programming language used for the AI pipeline.

OpenCV: Used for camera frame processing, Haar cascade face detection, image preprocessing, and solvePnP-based head-pose estimation, Frame capture, HSV processing, display, video recording

MediaPipe Face Landmarker: Used to extract facial landmarks required for drowsiness and head-pose estimation.

NumPy and SciPy: Used for numerical calculations, Euclidean distance, array processing, and geometric feature extraction.

ONNX Runtime: Used to run ONNX-based deep learning models and intermediate model representation for TRT conversion.

TensorRT: Used for optimized inference on NVIDIA Jetson hardware where supported.

MobileNetV2: Used as the lightweight CNN architecture for eye-state and emotion-related classification.

NVIDIA Jetson Orin Nano: Used as the edge deployment platform.

JSON: Used as the structured format for AI output values.

WebSocket and API host: Hosted on the Jetson Orin Nano by the software team for communication between the AI outputs and the software scoring system.

PyTorch: Model loading, GPU inference, ONNX export.

YOLOPv2: Multi-task road perception for lane detection.

IMX219: Downward-facing lane camera.

CARLA 0.916: Open-source simulator.

The main technology selection rationale for the AI subsystem is real-time performance and deployability. OpenCV and MediaPipe support fast computer vision processing. MobileNetV2 provides efficient deep learning inference on edge devices. ONNX and TensorRT support model portability and acceleration on NVIDIA hardware. JSON-based outputs simplify communication with the software subsystem.

The software subsystem consumed these AI outputs through the Jetson-hosted WebSocket and API layer, and was built using a separate technology stack selected to support local scoring, offline-first persistence, structured telemetry processing, authenticated backend ingestion, persistent historical storage, protected administration, and analytics.

The Phase 1 Android client was implemented primarily in Kotlin. Kotlin was selected because it is the standard modern language for Android development and provides strong support for null safety, coroutines, data classes, sealed types, and concise domain modelling. These features are useful in a safety-oriented application where telemetry values, session state, and scoring results must be represented clearly and reliably.

Android Studio was used as the main development environment for the mobile client. Gradle was used as the build system, with the Android Gradle Plugin and Kotlin build tooling managing compilation, dependencies, test execution, and APK generation. Kotlin Symbol Processing was used where required by Android libraries, and the Gradle version catalog helped keep dependency declarations organized.

Jetpack Compose was used for the Android user interface. Compose was selected because it supports declarative UI development and works naturally with ViewModels, StateFlow, and reactive state updates. This was important for screens that depend on changing driver state, session state, history records, settings, and backend/admin data. Material3 components were used to maintain a consistent Android interface style.

The Android client follows an MVVM-oriented structure. ViewModels are used to expose screen state and events to Compose screens, while domain and data layers remain separated from UI code. Kotlin coroutines, Flow, StateFlow, and SharedFlow were used for asynchronous behaviour, including telemetry updates, repository operations, UI state changes, and synchronization-related workflows. This asynchronous model was necessary because telemetry, database access, and network requests cannot safely block the user interface.

Room was used as the Android persistence library over SQLite. Room was selected because it provides structured local storage, compile-time query checking, DAO-based database access, and integration with Kotlin coroutines and Flow. The local database stores driver records, session summaries, safety events, escalation records, score-history points, vehicles, and synchronization metadata. This supports the offline-first requirement of the Android client.

OkHttp was used for lower-level network communication and WebSocket handling. WebSocket communication is required because telemetry arrives as a real-time stream rather than as isolated manual requests. Retrofit was used for HTTP-based backend API contracts because it provides clear service interfaces and structured request/response DTO handling. JSON was used as the transport format because it is lightweight, human-readable, and compatible with Android, Spring Boot, and the Jetson-side telemetry interface.

The Phase 2 backend-admin system was implemented in Java using Spring Boot. Java was selected because it is stable, widely supported, and suitable for structured backend systems. Spring Boot was selected because it provides mature support for REST APIs, web controllers, dependency injection, security, persistence, validation, configuration, and testing. Gradle was also used for backend build and dependency management.

Spring MVC was used to implement REST controllers and server-rendered page controllers. REST controllers support machine-facing and client-facing API workflows, such as completed-session upload. Page controllers support the administrator web interface. Thymeleaf was used for server-rendered administrator pages because it integrates directly with Spring MVC and allows backend-generated view models to be displayed without requiring a separate frontend framework.

Spring Security was used for authentication and access control. Session-based authentication protects administrator pages and protected routes, while API-client authentication protects the completed-session upload endpoint. BCrypt was used for password hashing so that plain-text passwords are not stored. This security structure separates human administrator access from machine-client upload access.

Spring Data JPA and Hibernate were used for backend persistence. These tools were selected because they provide repository abstractions, entity mapping, transaction support, and integration with relational databases. MySQL was used as the backend database because it is a mature relational database suitable for persistent fleet, driver, vehicle, session, event, escalation, score-history, and ingestion-issue records.

Jackson was used for JSON serialization and deserialization in the backend. DTOs were used to separate transport data from domain and persistence models. This is important because upload payloads, admin responses, database entities, and domain objects have different responsibilities and should not be treated as the same object.

The backend uses a layered architecture with domain models, persistence entities, repositories, mappers, validation services, command services, query services, security components, REST controllers, page controllers, and view models. This structure was chosen to keep business rules, database logic, validation, security, and presentation concerns separated. Mutating operations such as uploads, driver changes, vehicle changes, and password updates are handled separately from read operations such as dashboards, session details, score trends, and analytics.

JUnit/Jupiter was used for unit and integration-style testing. Mockito was used where mocking was required. Spring MVC test support and Spring Security test support were used to validate backend web behaviour, authentication, protected routes, and request handling. On the Android side, local unit tests were used for scoring, telemetry mapping, synchronization logic, credential policy behaviour, localization, and related software components.

Spring Boot Actuator, Micrometer, Prometheus registry support, Prometheus, and Grafana were used for monitoring-oriented backend support. These tools were selected because backend administration systems need operational visibility, especially when deployed as a server-side application. Monitoring support helps expose health and runtime metrics, while Grafana provides a visual dashboard for observing server behaviour.

Git and GitHub were used for version control and repository management. This allowed the software team to track changes across the Android client and backend-admin implementation, preserve implementation history, and support collaborative development.

The main standards and design practices used across both subsystems were RESTful API design, JSON-based data exchange, WebSocket telemetry streaming, layered architecture, MVVM for Android presentation logic, repository-based persistence access, DTO-to-domain mapping, deterministic scoring, local-first persistence, role-protected administration, authenticated machine upload, and preservation of uploaded historical session truth. These standards were selected to make the system explainable, testable, maintainable, and suitable for integration across the AI and software subsystems.

Mechatronics team's tools and technologies:

Tool / Technology	**Purpose**	**Change from Proposal**	**Status**
SolidWorks	3D CAD chassis modeling	No change	Complete
FDM 3D printing (PLA+ / TPU 98A)	Physical chassis fabrication	No change	Complete
ESP32 (Arduino / C++)	Real-time motor control, sensor acquisition, UART comms	No change	In use
60 A ESC	Brushless motor speed control	Motor-driver IC replaced by ESC	In use
D3542 1000KV brushless motor	Propulsion	DC motor replaced by brushless	Bench-verified
MG995 servo	Steering actuation	No change	In use
MPU6050 IMU	Acceleration/gyroscope for vehicle dynamics	No change	Verified; pending mount
Hall-effect speed sensor	Wheel speed / braking telemetry	No change	Verified; pending wheel mount
UART serial (ESP32 ↔ Jetson)	Bidirectional telemetry and command link	No change	Established and verified
SD-card Black Box (on Jetson)	Ground-truth telemetry logging	No change	Configured and ready
Dedicated voltage regulator	Isolated, stable Jetson power	Added — replaces generic power circuit	In procurement

Xbox Series X controller (BT)	Human control interface — analog throttle, steering, turn signals	Not in proposal; added during development	In use

Table 3.3.2: Mechatronics Tools and Technologies

3.4 Data Collection and Analysis

The Road Monitoring pipeline does not use an external dataset for lane deviation estimation. All parameters were determined empirically through live calibration on the physical track. The physical track was constructed from black cardboard sheets 50cm wide, matching the RC car's 40cm body width with approximately 5cm clearance per side. Yellow adhesive tape approximately 2–3cm wide was applied along both lane edges, providing strong colour contrast against the black surface.

The IMX219 camera was mounted on the car body pointing straight downward. At the mounting height used during testing, the two yellow tape edges appear approximately 300–360 pixels apart in the 1280x720 raw frame. A live calibration tool was written that displays the binary HSV mask alongside the raw camera frame in real time. HSV lower and upper bounds were adjusted interactively until both tape edges appeared as solid white blobs in the mask with no background noise. Final calibrated values were H in [15, 35], S in [60, 255], V in [60, 255].

Calibration of the geometric constants LANE_CENTER_CAL and LANE_WIDTH_CAL was performed by positioning the camera directly over the track centre and running a diagnostic script that printed detected lane centre pixel coordinates and lane widths frame by frame. These values were recorded separately at the raw 1280px resolution (used by the PyTorch pipeline) and confirmed at the 640px TRT input resolution (used by the TensorRT pipeline). A critical finding during validation was that the two resolutions require separate calibration constants, as the same pixel coordinate at 1280px corresponds to half the value at 640px.

Deviation accuracy was validated manually by moving the camera to known positions, fully left edge, centre, and fully right edge, and confirming that the score output matched expected values: approximately 0.8–1.0 at full departure, approximately 0.3–0.5 at moderate offset, and 0.0 at centre within the dead zone. Terminal logs printing deviation every 15 frames provided a quantitative record for each test session.

The in-cabin AI subsystem uses live camera frames during operation and pretrained models for driver-state analysis. The subsystem does not require storing raw driver video during normal runtime. Instead, frames are processed locally, and only the resulting indicators are passed to the software layer.

For drowsiness detection, the system analyzes eye and mouth landmarks from each detected face. EAR is calculated for eye closure, while MAR is calculated for mouth opening. The system also uses a CNN eye-state classifier to support open/closed eye detection. The drowsiness model achieved approximately 92% accuracy.

For emotion recognition, the system uses an eight-class emotion model. The model's full output includes neutral, happiness, surprise, sadness, anger, disgust, fear, and contempt. However, only sadness and anger are treated as meaningful distress classes in the final system. All other classes are mapped to neutral. The emotion model achieved approximately 85% accuracy.

The data analysis process uses temporal smoothing and frame-based persistence. This is necessary because driver monitoring should not react to a single noisy frame. For example, a single blink should

not create a drowsiness event, and a momentary expression change should not create a distress event. Temporal smoothing is applied as:

$$S_t = \alpha X_t + (1 - \alpha) S_{t-1}$$

where S_t is the smoothed score at time t, X_t is the current raw score, and α is the smoothing factor. Temporal smoothing makes the output more stable and suitable for the Driver Reliability Score.

The final processed data is represented as numerical scores, event flags, face-detection status, FPS, and timestamp. This makes the data lightweight, privacy-aware, and suitable for real-time integration with the software subsystem.

These structured indicators serve as the input boundary for the software subsystem, which does not collect raw video datasets, train AI models, or process mechanical sensor data directly at the hardware level. Instead, the software subsystem collects and processes structured telemetry and completed-session records produced through the integration boundary with the AI and Mechatronics subsystems. The main data handled by the software consists of normalized telemetry packets, local session records, scoring artifacts, upload payloads, backend historical records, and administrator analytics.

During an active driving session, the Android client receives structured telemetry values from the external telemetry source. These values include timestamp, speed, acceleration, obstacle distance, drowsiness score, emotional-risk value, lane deviation, and turn-signal state. The software does not require raw camera frames or raw biometric video. The AI subsystem is responsible for converting camera input into numerical indicators, while the Android client consumes those indicators as structured telemetry fields.

Before scoring, the Android client maps incoming transport data into domain-level telemetry packets. This preprocessing step includes field validation, unit conversion, and normalization where required. For example, speed received in kilometers per hour is converted into meters per second before being used in braking and distance calculations. Lane-deviation, drowsiness, and emotional-risk values are treated as normalized or threshold-based risk inputs for the scoring model. Invalid timestamps or non-increasing telemetry timestamps are handled by assigning a zero continuous-penalty time step, preventing unfair score deductions caused by malformed timing data.

The main analytical method used by the Android client is deterministic rule-based scoring. Each telemetry packet is processed by the scoring engine to update the current Driver Reliability Score. The analysis separates risk into continuous penalties, event penalties, and escalation penalties. Continuous analysis measures sustained risk over time, such as fatigue, lane deviation, and braking aggression. Event analysis detects discrete unsafe behaviours, such as high fatigue episodes, lane departures, harsh braking, and emotional distress. Escalation analysis detects repeated or combined risk patterns, such as repeated harsh braking within a short time window or fatigue occurring together with lane deviation.

The Android client also records session artifacts locally. These include the session summary, score-history points, detected safety events, detected escalation records, driver data, vehicle data, and synchronization metadata. Local storage is implemented using Room over SQLite. This supports offline-first operation, allowing a completed session to remain available even if backend connectivity is not available at the time the session ends.

After a session is completed, the Android client prepares a completed-session upload payload for the backend. This payload contains the session identity, driver identity, vehicle identity, timestamps, end status, validity status, final score, penalty totals, duration, distance, event count, escalation count, event records, escalation records, and score-history points. The payload represents the completed historical result of the Android scoring process.

The backend analyzes uploaded data through validation, persistence, and query-side aggregation. Validation checks whether the uploaded session structure is acceptable, whether required fields are present, whether referenced driver and vehicle records exist, and whether duplicate session uploads should be rejected or ignored. Warning-level issues are stored as ingestion issue flags where appropriate. This allows the backend to record upload concerns without silently changing the uploaded business truth.

The backend does not recompute the Android final score, penalty totals, events, escalations, distance, duration, or score history. Instead, it stores the uploaded values as historical records. This decision preserves traceability because the backend represents the completed session as produced by the Android client rather than creating a different version of the same session after upload.

Backend analysis is performed mainly through aggregate queries and dashboard calculations. The backend computes values such as average final score, total fleet distance, daily score trends, driver status counts, vehicle status counts, session validity counts, event-type counts, escalation-type counts, invalid-session rate, warning-session rate, completed-session rate, active-driver share, and busy-vehicle share. These analytics are used to support administrator review of driver reliability and fleet-level safety patterns.

The collected software data is therefore used in two stages. First, the Android client performs live local analysis during the driving session to produce the Driver Reliability Score and supporting artifacts. Second, the backend performs historical analysis after upload to support review, filtering, aggregation, and dashboard reporting. This two-stage approach keeps real-time scoring close to the telemetry source while still allowing centralized administrative analysis after the session is completed.

The Mechatronics subsystem produces the physical ground-truth data the rest of the project validates against. The ESP32 reads 6-axis acceleration and gyroscope data from the MPU6050 at a fixed sample rate and converts Hall-effect pulse counts into a linear wheel-speed estimate, packaging both into the outgoing telemetry packet at 20 Hz over USB serial.

Bench-test data collected during firmware verification covered the following measurement categories:

ESC and motor characterisation. At full throttle (unloaded, wheels off the ground), the motor was run at ESC_FWD_MAX_US = 1600 µs and the resulting behaviour observed on the Serial Monitor. The ramp-up time from neutral to full throttle was confirmed against the theoretical value: with ACCEL_STEP = 3 and RAMP_INTERVAL = 15 ms across the 60 µs forward band, the expected time-to-target is (60 / 3) × 15 = 300 ms, and the Serial Monitor trace confirmed currentThrottle reached 1600 µs within that window. The brake response — commanding ESC_REV_MAX_US = 1000 µs on LT press — was verified to produce an instantaneous snap in the Thr: output on the same serial line as the trigger press, confirming the bypass of the ramp gate.

Hall-effect pulse-per-revolution confirmation. The wheel was rotated by hand at low speed through one full revolution while the Serial Monitor displayed the running pulse count. With four magnets in alternating N-S-N-S polarity and CHANGE-mode interrupts capturing all four transitions, the counter was confirmed to increment by exactly four counts per revolution, validating the PULSES_PER_REV = 4 constant used in the speed calculation. A second manual test rotated the wheel at a known cadence (timed by stopwatch over ten revolutions) and compared the resulting speedKmh output against the hand-calculated value: v = (10 rev × 0.4084 m) / t_seconds × 3.6, confirming the speed formula within the measurement uncertainty of the manual timing method.

Accelerometer sign and axis confirmation. Three physical push tests were performed with the ESP32 and IMU powered on the bench, wheels off the ground, with the car held level. A sharp forward push produced a brief positive spike on accelFwdMps2, confirming ACCEL_SIGN = -1 and the getAccX() axis mapping. A sharp rearward push produced a negative spike. A nose-up tilt produced a positive

carPitch value (confirming the -AngleY mapping), and a roll to the right produced a positive carRoll value (confirming the +AngleX mapping). These results were logged from the Serial Monitor and compared against the expected sign conventions documented in Section 3.2.

Gyroscope offset calibration verification. The gyro offset calibration (calcGyroOffsets(true)) was run with the car motionless on a flat surface. The AngleZ drift rate before and after calibration was compared by logging AngleZ values over a 60-second stationary interval. Before calibration, AngleZ drifted at approximately 2–5 degrees per minute. After calibration, residual drift was reduced to below 0.5 degrees per minute, which is consistent with the MPU6050's specified residual bias after calibration and acceptable for the short-duration runs the platform performs.

JSON packet structure validation. A sample telemetry packet was manually traced through the Jetson FastAPI server's parsing path. The ESP32 Serial Monitor output was captured, and a single JSON line was copied and parsed manually against the schema expected by the server's ESP32 contract endpoint. All seven fields (speedKmh, steeringAngleDeg, accelerationMps2 , turnSignal, ignitionOn, stationary) were confirmed present, correctly typed, and within expected ranges. The stationary flag transition — from true to false — was verified by spinning the wheels above 0.5 km/h and observing the field change in real time.

Resting-offset correction verification. With the car on its chassis at rest, the raw getAccX() reading was logged over 200 samples (200 × 5 ms = 1 second) and averaged to establish the resting offset caused by the chassis not sitting perfectly level. The corrected accelFwdMps2 output was confirmed to read approximately 0.0 m/s^2 at rest, and a brief forward push produced a peak reading consistent with the physical acceleration applied, validating the offset subtraction formula a_fwd = ACCEL_SIGN × (a_x,raw − a_x,offset) × 9.81.

Intended downstream analysis will cross-reference Black-Box SD-card logs — timestamped Jetson-side recordings of each telemetry entry at 20 Hz — against AI detections and CARLA simulation outputs, to confirm the hardware behaves consistently with the virtual environment. The primary comparison axes are as follows.

Speed calibration against physical measurement: the Hall-effect speed estimate will be compared against a measured distance-over-time ground truth (a known-length track section timed by a stopwatch, or video frame-counting at a fixed frame rate), to quantify the percentage error introduced by wheel slip, magnet spacing tolerance, and the 500 ms integration window. Any systematic offset will be corrected via a per-platform calibration constant applied before the speedKmh field is written to telemetry.

Braking characterisation: the accelFwdMps2 signal during a hard-brake event (full LT press from forward driving) will be logged and compared against the theoretical maximum deceleration achievable given the ESC's reverse current limit and the platform's 2.4 kg mass. This provides an empirical drag coefficient and confirms whether the instantaneous brake command (currentThrottle snap to ESC_REV_MAX_US) produces the expected g-level spike in the IMU data.

Sim-to-real consistency check: CARLA simulation runs at equivalent speed and steering inputs will be compared against logged telemetry to quantify the gap between the simulated and real vehicle dynamics — specifically in yaw response to a steering step input. Discrepancies between the simulated yaw rate and the IMU-measured yaw rate (AngleZ derivative) provide the primary metric for sim-to-real transfer fidelity and inform whether a correction layer is needed before CARLA-trained algorithms are deployed on the physical platform.

Sensor fusion validation: the steeringAngleDeg field (derived from servo pulse) will be compared against yaw rate (AngleZ derivative from the IMU) at known straight-line and turning manoeuvres, to verify that the commanded steering angle correlates with the measured rotational response. A significant divergence between commanded and measured yaw at a given steering pulse indicates

mechanical slack, servo nonlinearity, or Ackermann geometry effects not captured by the linear pulse-to-angle mapping, and would motivate addition of a physical steering encoder in a future iteration.

4. Implementation

4.1 Development Process

The AI department was divided into two subsystems for the best efficiency, the subsystem that was responsible for the road monitoring and external camera pipeline was developed through six sequential implementation phases, each validated before proceeding:

The first phase focused on hardware bring-up. The IMX219 CSI camera was connected via ribbon cable to the Jetson's CAM1 port. The second phase established the YOLOPv2 baseline. The TorchScript model was loaded with torch.jit.load, run in FP16 mode on the Jetson GPU, and confirmed producing segmentation overlays on live camera frames. After the Jetson is connected to power the lane deviation model automatically runs and saved two videos simultaneously after it ends. The MKV video recording of both annotated and raw streams was implemented using OpenCV VideoWriter. Baseline inference throughput was measured at 8–12 FPS.

The third phase implemented the HSV lane deviation estimator. This was the most significant algorithmic development phase. The estimator was designed from first principles: HSV masking, morphological filtering, ROI cropping, horizontal histogram computation, peak group detection, left/right edge pair selection, offset computation, and temporal smoothing. A single-edge fallback was also implemented to handle partial occlusion cases.

The fourth phase implemented the output interface. The deviation score is written to /tmp/lane_deviation.txt on every processed frame. A crontab @reboot autostart script was configured to launch the pipeline automatically on power-on with appropriate startup delays for camera daemon stability.

The fifth phase performed TensorRT conversion to increase the camera’s fps. The TorchScript model was exported to ONNX, simplified with onnxsim, subjected to surgical graph node removal to eliminate an incompatible SequenceConstruct operator, and built into a TensorRT FP16 engine at 384x640 input resolution. A complete TRTEngine inference class was written to manage tensor allocation and asynchronous execution. Doing this boosted the fps from 8 to 20.

The sixth phase was fine tuning and polishing the model, the team mainly tuned LANE_CENTER_CAL, LANE_WIDTH_CAL, and SMOOTH_ALPHA, all of which were tuned iteratively based on observed deviation behaviour across left, centre, and right positions. An AI interface module was also built to send the final scores into the software team’s server.

The subsystem also simulated the lane deviation logic on CARLA autonomous driving simulator to ensure it was working well. The process was iterative. A minimal test.py script first verified the CARLA connection (host 127.0.0.1, port 2000) and map load. The main script was then built up in stages: vehicle spawn and manual keyboard control, RGB camera attachment, YOLOPv2 loading and inference, the lane-deviation estimator, speed/acceleration calculation, and finally the hybrid obstacle system. Each stage was tested by driving manually in CARLA and inspecting console output and debug images before moving to the next. Traffic/pedestrian spawning and weather control were split into separate scripts (spawn_city_life.py, change_carla_weather.py) so the core AI script could be reused unchanged across test scenarios.

The in-cabin AI subsystem was developed using an iterative development process. The implementation was divided into four main modules: face detection, drowsiness detection, emotion recognition, and head-pose estimation. Each module was implemented and tested separately before being connected into the final driver monitoring pipeline.

The first implementation phase focused on camera-frame processing and face detection. This stage was necessary because all other AI modules depend on locating the driver's face correctly. The frame is captured from the in-cabin camera, converted into the required format, and passed into the face detector. If a face is detected, the detected face region is used by the emotion model, while facial landmarks are extracted for the drowsiness and head-pose modules.

The second phase focused on drowsiness detection. This included the implementation of Eye Aspect Ratio, Mouth Aspect Ratio, CNN-based eye-state classification, and temporal frame counting. The main goal of this phase was to detect sustained eye closure and yawning while avoiding false alerts from normal blinking or short facial movements.

The third phase focused on emotion recognition. The emotion model used in the project produces eight possible emotion classes. However, the final system maps these into three operational categories: sad, angry, and neutral. This simplification was selected because sadness and anger are more relevant for driver-risk interpretation, while the remaining classes are treated as neutral to avoid unnecessary overinterpretation.

The fourth phase focused on head-pose estimation. This module estimates yaw, pitch, and roll from facial landmarks using a geometric pose-estimation method. The output supports attention monitoring by detecting whether the driver's face orientation remains away from the normal driving direction for multiple consecutive frames.

The fifth AI phase focused on integration with the Jetson Orin Nano environment. The WebSocket and API hosting on the Jetson was handled by the software team. The AI team's role was to generate processed outputs that could be transferred to that communication layer. Therefore, the AI implementation focuses on producing structured driver-state indicators instead of handling the complete software-side communication system.

Once these structured outputs were made available through the Jetson-hosted communication layer, the software subsystem began its own development process. The software subsystem followed an iterative development process rather than a strictly linear Waterfall model. This approach was selected because the project contained several dependent parts: Android scoring, local persistence, telemetry mapping, backend upload ingestion, administrator authentication, database storage, and analytics. Implementing all of these as one large final step would have made debugging and integration difficult. Instead, the software was developed in phases, with each phase adding a specific layer of functionality and validating it before moving to the next layer.

The first software phase focused on architecture and responsibility separation. The system was divided into two connected software components. Phase 1 was the Android/Kotlin thick client, responsible for local driver workflows, telemetry handling, scoring, offline-first persistence, and upload preparation. Phase 2 was the Java Spring Boot backend-admin system, responsible for authenticated upload ingestion, historical storage, administrator management, driver and vehicle management, session review, and analytics. This separation was treated as the foundation of the implementation because it defined which part of the system owns live scoring and which part owns historical administration.

The second software phase focused on the Android domain layer. Core models were created for telemetry packets, drivers, sessions, events, escalations, vehicles, score points, and session state. The Driver Reliability Score logic was implemented as deterministic domain logic. The scoring process was built around continuous penalties, event penalties, and escalation penalties. This phase was important

because the scoring model had to remain independent from UI screens, database code, and network transport details. Keeping the scoring engine isolated made it easier to test and easier to explain in the report.

The third software phase focused on Android persistence and offline-first behaviour. Room entities, DAOs, repositories, and database access paths were implemented to store drivers, sessions, safety events, escalation records, score-history points, vehicles, and synchronization metadata. This milestone allowed the Android client to preserve completed sessions locally instead of depending on immediate backend connectivity. The offline-first design was necessary because a driver-monitoring application must be able to continue scoring and storing a session even if internet access is unavailable during or immediately after a trip.

The fourth software phase focused on Android network and integration boundaries. WebSocket telemetry parsing and mapping were implemented to convert incoming JSON telemetry into domain-level telemetry packets. Backend API DTOs and mapper logic were added to prepare completed-session uploads. Jetson and vehicle-control client boundaries were also defined so that the Android client could interact with external vehicle-state and telemetry services through clear interfaces. This phase established the client-side contract between the software subsystem and the external AI and Mechatronics telemetry sources.

The fifth software phase focused on Android orchestration and user interface workflows. ViewModels and Compose screens were connected to the domain, persistence, and repository layers. Driver-facing screens, session-related workflows, history views, profile and settings areas, and admin-facing mobile access paths were implemented around observable state. This allowed the application to expose the underlying scoring and persistence features through usable mobile workflows while keeping UI code separate from business logic.

The sixth software phase focused on the backend project structure. A Spring Boot application was created with separate packages and responsibilities for domain models, persistence entities, repositories, mappers, validation services, command services, query services, security, REST controllers, page controllers, and view models. This layered structure was selected to avoid mixing upload processing, database access, security rules, and presentation logic in the same classes.

The seventh software phase focused on backend persistence and domain modelling. JPA entities and repositories were implemented for administrator accounts, API clients, drivers, vehicles, sessions, safety events, escalation records, score-history points, and ingestion issue records. MySQL was used as the persistent storage layer. This milestone allowed the backend to store completed-session uploads as historical records instead of treating uploaded sessions as temporary request data.

The eighth software phase focused on backend security and upload ingestion. Administrator authentication and protected routes were implemented for the admin web interface. API-client authentication was implemented for machine-to-machine completed-session uploads. The upload service was then developed to coordinate request validation, duplicate-session handling, driver and vehicle lookup, transactional persistence, anomaly or issue recording, driver aggregate updates, and upload response construction. This was a major implementation milestone because it completed the path from Android-produced completed sessions to backend historical storage.

The ninth software phase focused on backend administration and analytics. Server-rendered administrator pages were implemented for login, dashboard, drivers, driver creation, driver details, driver analytics, vehicles, vehicle creation and editing, sessions, session details, fleet analytics, settings, and profile. Query services were implemented to assemble dashboard metrics, global analytics, per-driver analytics, session lists, session details, score trends, event counts, escalation counts, and status summaries. This phase transformed the backend from a storage-only component into an administrator-facing fleet review system.

The tenth software phase focused on testing, correction, and integration review. Android build validation and local unit tests were used to verify core software behaviour such as scoring, telemetry mapping, synchronization, credential handling, and related logic. Backend tests and workflow checks were used to verify administrator authentication, protected access, driver and vehicle management, upload ingestion, persistence of child records, session review, and analytics output. This phase also identified remaining validation gaps, especially real-device Android testing, live Jetson telemetry testing, deployed-backend recovery testing, and full physical-system integration.

The key milestones achieved during implementation were the completion of the Android scoring engine, local Room persistence, telemetry mapping, backend upload contracts, synchronization readiness, Spring Boot backend structure, authenticated administrator access, authenticated API-client upload, MySQL historical persistence, driver and vehicle management, session review pages, fleet analytics, and dashboard views. Together, these milestones produced a complete software path from local session scoring to backend administrative review.

The Mechatronics subsystem used an iterative, work-package-driven process, front-loading planning, procurement, and design, then running firmware development and bench validation in parallel with chassis fabrication. Each firmware module was verified individually on the bench before integration was attempted, consistent with the bench-first methodology described in Section 3.1. The deliberate decoupling of electronics from mechanical fabrication meant that the full firmware stack was bench-proven before the chassis was available, so final assembly proceeds against components with a known-good baseline rather than an untested one.

Firmware evolved through the following sequence of bench-validated states, each treated as a precondition for the next:

ESC arming and core actuation. The ESC was confirmed to be in single-click forward/reverse mode during bench testing: a 1300 µs test pulse produced reverse rotation rather than active braking, which determined the entire brake strategy. The arming sequence was validated to hold ESC_NEUTRAL_US = 1500 µs cleanly for three seconds, with no sub-neutral pulse reaching the ESC during the arming window. Throttle ramping at ACCEL_STEP = 3 per 15 ms tick was confirmed against the theoretical 300 ms time-to-full-throttle. Proportional, instantaneous reverse/brake — snapping currentThrottle directly to the LT-proportional reverse depth without entering the ramp gate — was validated by observing the Thr: Serial Monitor output snap to the commanded value on the same line as the trigger press. The safe-start gate was confirmed to hold the ESC at neutral after controller pairing until both triggers were released, and to re-arm correctly on simulated reconnection.

Manual steering. Joystick-to-servo mapping was implemented and immediately found to be inverted on first test: left stick produced right wheel turn. The direction-inversion flag (STEER_REVERSED = true) was introduced and confirmed to correct the response. Steering was validated to track the full joystick travel across the servo pulse range [544, 2400] µs and to update on every loop iteration rather than on the ramp timer, confirming low-latency response independent of throttle state.

IMU integration. A dedicated standalone motion-test sketch was written first — separate from the car firmware — that printed plain-language movement labels (NOSE UP, LEAN LEFT, TURNING RIGHT) alongside raw axis values at 5 Hz. This confirmed axis sign conventions and the -AngleY pitch mapping empirically before any integration into the control loop. The gyro calibration (calcGyroOffsets(true)) was validated by logging AngleZ drift over 60 seconds before and after calibration: pre-calibration drift was approximately 2–5 degrees per minute; post-calibration residual was below 0.5 degrees per minute. The accelerometer resting-offset correction was validated by confirming accelFwdMps2 read approximately 0.0 m/s² at rest on the bench, and

produced a physically consistent spike on a manual forward push. The IMU read was confirmed non-blocking: the 50 Hz sample rate introduced no measurable latency impact on the 15 ms throttle ramp or on steering update rate, verified by watching Thr: update frequency remain unchanged before and after IMU code was merged.

Hall-effect wheel-speed sensing. The US1881 was initially wired without a pull-up resistor, producing a floating input that triggered spurious interrupts and reported grossly incorrect speeds. The fix — INPUT_PULLUP in firmware plus a 12 kΩ hardware pull-up to 3.3 V with VDD at 5 V — eliminated the spurious triggering. CHANGE-mode interrupts were confirmed to produce exactly four counts per manual wheel revolution, validating the PULSES_PER_REV = 4 constant against physical measurement. A timed ten-revolution hand-spin test confirmed the speedKmh output matched the hand-calculated value within manual timing uncertainty. The 500 ms windowed calculation was confirmed to return 0.0 km/h correctly within one window period after the wheel stopped, with no residual count leakage.

Jetson telemetry. The JSON packet was constructed incrementally, field by field, and each field was validated against the shared schema before the next was added. A sample packet was manually copied from the Serial Monitor and parsed through the Jetson FastAPI server's json.loads() path to confirm correct typing and field names. The DEBUG_HUMAN flag was tested in both states: with true, human-readable startup messages appeared and were confirmed to cause a parse exception on the server; with false, only clean JSON lines were emitted and parsed cleanly. The 20 Hz output rate was confirmed by timestamping 100 consecutive packets and computing the mean inter-packet interval, which was 50.1 ± 0.4 ms, within acceptable jitter for the application.

4.2 Detailed Implementation

The final in-cabin AI implementation is organized around the main Python entry point and the AI modules in the project structure. The main pipeline initializes the camera stream, FPS counter, face detector, landmark tracker, drowsiness detector, emotion classifier, head-pose estimator, and telemetry-related components. During runtime, the system reads each frame, detects the face, extracts landmarks, runs the AI modules, and produces processed driver-state outputs.

The face detection module was implemented using OpenCV Haar cascade detection. The frame is first converted to grayscale, and histogram equalization is applied to improve contrast. The detector then searches for frontal faces using the configured cascade model. If more than one face is detected, only the largest face is selected because the in-cabin camera is intended to monitor the driver, who is expected to be the closest face in the frame. This reduces unnecessary processing and lowers the chance of using irrelevant background faces.

The face detector uses parameters such as a scale factor of 1.1, a minimum neighbor value of 5, and a minimum face size of 60 × 60 pixels. These values were selected to balance detection sensitivity and false-positive reduction. A lower minimum neighbor value could detect more faces but may also increase false detections. A higher value reduces false detections but may miss the driver's face under difficult lighting conditions.

The facial landmark tracker uses a face landmark model to extract key points from the detected face. These landmarks are used by both the drowsiness and head-pose modules. The landmark tracker attempts to use GPU acceleration when available and falls back to CPU processing if GPU delegation is not available. This improves deployability because the system can still function even if the GPU delegate fails.

The drowsiness module combines rule-based geometry and CNN-based classification. First, the system extracts eye landmarks and calculates the Eye Aspect Ratio for both eyes. The left and right EAR values are averaged to estimate the driver's eye openness. If the average EAR is below the threshold of 0.25, the eyes are considered potentially closed. However, the system does not immediately produce a drowsiness event. Instead, it uses a consecutive-frame counter. Eye closure must continue for 20 consecutive frames before the system treats the situation as drowsiness. This is necessary because normal blinking can temporarily reduce the EAR value.

The EAR equation used by the system is:

$$EAR = \frac{\| p_2 - p_6 \| + \| p_3 - p_5 \|}{2 \| p_1 - p_4 \|}$$

The drowsiness module also calculates Mouth Aspect Ratio from mouth landmarks. The MAR threshold is 0.60, and mouth opening must continue for 15 consecutive frames before the system treats it as yawning. This prevents short mouth movements or speech from being immediately interpreted as fatigue.

The MAR equation used by the system is:

$$MAR = \frac{\| p_2 - p_8 \| + \| p_3 - p_7 \| + \| p_4 - p_6 \|}{3 \| p_1 - p_5 \|}$$

The drowsiness module also uses a CNN-based eye-state classifier as a secondary confirmation layer. The CNN classifier supports the geometric EAR signal, providing an independent assessment of eye closure before a drowsiness state is raised. The emotion classification module processes the detected face region through the MobileNetV2-based eight-class FER+ model. The eight output classes are mapped into three operational categories: sad, angry, and neutral. Classes that do not correspond to sadness or anger are collapsed to neutral. The head-pose estimation module uses the solvePnP algorithm with a predefined set of 3D facial reference points and their 2D landmark correspondences to estimate yaw, pitch, and roll. Yaw and pitch are used to detect whether the driver is looking away from the forward direction for multiple consecutive frames.

YOLOPv2 Inference

YOLOPv2 is a multi-task convolutional neural network performing simultaneous traffic object detection, drivable area segmentation, and lane line segmentation from a single forward pass. In the baseline PyTorch configuration, the model is loaded as a TorchScript file and executed in FP16 mode on the CUDA device:

```
[pred, anchor_grid], seg, ll = model(img)
```

For each camera frame, the image is passed to the YOLOPv2 model, which performs multiple perception tasks in a single forward pass. The model produces three outputs: object detection predictions (pred), drivable area segmentation (seg), and lane line segmentation (ll). The raw segmentation outputs are converted into binary masks using the driving_area_mask() and lane_line_mask() functions. These masks are then overlaid onto the original camera image using show_seg_result(), allowing the detected road area and lane markings to be visualized in real time. During testing, the team added these masks to check how yolopv2 was behaving and detecting on the fake track; therefore, it was primarily used for visualization rather than as the main source of lane-deviation estimation.

Lane Deviation Estimator

The LaneDeviationEstimator class implements the complete lane detection and deviation scoring logic. The input frame is first converted to the HSV color space to improve robustness under varying lighting conditions. A color threshold is then applied to isolate lane-like (yellow) markings, followed by morphological opening and closing operations using a 5×5 kernel to remove small noise and fill gaps in the detected lane regions.. To reduce irrelevant information, a region of interest (ROI) is defined over the lower 20% to 95% of the frame height, removing both the vehicle body region and distant background. Within this ROI, a horizontal pixel histogram is computed to identify dense lane-mark regions, which is then smoothed using a moving average kernel of size 15 to reduce fragmentation.

Lane boundary candidates are extracted by detecting continuous peak regions in the histogram and grouping adjacent columns. These candidates are evaluated based on geometric consistency, including expected lane width constraints and temporal stability with respect to the previous frame. The best left and right lane pair is selected, and the lane center is computed from their midpoint. The final lane deviation is calculated as the offset between the image center and the estimated lane center. When a valid pair is found, the raw deviation score is computed as:

$$raw = \min\left(\frac{|lane\_Center - lane\_Center\_Cal|}{lane\_Width/2}, 1\right)$$

where the lane center is calculated as the midpoint between the detected lane edges:

$$lane\ center = \frac{x_{left} + x_{right}}{2}$$

The lane width is defined as:

$$lane\ width = x_{right} - x_{left}$$

The value lane_center_cal represents the calibrated center position of the lane when the vehicle is correctly aligned during initialization. To reduce the effect of small measurement noise during straight driving, a threshold of 0.04 is applied, forcing small deviations to zero.

Finally, temporal smoothing is applied using an exponential moving average:

$$dt = \alpha \cdot \mathrm{raw_t} + (1 - \alpha)\, d_{t-1}$$

where $\alpha = 0.6$.

A single-edge fallback is used when only one lane marking is visible, which can happen during sharp turns or partial occlusions. In this case, the visible edge is classified as either the left or right lane boundary using simple heuristics based on its position in the frame, the previous drift direction, and its distance from the last known lane edges.

Once classified, the lane center is estimated using the previous lane width:

$$est\ center = c_x + \frac{lane\ width}{2}\ (if\ left\ edge)$$

$$est\ center = c_x - \frac{lane\ width}{2}\ (if\ right\ edge)$$

where $c_x$is the detected edge position and the lane_width is the most recently measured lane width.

If no lane markings are detected, the system temporarily retains the previous deviation value for a short grace period of 3 frames to handle brief occlusions. After this period, the value gradually decays by a factor of 0.80 per frame until valid lane information is detected again.

TensorRT Optimisation

TensorRT is NVIDIA's inference optimization library that improves model speed by fusing operations, using hardware-specific GPU kernels, and reducing numerical precision. In this project, YOLOPv2 was deployed using TensorRT in FP16 mode on the Jetson Orin Nano, achieving approximately 1.6–2.8× faster inference compared to the PyTorch FP16 implementation. To deploy the model, it was first exported from PyTorch to ONNX format using torch.onnx.export which preserved all three outputs: object detection, drivable area segmentation, and lane line segmentation. The ONNX model was then simplified using onnxsim to remove redundant operations and optimize the computation graph.

During conversion, an unsupported ONNX operation (SequenceConstruct) was encountered, which is not supported by TensorRT. This operation was removed by modifying the ONNX graph and exposing the underlying detection tensors directly as separate outputs. This step was necessary because TensorRT can process these outputs independently without requiring list grouping.

The final optimized ONNX model was then converted into a TensorRT engine using a fixed input shape of (1, 3, 384, 640), FP16 precision. The resulting engine file is loaded at runtime by a custom TRTEngine class, which performs inference using TensorRT's asynchronous execution API to minimize latency and avoid CPU-GPU data transfer overhead. Doing this increased the fps from 8-10 to 18-22.

Output Interface and Autostart

The system outputs two video recordings for each session: an annotated video and a raw video. The annotated video includes the segmentation overlay, FPS counter, lane deviation value, and a color-coded status bar, while the raw video contains the unprocessed camera feed. The lane deviation is visualized using a color scheme for easier interpretation: green indicates safe driving conditions (deviation < 0.3), orange indicates caution (0.3–0.6), and red indicates high deviation (≥ 0.6).

To ensure autonomous operation, the system is configured to start automatically on boot using a crontab @reboot entry. This runs a startup script that initializes the camera service, waits for system readiness, and then launches the main inference pipeline. With auto-login enabled, the interface opens automatically without user interaction, allowing the system to run fully independently.

Finally the score reaches the software's server via an interface module built alongside the software's WebSocket.

CARLA Simulation

Lane-deviation estimator

The YOLOPv2 lane mask is cropped to a region of interest covering the lower part of the image (20%–95% of image height). Lane pixels in this ROI are converted into an x-axis histogram, smoothed with an averaging kernel of size 15, and grouped into peaks. Peaks left and right of the image centre are paired, rejecting pairs with an unrealistic lane width (<80 or >560 px) or insufficient pixel support/vertical spread, and the best-scoring pair (smallest centre error plus continuity with the previous frame) is selected. Deviation is computed as abs(vehicle_center - lane_center) / (lane_width/2), clamped to 1.0, snapped to 0.0 below 0.03, and smoothed over time with an exponential factor of 0.35. If no lane pixels are found, deviation resets to 0.0; if pixels exist but no valid pair is found, the previous value is kept. A keyboard-start flag forces lane_deviation to 0.0 until the UP or DOWN key is pressed for the first time, after which it behaves normally even if the vehicle stops, since speed is deliberately not used to zero the value.

Speed and acceleration:

Speed is the magnitude of CARLA's vehicle velocity vector converted to km/h; acceleration is CARLA's acceleration vector projected onto the vehicle's forward vector (dot product), giving longitudinal acceleration in m/s^2.

Hybrid obstacle detection:

Actor-based detection iterates CARLA's vehicle.* and walker.pedestrian.* actors, projects each onto the ego vehicle's forward/right vectors to get forward and lateral distance and reports the closest actor within 40 m ahead and 8 m lateral half-width, returning its CARLA type_id. If no actor is detected, a sensor.other.obstacle sensor acts as a fallback for static/parked car-like map meshes; its callback filters out non-vehicle hits (traffic lights, signs, poles, vegetation, buildings, etc.) by keyword and only accepts vehicle-like keywords, labelling accepted hits as static_obstacle with a 0.25 s timeout. This combination was needed because CARLA's built-in obstacle sensor alone also triggers on street furniture, and because many parked cars are static meshes rather than spawned vehicle actors.

The software implementation consists of two connected parts: the Android/Kotlin client and the Java Spring Boot backend-admin system. The Android client is responsible for live session processing, local scoring, local persistence, telemetry mapping, driver workflows, and upload preparation. The backend-admin system is responsible for authenticated upload ingestion, persistent historical storage, secure administrator access, driver and vehicle management, session review, and analytics. The two parts are connected through completed-session upload contracts and shared session semantics.

Android Client Implementation

The Android client was implemented as a thick-client mobile application. This means that the main driving-session logic is executed locally on the device rather than depending on a remote server during an active session. This design was chosen because the driver reliability score must continue updating even when internet connectivity is unavailable or unstable. The Android application therefore owns the live scoring process, while the backend receives the completed session only after it is finalized.

The Android codebase was structured into separate layers. The domain layer contains business models and scoring logic. The data layer contains Room entities, DAOs, database configuration, and repository implementations. The network layer contains telemetry parsing, WebSocket communication, backend DTOs, Jetson-control DTOs, and API interfaces. The orchestration layer connects active-session state, telemetry processing, scoring, persistence, and synchronization. The UI layer contains ViewModels and Compose screens.

Telemetry Packet Processing

The telemetry pipeline begins when the Android client receives structured JSON telemetry from the external telemetry source. The expected telemetry fields include timestamp, speed, acceleration, obstacle distance, drowsiness score, emotional-risk value, lane deviation, and turn-signal state. These values are not used directly inside UI code. Instead, the telemetry parser and mapper convert incoming transport data into a domain-level telemetry packet.

The telemetry mapper is responsible for normalizing values before they reach the scoring engine. For example, speed received in kilometers per hour is converted into meters per second so that braking, deceleration, and distance calculations use consistent units. The mapper also protects the domain layer from transport-specific naming or formatting differences. This prevents network DTOs from leaking into the scoring logic.

A simplified representation of the telemetry processing flow is: receive WebSocket JSON telemetry → parse JSON into telemetry DTO → map DTO into domain TelemetryPacket → validate timestamp and physical values → pass TelemetryPacket into active-session processing → update score, events, escalations, and score history → persist session state and artifacts locally.

This separation makes the telemetry boundary replaceable. If the external telemetry format changes in the future, the mapper can be updated without rewriting the scoring engine or UI screens.

Driver Reliability Scoring Implementation

The Driver Reliability Score was implemented as deterministic domain logic. The scoring engine processes one telemetry packet at a time and updates the current session state. The scoring implementation uses three categories of penalties: continuous penalties, event penalties, and escalation penalties.

Continuous penalties are applied to sustained risk signals over time. These include fatigue risk from drowsiness values, lane-deviation risk from lane position values, and braking-aggression risk from speed-derived deceleration. A time-step calculation is used so that penalties scale with elapsed time. Invalid or non-increasing timestamps do not create penalty growth. A maximum time step is also applied to prevent large timestamp gaps from causing unfair score reductions.

Temporal smoothing is applied to reduce the effect of short-term noise. This is important because telemetry values from AI and physical sensors can fluctuate between packets. The smoothing logic allows the score to respond to sustained risk while avoiding overreaction to a single noisy value.

Event penalties are used for discrete unsafe behaviours. These include high-fatigue episodes, lane-departure events, harsh-braking events, and emotional-distress episodes. Each event type has a defined threshold and penalty value. The event detector checks whether the telemetry stream satisfies the required condition for each event type, such as fatigue persisting for a required duration or braking deceleration exceeding the harsh-braking threshold.

Escalation penalties are used when risk patterns become more serious than isolated events. The escalation detector identifies repeated or combined behaviours, such as repeated harsh braking in a short window, fatigue occurring together with lane deviation, prolonged lane drift, or multiple events occurring close together. This allows the score to reflect not only the existence of individual unsafe actions, but also their repetition and combination.

A simplified representation of the scoring process is: processTelemetry(sessionId, state, packet) → calculate continuous risk from fatigue, lane deviation, and braking → update cumulative continuous penalty → detect new safety events → update cumulative event penalty → detect escalation patterns → update cumulative escalation penalty → calculate current score as 100 minus total penalties → clamp current score between 0 and 100 → create score-history point → return scoring result with score, penalties, events, and escalations.

The scoring engine was kept independent from UI, database, and network code. This was a deliberate design choice. It makes the score easier to test, easier to explain, and safer to modify because scoring behaviour is not hidden inside presentation logic or persistence logic.

Active Session and Session State

The active-session implementation keeps runtime state during a driving session. This includes the current score, previous telemetry values, smoothed risk values, cumulative penalties, detected event state, escalation windows, distance accumulation, and score-history information. Runtime session state is different from the persisted historical session record. The active state is used while the session is running; the historical record is produced when the session ends.

This distinction is important because active-session data may include temporary counters and rolling windows that are not useful as permanent historical records. The final persisted session contains the session summary and the artifacts required for review: events, escalations, and score-history points. This prevents the database from storing unnecessary internal runtime details while still preserving the evidence needed to explain the final score.

Room Persistence Implementation

Room was used to implement local persistence on the Android device. The local database stores driver records, session summaries, safety events, escalation records, score-history points, vehicle records, and synchronization metadata. DAOs provide structured access to the database, while repositories hide direct Room access from the rest of the application.

The persistence layer supports offline-first operation. When a driving session ends, its summary and child records can be stored locally even if the backend is unavailable. Synchronization metadata is stored with the session so that the application can later determine whether a completed session has already been uploaded, whether it still needs synchronization, and when synchronization occurred.

The local storage model separates session summaries from child artifacts. A session summary stores high-level information such as session identity, driver identity, vehicle identity, timestamps, final score, duration, distance, event count, escalation count, and synchronization state. Event records store individual safety events. Escalation records store detected escalation patterns. Score-history records store score progression during the session. This structure supports both summary screens and detailed session-review screens.

Repository and Synchronization Implementation

Repository interfaces were used to isolate the rest of the application from database and network implementation details. The Android client can request sessions, drivers, events, score history, or synchronization updates through repository operations rather than directly accessing DAOs or API services. This approach makes the code easier to maintain and allows mock repositories to be used during testing or development.

The synchronization implementation prepares completed sessions for backend upload. A completed-session upload contains the session summary and its related child records. The upload path is intentionally based on completed sessions rather than live telemetry streaming. This matches the backend responsibility: the backend receives historical completed-session artifacts, not raw active-session state.

The synchronization flow can be summarized as: find locally completed sessions that are not synchronized → load session summary and child records → map local database records into backend upload DTOs → send upload request to authenticated backend endpoint → if upload succeeds, mark the local session as synchronized → if upload fails, preserve the local records for later retry. This approach prevents data loss during connectivity failures. A failed upload does not delete the local session. The session remains available for retry.

Android UI and ViewModel Implementation

The Android user interface was implemented using Jetpack Compose. ViewModels expose screen state to the UI and coordinate calls to repositories, scoring workflows, and backend access paths. This follows an MVVM-oriented structure where the UI observes state rather than directly controlling business logic.

Driver-facing workflows include login, home, session-related screens, history, profile, and settings. Admin-facing mobile workflows include backend-facing access paths for reviewing fleet data where

applicable. The UI layer does not calculate the Driver Reliability Score directly. Instead, it displays values produced by the domain and repository layers. This keeps presentation code separate from safety-related logic.

Backend-Admin Implementation

The backend-admin system was implemented as a Java Spring Boot application. It uses a layered backend structure containing domain models, persistence entities, repositories, mappers, validation services, command services, query services, security components, REST controllers, page controllers, and view models. The backend was designed to act as the central historical administration system for completed driving sessions.

The backend does not perform live scoring. It does not process raw camera frames, raw vehicle sensor streams, or active telemetry packets. Its main role is to receive completed sessions from the Android client upload boundary, validate them, store them, and make them available to administrators.

Backend Domain and Persistence Implementation

The backend domain model represents the main concepts required for fleet administration: drivers, vehicles, driving session records, safety event records, escalation records, score-point records, administrator accounts, API clients, and ingestion issue flags. Persistence is implemented using JPA entities and Spring Data repositories with MySQL as the database.

Driver records store driver identity, account status, aggregate reliability information, total sessions, total distance, and timestamps. Vehicle records store vehicle identity and administrative state. Session records store completed-session summaries, including final score, penalty totals, distance, duration, validity, upload-processing status, and references to driver and vehicle records. Event, escalation, and score-point records preserve the child artifacts uploaded with the session. This persistence structure supports detailed historical review. Administrators can view not only the final score but also the supporting events, escalation patterns, score progression, and ingestion status.

Backend Upload Ingestion Implementation

The completed-session upload endpoint is one of the most important backend components. It receives completed-session payloads from an authenticated API client. The backend upload workflow includes authentication, validation, duplicate checking, driver lookup, vehicle lookup, mapping, persistence, ingestion issue handling, aggregate updates, and response construction.

The upload ingestion process can be summarized as: authenticate API client → receive completed-session upload request → validate required fields and structure → check for duplicate session identifier → verify referenced driver and vehicle records → map upload DTOs into domain and persistence records → persist session summary and child artifacts transactionally → record warning-level ingestion issues if needed → update driver aggregate fields → return upload response.

The upload workflow is transactional so that the session summary and related child artifacts are stored consistently. If a hard validation failure occurs, the backend rejects the upload through the error response path rather than partially storing an invalid session.

A key implementation rule is that the backend preserves the uploaded historical truth. It does not recompute the uploaded final score, continuous penalty, event penalty, escalation penalty, duration, distance, events, escalations, or score history. This is necessary because the Android client owns the live scoring sequence. The backend stores and reviews the completed result.

Backend Validation and Ingestion Issues

The backend validation logic is separated from controllers so that upload rules are centralized and reusable. Structural validation checks whether the request has the required fields, valid identifiers, valid timestamps, valid numeric ranges, and acceptable child-record structures. Soft validation or anomaly analysis can record warning-level ingestion issues when data is accepted but contains conditions worth reviewing.

Ingestion issue records are useful because they allow the backend to preserve uploaded values while still documenting concerns. For example, a session may be processed with warnings rather than silently modified. This supports auditability and avoids hidden changes to uploaded business data.

Backend Security Implementation

The backend uses two separate access paths: administrator access and API-client upload access. Administrator access is protected through session-based login and role-protected routes. This prevents unauthenticated users from accessing driver records, vehicle management, session review, dashboard pages, settings, and profile pages.

API-client authentication protects the upload endpoint. This separates human administrator access from machine-client upload access. The Android or upload client does not need to act as an administrator to submit completed sessions. This separation improves security because upload permissions and administration permissions are not treated as the same capability. Password handling uses BCrypt hashing so that plain-text passwords are not stored. This is an important baseline security requirement for both administrator and driver-related authentication workflows.

Backend Admin Pages and Query Services

The backend provides server-rendered administrator pages using Spring MVC and Thymeleaf. Implemented pages include login, dashboard, drivers, driver creation, driver detail, driver analytics, vehicles, vehicle creation and editing, sessions, session detail, fleet analytics, settings, and profile.

Query services assemble the data required by these pages. Read workflows are separated from command workflows. Upload ingestion and driver management are command-side operations because they change data. Dashboard metrics, session lists, driver analytics, score trends, event counts, and escalation counts are query-side operations because they read and aggregate stored data. This separation makes backend behaviour easier to maintain. Changes to analytics views do not require modifying upload ingestion, and changes to upload rules do not require rewriting dashboard pages.

Backend Analytics Implementation

Backend analytics are computed from stored completed-session records. The analytics layer calculates average final scores, total fleet distance, daily score trends, category counts, event counts, escalation counts, invalid-session rates, warning-session rates, completed-session rates, active-driver share, and busy-vehicle share. These calculations support fleet-level and driver-level review. A fleet administrator can inspect overall system activity, identify problematic drivers or sessions, review trends over time, and examine event or escalation distributions. The backend analytics are therefore designed for historical review and decision support, not for live scoring.

Integration Between Android and Backend

The integration between the Android client and backend is based on completed-session upload contracts. The Android client produces the completed session, including summary fields and child artifacts. The backend receives this payload through the upload API, authenticates the request, validates the structure, and stores the records.

This integration boundary keeps the responsibilities clear. The Android client is responsible for live telemetry interpretation and scoring because it processes the ordered telemetry stream during the session. The backend is responsible for historical persistence and administration because it has the database, security, dashboards, and analytics services. The integration design also supports future extension. More analytics, filters, reports, or administrator tools can be added on the backend without changing the scoring engine. Likewise, scoring thresholds or Android-side session logic can be modified without changing the backend's responsibility to preserve uploaded completed-session records.

Monitoring and Operational Support

The backend includes monitoring-oriented support through Spring Boot Actuator, Micrometer, Prometheus registry support, Prometheus, and Grafana. These tools provide visibility into server behaviour and runtime metrics. Although the capstone implementation is still a prototype, monitoring support is important because backend administration systems require operational awareness when deployed.

The implementation still requires production hardening before real fleet use. Secrets should be moved out of source code, HTTPS should be enforced, database migrations should be managed through migration tooling, backups should be configured, and deployment recovery procedures should be tested. These items are outside the core prototype implementation but are necessary for production readiness.

Summary of Implementation

The final software implementation provides a complete path from telemetry intake to local scoring and backend administration. The Android client processes telemetry, applies deterministic reliability scoring, detects events and escalations, stores completed sessions locally, and prepares upload payloads. The backend authenticates uploads, validates completed-session records, persists historical data, protects administrator access, and provides dashboards and analytics. Together with the AI pipeline running on the Jetson Orin Nano, this implementation satisfies the main system goal of transforming live camera input and vehicle telemetry into explainable, persistent, and administratively reviewable driver-reliability records.

Planning and procurement (WP1–WP2). The reporting period began with subsystem scope definition, WBS construction, and a detailed Bill of Materials covering all electrical, mechanical, and consumable items. Local market research in Turkey identified the RPLIDAR A1M8 as significantly over budget and unavailable at the required specification within the project timeline, leading to its formal removal from scope and replacement with an obstacleDistanceMeters placeholder field in the telemetry schema. Primary parts were procured through Robotistan and Trendyol; two items (screws and an ESC) were acquired personally due to supplier website issues. The ESC sourced was the Sunisfa WSDT-60A, a 60 A forward/reverse brushless unit confirmed compatible with a 3S LiPo and the D3542 1000KV motor. Component sourcing within Turkey proved to be a recurring constraint throughout the project: preferred TVS diodes for the power protection circuit were unavailable locally and were substituted with back-to-back BZX84C15 Zener pairs at the input and BZX84C12 at the output. The XL4016 buck converter was chosen as a locally available alternative to the preferred LTC3780 for the Branch 2 Jetson supply.

Mechanical design (WP3). A complete chassis was designed from scratch in SolidWorks, departing from the Tarmo5 reference design to accommodate the specific component set. Dedicated mounting locations were designed for the Jetson Orin Nano, ESP32 DOIT DevKit V1, MPU6050 IMU, US1881 Hall-effect sensor and its magnet disc, D3542 brushless motor and WSDT-60A ESC, 3S LiPo battery (2200 mAh, XT60), and MG995 steering servo. The IMU mount incorporates vibration-dampening

compliance to reduce high-frequency noise from motor switching and wheel impacts coupling into the accelerometer. The Hall-effect sensor mount positions the US1881 at a fixed, calibrated air gap from the four-magnet disc on the wheel hub, with the disc designed for alternating N-S-N-S polarity at 90-degree spacing on a 60 mm diameter circle. The chassis integrates the acquired 80 mm suspension geometry and accounts for motor and driveshaft load distribution. Karam led the SolidWorks design and FDM fabrication; Ashraf led the electronics, firmware, and power architecture.

3D printing and fabrication (WP4). FDM printing used PLA+ for structural parts requiring rigidity (main chassis plate, motor mount, ESC bracket, Jetson tray) and TPU 98A for flexible interfaces (vibration isolators at IMU and camera mounts, bump stops). Print parameters were tuned iteratively, with minor dimensional accuracy revisions across approximately three print cycles; no full-restart failures occurred. The perfboard for the Branch 2 power protection circuit was sized at 70 × 90 mm to accommodate the P-MOSFET reverse-polarity protection, EMI filter capacitors, and XL4016 buck converter module.

ESP32 firmware (WP5). The complete firmware was written in C++/Arduino targeting the ESP32-D0WD-V3 (DOIT DevKit V1, 240 MHz dual-core, 520 KB SRAM), organised as independently bench-verified, non-blocking modules multiplexed in a single loop() without any blocking delay() calls after setup() completes. The Bluetooth controller link uses the XboxSeriesXControllerESP32_asukiaaa library; the IMU uses MPU6050_tockn; PWM is generated via the ESP32 LEDC peripheral at 50 Hz, 16-bit resolution on both ESC and servo channels.

The **ESC arming sequence** writes ESC_NEUTRAL_US to the ESC signal pin before any other output and holds it for three seconds, satisfying the ESC's power-on neutral-detection window and structurally preventing any reverse pulse from reaching the ESC during arming:

```
void armESC() {
  if (DEBUG_HUMAN) Serial.println("Arming ESC at NEUTRAL...");
  writeEscUs(ESC_NEUTRAL_US);
  delay(3000);
  if (DEBUG_HUMAN) Serial.println("ESC armed.");
}
```

The **safe-start gate** holds the vehicle at neutral after the controller connects until both RT and LT are confirmed below TRIG_DEAD_ZONE, preventing a motor launch if a trigger is held during the BLE pairing handshake. The gate re-arms automatically on every reconnection:

```
if (!readyToDrive) {
  currentThrottle = ESC_NEUTRAL_US;
  writeEscUs(ESC_NEUTRAL_US);
```

```
  static bool hinted = false;
  if (rt <= TRIG_DEAD_ZONE && lt <= TRIG_DEAD_ZONE) {
    readyToDrive = true; hinted = false;
    if (DEBUG_HUMAN) Serial.println("Ready to drive.");
  } else if (!hinted && DEBUG_HUMAN) {
    Serial.println("Triggers held - release both to arm.");
    hinted = true;
  }
  return;
}
```

The **controller-disconnect fail-safe** counts consecutive disconnected loop iterations and, after DISCONNECT_LIMIT = 200 loops (~75 ms at operating speed), forces currentThrottle and the ESC output to neutral and clears readyToDrive so the safe-start gate must be re-satisfied on reconnection:

```
} else {
  disconnectCount++;
  readyToDrive = false;
  if (disconnectCount > DISCONNECT_LIMIT) {
    currentThrottle = ESC_NEUTRAL_US;
    writeEscUs(ESC_NEUTRAL_US);
  }
}
```

A known electrical limitation was identified during steering tests: the servo's inrush current, returning through the shared ESP32 ground reference rather than directly to the ESC's BEC ground, caused brief ground-bounce events that the ESC interpreted as below-neutral (reverse) pulses. This was characterised by observing the Thr: output remain at 1500 while the motor briefly twitched into reverse during hard steering inputs. The root cause was diagnosed as the servo ground return path passing through the ESP32 ground node and the thin ESC signal-ground wire, momentarily shifting the ESC's signal reference. The complete hardware fix — relocating the servo's ground return wire from the ESP32 GND pin to the ESC's BEC ground pin — was identified and documented; the firmware

mitigation of holding ESC_NEUTRAL_US at exactly 1500 µs (not 1499) was applied to reduce the margin at which noise crosses into the reverse band.

USB serial telemetry (WP6). The ESP32 transmits a single JSON object per line over its USB-to-serial bridge at 115200 baud, every 50 ms (20 Hz). The Jetson reads this stream from /dev/ttyUSB0 or /dev/ttyACM0 — the same USB-C cable used for flashing, eliminating the need for a dedicated UART pair. The DEBUG_HUMAN compile-time flag gates all human-readable Serial.println() calls in setup() and loop(); when false, the serial port carries only clean JSON. This flag must be set to false before the cable is plugged into the Jetson, since the FastAPI server's json.loads() parser raises an exception on any non-JSON line, which would interrupt the telemetry stream. The flag is set to true during all bench development to preserve human-readable diagnostic output in the Serial Monitor:

```
#define DEBUG_HUMAN true   // set FALSE before connecting to Jetson


// In setup():
if (DEBUG_HUMAN) Serial.println("Waiting for Xbox controller...");


// In loop():
if (DEBUG_HUMAN) {
  Serial.print("RT:"); Serial.print(rt);
  Serial.print(" Thr:"); Serial.print(currentThrottle);
  Serial.print(" Spd:"); Serial.println(speedKmh, 2);
}
```

SD-card Black Box (WP7). The Jetson-side logger records each incoming telemetry entry with a synchronised system timestamp to the micro SD card, producing a time-indexed CSV log of all seven telemetry fields per packet at 20 Hz. File structure, write latency, and timestamp synchronisation were confirmed functional in bench tests. Cross-referencing these logs against AI pipeline detections and CARLA simulation outputs is scheduled for the track integration phase, where the primary analysis axes are speed calibration against physical measurement, braking characterisation against the expected deceleration g-level from the IMU, and sim-to-real yaw-rate consistency between CARLA steering inputs and the MPU6050 AngleZ derivative.

Bench-level integration testing (WP8). With chassis assembly pending print completion, all electronics were validated in isolation on the bench. The brushless motor ran at approximately 34,000–39,000 RPM unloaded at full throttle (ESC_FWD_MAX_US = 1600 µs, 3S LiPo at 11.1 V nominal, D3542 1000 KV: theoretical no-load ≈ 11,100 RPM at full voltage, measured higher due to ESC advance timing and unloaded conditions). All firmware modules were individually validated as described in Section 4.1. USB serial telemetry was confirmed stable with no observed packet loss during a sustained 10-minute bench run at 20 Hz, producing approximately 12,000 consecutive JSON packets

without a parse error on the Jetson side. SD-card logging was validated against a 60-second bench run, confirming all 1,200 entries were written and timestamped correctly with no gap larger than 3 ms between consecutive records.

4.3 Department-Specific Contributions

The project was developed as a multidisciplinary capstone system involving Software, AI, and Mechatronics contributions. From the Software Engineering perspective, the main responsibility was to transform external telemetry and AI indicators into a complete driver-monitoring software pipeline. The software subsystem did not replace the AI or Mechatronics work; instead, it integrated their outputs through defined data contracts and converted those outputs into scoring, persistence, upload, administration, and analytics workflows.

Software Department Contribution

The Software Department produced the Android client and the backend-admin system. The Android client forms the live-session side of the software system, while the backend-admin system forms the historical administration side. Together, they create the complete path from telemetry intake to driver reliability scoring and administrator review.

The Phase 1 Android contribution focused on the mobile thick-client application. The Android application was implemented using Kotlin, Jetpack Compose, ViewModels, repositories, Room persistence, DTO mapping, WebSocket telemetry handling, and backend API contracts. Its main technical contribution is that it performs live session processing locally on the device. During a driving session, the Android client receives structured telemetry, maps it into domain telemetry packets, calculates the Driver Reliability Score, detects safety events, detects escalation patterns, stores score-history points, and saves the completed session locally.

A major part of the Android contribution was the Driver Reliability Score pipeline. The scoring engine was implemented as deterministic domain logic rather than as UI-level logic. It calculates continuous penalties for sustained risk signals, event penalties for discrete unsafe behaviours, and escalation penalties for repeated or combined risk patterns. This design makes the scoring behaviour explainable and testable. It also allows the software to show how the final score was produced instead of presenting the score as an unexplained number.

The Android side also contributed offline-first persistence. Room and SQLite were used to store driver data, session summaries, safety events, escalation records, score-history records, vehicles, and synchronization metadata. This allows the application to preserve completed sessions even if backend connectivity is unavailable after a trip. The synchronization workflow can later upload completed sessions to the backend once the connection is available.

The Android integration layer contributed communication boundaries with the external telemetry source, Jetson/vehicle-control interface, and backend APIs. WebSocket handling supports the real-time telemetry stream, while HTTP API contracts support completed-session upload and backend interaction. The Android client therefore acts as the software bridge between live telemetry and persistent backend records.

The Phase 2 backend contribution focused on the Spring Boot administration system. The backend was implemented using Java, Spring Boot, Spring MVC, Spring Security, Spring Data JPA, Hibernate, Thymeleaf, MySQL, validation services, command services, query services, and analytics services. Its main technical contribution is authenticated ingestion, historical persistence, protected administration, and fleet analytics.

The backend upload endpoint receives completed-session payloads from an authenticated API client. The upload service validates the request, checks driver and vehicle references, handles duplicate sessions, stores the session summary, stores child records such as events, escalations, score-history points, and ingestion issues, updates driver aggregate fields, and returns a structured upload response. This endpoint is the main connection between the Android scoring process and the backend historical database.

The backend also contributed secure administrator access. Administrator pages and protected routes require authenticated access. The backend separates administrator login from machine-to-machine upload authentication. This distinction is important because the permission to upload completed sessions is not the same as the permission to manage drivers, vehicles, sessions, and settings.

The backend administrator interface includes dashboard views, driver management, vehicle management, session lists, session details, driver analytics, fleet analytics, settings, and profile pages. These pages allow administrators to review uploaded completed sessions and inspect supporting evidence such as penalty totals, events, escalations, score history, validity status, and upload-processing status.

The backend analytics contribution includes average final score, total fleet distance, daily score trends, event-type counts, escalation-type counts, driver status counts, vehicle status counts, session status counts, invalid-session rate, warning-session rate, completed-session rate, active-driver share, and busy-vehicle share. These analytics summarize uploaded historical records without recomputing the Android-generated session score.

AI Department Contribution

Halit Barboti and Emad Alden Aleassa were both responsible for the in-cabin AI subsystem. Their work covered the implementation and integration of the face detector, drowsiness detector, emotion recognition module, and head-pose estimation module. The responsibility was shared between both members because the modules are connected inside one driver-monitoring pipeline and depend on the same camera input and face-detection stage.

The main AI contribution was the development of a modular perception system that converts in-cabin camera frames into driver-state indicators. The face detector provides the detected driver face region. The drowsiness detector identifies fatigue-related behavior using EAR, MAR, CNN eye-state classification, and temporal logic. The emotion classifier detects sadness and anger as distress-related indicators while mapping other emotion outputs to neutral. The head-pose estimator detects whether the driver's head orientation suggests possible distraction.

Halit Barboti contributed to in-cabin AI module implementation, the face-detection pipeline, drowsiness pipeline support, emotion and head-pose integration support, AI output preparation for the Jetson-hosted communication layer, and testing and documentation of the in-cabin subsystem, representing 25% of the AI team effort. Emad Alden Aleassa contributed to in-cabin AI module implementation, drowsiness detection logic, EAR and MAR-based fatigue detection, emotion model integration, head-pose module support, threshold tuning, and testing and documentation of the in-cabin subsystem, also representing 25% of the AI team effort.

The Road Monitoring subsystem consisting of Rama Tamimi and Joud Wardeh was responsible for the external camera pipeline running on the NVIDIA Jetson Orin Nano. This contribution covers the full pipeline from CARLA simulation, raw IMX219 camera frame acquisition through YOLOPv2 inference, HSV-based lane deviation estimation, TensorRT optimisation, and deviation score output. The work also includes the physical construction of the test track, camera calibration, and the autostart configuration for unattended vehicle operation.

The system was evaluated using a custom physical test track built from cardboard with yellow tape lane markers, designed to provide high contrast for reliable lane detection. A Jetson Orin Nano with an IMX219 CSI camera was integrated using a GStreamer pipeline. YOLOPv2 was deployed on the device in TensorRT format and successfully generated real-time segmentation outputs, although maximizing its accuracy was not possible. An attempt was made by creating a small custom dataset was collected from the physical track, but fine-tuning was not possible due to the limitations of the exported TorchScript model.

To address this, a custom HSV-based LaneDeviationEstimator was developed, using color filtering, ROI cropping, histogram analysis, and peak detection to compute a stable lane deviation score with smoothing and fallback handling for partial or missing detections. To achieve real-time performance, the model was optimized using TensorRT FP16 conversion, improving inference speed from 8–12 FPS to approximately 19 FPS on average. Finally, the system outputs were made accessible through a lightweight file-based interface, and both annotated and raw videos were recorded for analysis.

Another contribution made by Joud Wardeh and Rama Tamimi is simulating the logic on CARLA. Such as the simulation environment, the camera-to-YOLOPv2-to-lane-deviation pipeline, vehicle telemetry (speed/acceleration), the hybrid obstacle-detection system, and the traffic/weather test scripts. The resulting JSONL telemetry stream (lane_deviation, speed_kmh, acceleration_mps2, obstacle_detected, obstacle_distance_m, obstacle_type) is the interface this sub-team provides to the rest of the system, and an optional WebSocket client allows the same telemetry to be forwarded live to a Jetson-based component for further processing by other parts of the project.

Mechanical design and fabrication (Karam)

A complete vehicle chassis was designed in SolidWorks. The center chassis structure, along with the dedicated mounts for the IMX219 camera and the Jetson Orin Nano, was modeled entirely from the ground up rather than adapted from any existing platform. The model provides dedicated mounting locations for every major component. Two design considerations were treated as first-order: vibration management (dampening measures at the IMU and sensor mounts to reduce noise in harsh-braking and aggressive-turning telemetry) and mechanical load handling (motor/driveshaft load distribution, 80 mm suspension integration, and wheel-hub interfaces for the Hall-effect magnet disc). FDM printing used PLA+ for structural parts and TPU 98A for flexible interfaces. No full-restart print failures occurred. Karam also specified and is sourcing the dedicated voltage regulator (< 0.5 V fluctuation requirement under peak ESC/motor load).

ESP32 firmware (Karam and Ashraf). The complete C++/Arduino firmware was developed collaboratively across all modules, with the two members dividing and cross-checking the work. Karam led the motor and steering control path: brushless motor PWM control via the 60 A ESC with a safe-start gate and proportional reverse/brake, and MG995 servo steering mapped from the Xbox controller joystick. Ashraf led the sensing and communication path: MPU6050 IMU acquisition with gyro calibration and accelerometer offset auto-zero, interrupt-driven Hall-effect speed sensing with speed computation in km/h, and the 20 Hz JSON telemetry stream to the Jetson over USB serial matching the shared schema. The remaining modules — the two LED turn signal outputs (LB/RB) and the controller-disconnect fail-safe — were implemented jointly. The heading-assist P-controller (IMU yaw feedback loop for straight-line holding and commanded-angle turns) was prototyped and validated but subsequently removed when LB was repurposed for the left turn signal.

Embedded electronics, firmware verification, and logging (Ashraf). Ashraf conducted bench-level verification of the sensor hardware and the serial communication link, confirmed the USB serial packet integrity, and implemented and validated the SD-card Black-Box logger on the Jetson side, confirming log file structure and timestamp format.

Shared work. Both members handled component procurement and bench-level integration testing. Rather than building every subsystem in isolation, the team studied a range of existing RC and embedded-control designs, then modified, reverse-engineered, and adapted the relevant parts — electronics layouts, firmware control patterns, and integration approaches — to fit the specific requirements of this platform, integrating them into the custom-designed chassis. The concrete deliverables are: the custom SolidWorks chassis design and its 3D-printed physical realization; the complete modular ESP32 firmware suite; the USB serial JSON telemetry link and packet protocol; the SD-card Black-Box ground-truth logger; two turn signal LEDs with firmware integration; and the isolated power architecture (voltage regulator in procurement).

AI Department Integration Contribution

The AI Department contributes the perception and behavioural-risk indicators used by the software subsystem. From the software perspective, the AI output is treated as structured telemetry rather than as raw camera data. The key AI-related fields consumed by the software include drowsiness score, emotional-risk value, and lane-deviation value. These values are received by the Android client through the telemetry integration boundary and are used as inputs to the Driver Reliability Score model.

The software subsystem does not train the AI models or define the internal computer-vision implementation. Instead, it expects the AI subsystem to provide numerical indicators in an agreed format. This separation allows the AI implementation to improve independently while keeping the Android scoring and backend ingestion contracts stable.

AI integration is important because the Driver Reliability Score depends on behavioural-risk values that cannot be produced by ordinary backend administration alone. For example, drowsiness values support fatigue penalties and high-fatigue events. Lane-deviation values support lane-deviation penalties, lane-departure events, fatigue-with-lane-deviation escalations, and prolonged lane-drift escalations. Emotional-risk values support emotional-distress event detection. The Android client converts these AI outputs into explainable scoring artifacts.

Mechatronics Department Integration Contribution

The Mechatronics department contributes the physical vehicle platform, sensor-side telemetry, and vehicle-state interaction layer. From the software perspective, the Mechatronics subsystem provides structured vehicle-related values — speed, acceleration and motion behaviour, steering angle, turn-signal state, ignition state, and stationary state — packaged as a JSON telemetry stream transmitted at 20 Hz over USB serial from the ESP32 to the Jetson Orin Nano, and forwarded by the Jetson's FastAPI server to the Android client and scoring pipeline via WebSocket.

The software subsystem does not implement the mechanical assembly, motor-control firmware, sensor mounting, or physical power design. Instead, it consumes the resulting structured vehicle telemetry through the integration boundary. This allows the Android client to use vehicle values in scoring and session calculations without depending on the internal hardware implementation. The field-for-field JSON schema was agreed between the Mechatronics and Software departments as the formal interface contract early in the project, so both sides could develop independently against a stable specification and integration required no field-name translation or post-processing adapter.

Mechatronics integration is important for the software scoring model because several score components require vehicle-state data. Speed values are used for speed conversion, distance calculation, and speed-derived deceleration estimation. Deceleration, derived from the ESP32's MPU6050 accelerometer fore/aft channel (accelFwdMps2), is used for harsh-braking detection and

braking-aggression penalties. Obstacle distance is stored and displayed as part of session telemetry and is designed to support future risk logic once a ranging sensor is fitted; the current placeholder value of 999.0 m is a documented, schema-compliant sentinel that the scoring pipeline handles without modification. Turn-signal and vehicle-state values (turnSignal, ignitionOn, stationary) represent driving context and support future improvements to event classification and session boundary detection.

Jetson Orin Nano bring-up and environment provisioning. In addition to the firmware and sensor work, the Mechatronics subsystem was responsible for the full bring-up of the Jetson Orin Nano, which serves as the shared compute platform consumed by the AI and Software departments. This work included flashing the Jetson with the official NVIDIA JetPack distribution (Ubuntu 20.04 LTS base, L4T), configuring the system for headless and networked operation, and validating the IMX219 camera module (ribbon cable, device-tree overlay, and GStreamer pipeline) that the AI perception pipeline depends on. The Jetson's USB serial port was confirmed readable at /dev/ttyUSB0 at 115200 baud, and the port permissions were configured (dialout group) so the FastAPI server process could open the port without elevated privileges.

This bring-up contribution is significant because the Jetson is not a plug-and-play device: flashing requires NVIDIA SDK Manager running on a host Ubuntu machine, the device must enter recovery mode via a physical USB-C connection to the host, and the post-flash configuration involves package installation, camera device-tree verification, and network setup. Completing this work on behalf of all three departments ensured that the AI team could focus on model development and the Software team could focus on the Android client and backend, with the shared platform already operational.

Overall System Integration

The complete system integrates the departments through a staged data flow. The AI and Mechatronics subsystems produce structured telemetry and risk indicators. The Android client receives those values, maps them into domain telemetry packets, performs local scoring, detects events and escalations, stores the completed session locally, and prepares the upload payload. The backend receives the completed session, authenticates the upload, validates the structure, stores the records in MySQL, and presents the results through protected administrator pages and analytics dashboards.

This integration preserves a clear responsibility boundary. AI is responsible for perception-related risk indicators. Mechatronics is responsible for the physical vehicle platform and vehicle telemetry. The Android client is responsible for live scoring and local session production. The backend is responsible for completed-session ingestion, historical storage, administration, and analytics. This division prevents the project from becoming a single tightly coupled system and makes each department's contribution easier to test, explain, and improve.

The main technical result of the combined work is an end-to-end driver-monitoring workflow. A driving session begins with live telemetry, continues through Android scoring and local persistence, ends with completed-session upload, and becomes available for backend administrator review. This shows how the department-specific contributions combine into one fleet driver monitoring system rather than remaining as isolated subsystems.

4.4 Integration and System Assembly

The Mechatronics subsystem contributes to system integration at two levels: the physical vehicle platform and its firmware telemetry output, and the Jetson Orin Nano bring-up that underpins every other department's runtime environment.

At the vehicle level, the ESP32 firmware packages seven telemetry fields into a single JSON object per line, transmitted at 20 Hz over USB serial to the Jetson. The Jetson's FastAPI server reads this stream from /dev/ttyUSB0 or /dev/ttyACM0 and forwards it over WebSocket to the Android client. The integration boundary is the JSON schema itself, agreed between the Mechatronics and Software departments as a formal contract before either side began implementation. Because the schema was fixed early, the Software team could develop its telemetry mapper and scoring engine against a stable specification, and the Mechatronics firmware could evolve independently as long as the field names, types, and units did not change. No adapter layer or field-name translation was required at integration time.

The seven fields in the ESP32 contract and their firmware origins are as follows. speedKmh is derived from the US1881 Hall-effect interrupt counter using a 500 ms windowed pulse-count calculation: $v = (\text{pulses} / \text{PULSES\_PER\_REV}) \times C / \Delta t \times 3.6$, where $C = \pi \times 0.130$ m. accelerationMps2 is the MPU6050 fore/aft accelerometer channel corrected for resting-offset bias and scaled from g to m/s^2 as $a_{fwd} = \text{ACCEL\_SIGN} \times (a_{x,raw} - a_{x,offset}) \times 9.81$. steeringAngleDeg is a commanded value back-calculated from the last servo pulse written: $\theta = (\text{PW\_last} - \text{PW\_center}) / \text{STEER\_MAX\_DELTA} \times \text{STEER\_FULL\_DEG}$. obstacleDistanceMeters is a documented placeholder constant (999.0) reflecting the absence of a ranging sensor in this iteration; the field is present in every packet so the Software scoring engine requires no conditional handling. turnSignal is mapped to the Xbox controller's bumper buttons (LB/RB) and reflected in the outgoing packet. ignitionOn is always true while the firmware is running and the ESC is armed. stationary is set true when $\text{speedKmh} \leq 0.5$, providing a low-cost vehicle-stopped indicator derived from the Hall sensor without requiring a dedicated state machine.

One integration challenge specific to the Mechatronics boundary was the DEBUG_HUMAN compile-time flag. During bench development, human-readable Serial.println() startup messages and diagnostic output are essential. However, the Jetson FastAPI server's json.loads() parser raises an exception on any non-JSON line, which silently breaks the telemetry stream. The solution was a single #define DEBUG_HUMAN flag that gates every non-JSON Serial output in setup() and loop(). Setting this flag to false before connecting the USB cable to the Jetson produces a port that carries only clean JSON, and the parser receives every packet without error. This was validated by running a 10-minute bench telemetry session and confirming zero parse errors on the Jetson side across approximately 12,000 consecutive packets.

A second integration challenge was ground-reference noise coupling between the steering servo and the ESC signal line. During bench integration testing, hard steering inputs produced brief reverse twitches in the motor, caused by the servo's inrush current returning through the shared ESP32 ground node and momentarily shifting the ESC's signal reference below neutral. This was characterised by observing the Thr: output hold at 1500 µs on the Serial Monitor while the motor briefly reversed, confirming the disturbance was electrical rather than a firmware logic error. The firmware mitigation — holding the idle pulse at exactly 1500 µs rather than 1499 — reduced the margin at which noise crosses into the reverse band. The complete hardware fix (relocating the servo ground return to the ESC's BEC ground pin) was identified, documented, and is scheduled for the next assembly iteration.

At the platform level, the Mechatronics subsystem was responsible for the complete bring-up of the Jetson Orin Nano that all three departments depend on. This work went significantly beyond cable connection. Flashing the Jetson required running NVIDIA SDK Manager on a host Ubuntu machine, placing the device in USB recovery mode via a physical jumper, and executing a full JetPack flash that installs the L4T Linux base, CUDA, cuDNN, TensorRT, and the NVIDIA multimedia stack. Post-flash configuration included setting up a user account, configuring the network for both local (Jetson-to-

Android) and internet (package installation) access, verifying the IMX219 camera module via device-tree overlay and a GStreamer pipeline test, and configuring USB serial port permissions so the FastAPI server process could open /dev/ttyUSB0 without elevated privileges (dialout group membership).

The in-cabin AI subsystem was integrated as a processing layer inside the overall driver monitoring system. The AI pipeline receives frames from the in-cabin camera, processes the driver's face, and outputs structured indicators. These outputs are then made available to the Jetson-hosted WebSocket and API system implemented by the software team.

The integration design separates AI perception from software scoring. The AI subsystem does not directly calculate the final Driver Reliability Score. Instead, it produces reliable driver-state values that the software layer can use in its scoring logic. This separation improves modularity because the software team can adjust the scoring rules without changing the AI detection modules.

The AI-to-software output is designed as processed data, not raw video. This supports privacy and reduces communication overhead. Instead of transferring driver-facing frames, the AI layer provides values such as drowsiness status, emotion status, head-pose status, face presence, FPS, and timestamp. An example AI output structure is shown below:

json

```
{
  "timestamp": 1710000000.25,
  "face_present": true,
  "drowsiness_level": 2,
  "drowsy_event": true,
  "emotion_label": "Anger",
  "emotion_confidence": 0.85,
  "distress_event": true,
  "head_pose_status": "distracted",
  "fps": 30.0
}
```

The Road Monitoring pipeline is connected to the rest of the system using a simple file-based interface, which keeps it fully independent from other components. The lane detection runs as a separate process and does not depend on how or when its output is used.
The lane deviation value is continuously written to /tmp/lane_deviation.txt for every frame. A bridge module (ai_inference.py) reads this file and provides the latest value to the fleet server.
The server reads this value every 50 ms in a background thread and includes it in the telemetry data sent to clients. This setup avoids the need for sockets, shared memory, or direct communication between processes.
Because of this design, both the lane detection pipeline and the server can run, restart, or fail independently. If the file is not available during startup or temporary failures, the system safely returns a default value of 0.0 to prevent errors.

The WebSocket and API host were programmed by the software team on the Jetson Orin Nano. The AI department ensured that the detection results could be represented in a format suitable for that integration. This approach allowed the AI team to focus on detection accuracy and module reliability, while the software team handled communication, application logic, and final scoring integration.

The software integration timeline followed three main implementation milestones. Phase 1, completed on 29 April 2026, established the Android client as the local scoring and session-production component. This included the driver-facing workflow, telemetry mapping, deterministic scoring logic, local persistence, event and escalation handling, and completed-session preparation. Phase 2, completed on 15 May 2026, established the Spring Boot backend-admin system as the authenticated ingestion, historical storage, and administrator-review component. This included protected

administrator access, driver and vehicle management, completed-session upload ingestion, MySQL persistence, session review, and analytics pages. The final integration milestone took place on 15 June 2026, when the Android client, Jetson-hosted telemetry service, AI outputs, Mechatronics telemetry, and backend upload path were connected into one end-to-end workflow. This chronology reflects the intended dependency order: the Android client first produced scored session records, the backend then provided the storage and review authority for completed sessions, and the final integration connected both phases with the vehicle-side telemetry and AI outputs.

The complete system was then assembled by connecting the AI, Mechatronics, Android, and backend components through defined interfaces. The purpose of integration was to ensure that the output of one subsystem could be consumed reliably by the next subsystem without requiring each department to understand the internal implementation of the others. From the Software Engineering perspective, the final integrated system follows a staged pipeline: external telemetry generation, Android live-session processing, local session persistence, backend upload, backend historical storage, and administrator review.

The first integration point is between the AI and Mechatronics subsystems and the Android client. The AI subsystem produces behavioural-risk indicators such as drowsiness, emotional-risk value, and lane-deviation value. The Mechatronics subsystem provides vehicle-related telemetry such as speed, acceleration or motion behaviour, , turn-signal state, and vehicle-state information. These values are exposed to the Android client as structured telemetry packets. The Android client does not process raw camera frames or raw mechanical sensor signals directly. Instead, it depends on the agreed telemetry contract.

The telemetry contract was one of the most important integration decisions. The Android client expects fields such as timestamp, speed, acceleration, drowsiness score, emotional-risk value, lane deviation, and turn-signal state. By using structured JSON telemetry, the software subsystem can remain independent from the internal AI model implementation and the internal hardware-control implementation. This means that changes inside the AI or Mechatronics subsystem can be handled at the telemetry boundary rather than requiring a rewrite of the scoring engine.

The second integration point is inside the Android client. Once telemetry is received, the Android telemetry mapper converts the incoming transport-level values into domain-level telemetry packets. These packets are passed to the active-session and scoring logic. The scoring engine updates the Driver Reliability Score, continuous penalties, event penalties, escalation penalties, distance calculation, duration calculation, and score-history points. The result is then persisted locally using Room. This internal assembly connects networking, domain logic, persistence, and UI state while keeping them separated into maintainable layers.

The third integration point is between the Android client and the backend-admin system. This integration occurs after a driving session is completed. The Android client prepares a completed-session upload payload containing the session summary, final score, penalty totals, duration, distance, safety events, escalation records, score-history points, validity state, and synchronization metadata. The backend receives this payload through an authenticated upload API. This design avoids sending every raw telemetry packet to the backend during the live session and instead transfers the completed historical result after the Android scoring pipeline has finished.

The backend upload endpoint authenticates the API client, validates the request structure, checks duplicate sessions, verifies driver and vehicle references, maps the uploaded data into backend records, and persists the session and its child artifacts in MySQL. The upload is handled transactionally so that the session summary and its related records remain consistent. If the upload cannot be accepted, the Android client can preserve the local session and retry later.

The fourth integration point is between backend persistence and the administrator interface. Once uploaded session data is stored, backend query services prepare the information for administrator

pages. The dashboard, driver pages, vehicle pages, session list, session detail page, driver analytics, and fleet analytics views all read from the stored historical records. This allows the backend to present the completed driver-monitoring result as an administrative review system rather than as a raw data dump.

Several integration challenges were encountered during system assembly. The first challenge was maintaining consistent field meanings across subsystems. A value such as lane deviation can be interpreted either as a normalized AI output or as a physical measurement. To reduce ambiguity, the software scoring model treats such values through explicit thresholds and documents the interpretation used in the formulas. Similarly, speed is converted into meters per second before it is used for braking and distance calculations.

The second challenge was timing consistency. Telemetry packets depend on timestamps, and invalid timestamp order can produce incorrect penalty growth. The Android scoring model therefore applies a bounded time-step rule. If a timestamp is invalid or does not increase, the time step is treated as zero. If a time gap is too large, the continuous-penalty time step is capped. This prevents malformed timing data from unfairly reducing a driver's score.

The third challenge was connectivity reliability. The Android client cannot assume that backend connectivity will always be available during or immediately after a driving session. The solution was an offline-first design. Completed sessions are stored locally first, together with their events, escalations, score history, and synchronization state. Upload to the backend is treated as a later synchronization step. This prevents completed sessions from being lost when a network failure occurs.

The fourth challenge was preserving historical truth after upload. If the backend recalculated the final score using different code, thresholds, or assumptions, the uploaded session could change after the fact. To avoid this, the backend does not recompute the Android-generated final score, penalty totals, duration, distance, events, escalations, or score history. It validates and stores the uploaded result as the completed session record. Backend analytics then summarize stored records rather than replacing them.

The fifth challenge was access separation. The system needs both human administrator access and machine upload access, but these are not the same type of permission. The backend therefore separates administrator session login from authenticated API-client upload. Administrators can access protected pages for reviewing and managing the system, while upload clients can submit completed sessions through the upload endpoint without receiving administrator privileges.

The assembled system therefore operates as one integrated workflow. AI and Mechatronics provide structured telemetry. The Android client receives the telemetry, performs local scoring, detects events and escalations, stores the completed session, and prepares the backend upload. The Spring Boot backend authenticates the upload, stores the completed session in MySQL, and exposes the result through protected administrator pages and analytics. This integration creates the complete software-supported driver-monitoring pipeline required by the project.

4.5 Challenges and Solutions

One challenge was distinguishing normal blinking from real drowsiness. A single blink naturally causes the Eye Aspect Ratio to drop, which could lead to false alerts if the system reacts to only one frame. This was solved by using temporal persistence. The system requires the eyes to remain closed for 20 consecutive frames before producing a drowsiness event.

Another challenge was the limitation of using only geometric landmark measurements. EAR and MAR can become unstable when the lighting is poor, the driver turns their head, or the camera view is partially blocked. To reduce this problem, the drowsiness module combines EAR with a CNN-based

eye-state classifier. The CNN model provides an additional visual check for eye-open and eye-closed classification.

A third challenge was emotion-model reliability. Emotion recognition models can be sensitive to lighting, face angle, and neutral facial expressions. The original model provides eight emotion classes, but not all of them are useful for the project's scoring logic. The solution was to map the eight-class output into three operational categories: sad, angry, and neutral. This simplifies the output and reduces the risk of overinterpreting emotions that are not needed by the prototype.

A fourth challenge was the strong neutral bias in the emotion model. The emotion model often produced a high neutral score even when other expressions were present. To reduce this issue, the implementation applies score debiasing and smoothing over a rolling prediction buffer. This makes sadness and anger more visible when detected consistently across multiple frames.

A fifth challenge was head-pose false positives. Drivers naturally move their heads for short moments, and these movements should not immediately be treated as distraction. This was solved by using yaw and pitch thresholds together with consecutive-frame counting. The system only reports distraction when the head orientation remains outside the accepted range for multiple frames.

A sixth challenge was real-time performance on the Jetson Orin Nano. Running every model on every frame can increase latency and reduce FPS. The implementation reduces computational load using inference skipping, landmark skipping, cached model outputs, ONNX Runtime, and TensorRT acceleration where supported. This allows the subsystem to remain suitable for edge deployment while still producing useful driver-state indicators.

The seventh challenge was encountered by the road monitoring ai subsystem, which was that TensorRT 10.3 rejects ONNX due to SequenceConstruct. This came from the model grouping detection outputs into a list during export. TensorRT does not support this type of dynamic operation, so the model failed to load. The issue was fixed by modifying the ONNX graph and removing the SequenceConstruct node, exposing the three detection outputs directly as separate tensors. This allowed the model to run in TensorRT without changing its outputs.

The eighth challenge was that the lane deviation score sometimes stayed high even after the vehicle returned to the center because the previous value was being held when lane markings were lost. This was fixed by replacing the “hold last value” behavior with gradual decay. After a short grace period, the deviation now slowly decreases each frame until it returns to zero if no lane is detected. This makes the output more stable and realistic during temporary loss of lane markings.

Beyond the AI-specific challenges, the software subsystem also encountered a distinct set of implementation challenges because the final system was not a single isolated application. It included an Android thick client, a Spring Boot backend server, a Jetson-side API and WebSocket telemetry bridge, an administrator web interface, a MySQL persistence layer, upload synchronization, and monitoring through Prometheus and Grafana. The main challenge was therefore to make these software components work together as one coherent driver-monitoring pipeline while keeping each component maintainable and testable.

One major software challenge was defining clear responsibility boundaries between the Android client, backend server, Jetson service, and administrator interface. Early in the design, it was possible for responsibilities to overlap, especially around live scoring, session ownership, and backend analytics. The solution was to assign one primary responsibility to each software component. The Android client owns live session processing, scoring, local persistence, and completed-session preparation. The backend server owns authenticated upload ingestion, historical persistence, administrator access, and analytics. The Jetson-side software owns the local telemetry API and

WebSocket stream. The admin web interface owns human review and management workflows. This separation reduced coupling and made the system easier to explain, test, and extend.

A second software challenge was supporting live driver scoring without relying on constant backend connectivity. During a driving session, the Android client may be connected to the Jetson local network rather than to the public backend server. The application also cannot assume that internet access will be available at the exact moment a session ends. The solution was an offline-first Android design. The scoring engine, event detection, escalation detection, score-history generation, and completed-session storage were implemented locally. Room and SQLite were used to preserve completed sessions, events, escalations, score points, and synchronization metadata on the device. Backend upload was treated as a later synchronization step rather than as a requirement for live scoring.

A third software challenge was real-time telemetry handling. The Android application had to consume a continuous stream of telemetry while also maintaining UI responsiveness, session state, scoring state, and local persistence. The solution was to use a WebSocket-based telemetry path with structured JSON messages and asynchronous processing. Incoming telemetry is mapped into domain telemetry packets before it reaches the scoring engine. This prevents network transport details from entering the domain logic and allows the telemetry contract to remain separate from the scoring implementation.

A fourth software challenge was implementing the Jetson-side software as a stable integration bridge rather than as a simple script. The Jetson service needed to expose health and status endpoints, vehicle-state information, authority control, and a telemetry WebSocket stream. The solution was to implement the Jetson bridge as an API-oriented service with clear HTTP endpoints and a WebSocket telemetry endpoint. The service exposes state and health information to the Android client and provides a controlled way for the Android client to claim, use, and release the vehicle session. This makes the Jetson software part of the integration architecture rather than an unmanaged telemetry source.

A fifth software challenge was preventing conflicting control or telemetry sessions. In an integrated prototype, more than one client or repeated connection attempt could create confusion about whether the vehicle was available, pending, or already in use. The solution was to use an authority-state workflow. The Jetson-side API exposes vehicle and session state and supports a controlled claim process. The Android client checks availability, requests pending authority, claims the session, sends heartbeat updates during use, and releases authority when the session ends. This reduces the risk of multiple active clients treating the same vehicle as available at the same time.

A sixth software challenge was inconsistent timing and noisy telemetry. The scoring model depends on packet timestamps, speed values, lane deviation values, drowsiness values, and braking behaviour. If timestamps are invalid or if there is a large gap between packets, a continuous penalty could become unfairly large. If telemetry fluctuates from packet to packet, the score could overreact to noise. The solution was to apply timestamp validation, a bounded time step, unit conversion, and exponential moving average smoothing. Invalid or non-increasing timestamps produce a zero continuous-penalty time step, and excessive time gaps are capped. Smoothing reduces sensitivity to single-packet noise while preserving sustained risk behaviour.

A seventh software challenge was making the Driver Reliability Score explainable. A single final score without supporting details would not be useful for administrator review. The solution was to design the scoring pipeline around separate continuous penalties, event penalties, and escalation penalties. Continuous penalties explain sustained risk. Event penalties explain specific unsafe behaviours. Escalation penalties explain repeated or combined risk patterns. The Android client also stores score-history points, detected events, and detected escalations so that the backend and admin web interface can show supporting evidence rather than only the final number.

An eighth software challenge was keeping Android scoring and backend storage consistent without duplicating the same scoring logic in two places. If the backend recalculated the session score after upload, it could produce a different result because it would not have the same live runtime context as the Android client. The solution was to preserve a strict responsibility boundary. The Android client produces the completed scored session, and the backend stores the uploaded score, penalties, events, escalations, duration, distance, and score history as historical truth. The backend validates and aggregates uploaded records, but it does not silently recompute the completed session result.

A ninth software challenge was designing the completed-session upload contract. A completed session is not just one row of data. It includes the session summary, driver reference, vehicle reference, final score, penalty totals, status fields, event records, escalation records, score-history records, and ingestion issue handling. The solution was to use explicit DTOs, mapper classes, validation services, and transactional persistence. The backend upload service authenticates the API client, validates the request, checks driver and vehicle references, handles duplicate session identifiers, stores the session and child records, updates aggregate fields, and returns a structured response.

A tenth software challenge was aligning Android and backend data shapes during integration. Backend responses and upload payloads can become fragile if one side expects a primitive value while the other side sends an object, or if a list element has a different structure than expected. The solution was to standardize DTOs and mapping rules between the Android and backend layers. The project used explicit request and response models rather than relying on loosely typed JSON access. This reduced mapping errors and made future contract changes easier to locate.

An eleventh software challenge was implementing secure access for different types of users and clients. The system has human administrator access, driver-related access, and machine-to-machine upload access. These should not share the same permission model. The solution was to separate authentication paths. Administrator pages are protected through session-based login and role-protected routes. Completed-session upload is protected through authenticated API-client access. Passwords are stored using hashing rather than plain text. This separation prevents upload capability from becoming full administrator access.

A twelfth software challenge was building the administrator web interface in a way that supports review rather than only database visibility. Administrators need to understand driver history, session details, event distribution, score trends, upload status, and fleet-level summaries. The solution was to implement query-side services and Thymeleaf admin pages for dashboards, drivers, driver details, driver analytics, vehicles, sessions, session details, fleet analytics, settings, and profile. Query services prepare view models so that pages can show meaningful summaries instead of exposing raw database tables directly.

A thirteenth software challenge was backend deployment and operational visibility. A backend server can appear correct in the repository but still fail during deployment due to service configuration, process failure, port conflicts, database configuration, or runtime errors. The solution was to deploy the backend as a managed server-side application and add monitoring-oriented support. Spring Boot Actuator, Micrometer, Prometheus registry support, Prometheus, node exporter, and Grafana were used to observe server and system behaviour. Grafana dashboards provide runtime visibility, while service logs and health endpoints support troubleshooting.

A fourteenth software challenge was preserving maintainability while the project expanded. The original software focus was mainly the Android thick client and scoring model, but the final work expanded into Android, backend administration, upload APIs, Jetson integration, deployment, and monitoring. The solution was to keep the code organized into layers and contracts. Android code was separated into UI, ViewModel, orchestration, domain, data, and network responsibilities. Backend code was separated into controllers, services, repositories, mappers, validation, security, persistence, and view models. This structure made the expanded scope manageable.

Overall, the main software solution was not one isolated fix but a complete integration architecture. The Android client handles live local scoring and offline-first session production. The Jetson-side software provides API and WebSocket telemetry access. The backend server handles authenticated ingestion, persistence, security, and analytics. The admin web interface provides human review and management. Grafana and monitoring tools provide operational visibility. Together with the AI pipeline solutions on the Jetson Orin Nano, these combined solutions address the major implementation challenges across both subsystems and support the final end-to-end driver monitoring workflow.

ESC mode and the absence of a true active brake. The most consequential hardware discovery during bench testing was that the WSDT-60A ESC is in single-click forward/reverse mode, not forward/brake mode. A test pulse of 1300 μs confirmed this: the motor spun in reverse rather than braking. This ruled out the simplest braking implementation (commanding below neutral = brake) and required a different approach. The solution was to use the single-click ESC's own physics: commanding reverse against a forward-spinning motor produces a braking torque as the ESC fights the motor's momentum. Brake depth was made proportional to LT trigger pressure by mapping trigger travel directly to reverse pulse depth — light press maps near the reverse break-loose point for gentle deceleration; full press snaps to ESC_REV_MAX_US for maximum opposing torque. Critically, this snap bypasses the ramp gate entirely: the brake command overwrites currentThrottle directly, so it responds on the same loop iteration as the trigger press regardless of where the ramp had coasted to in the interim. This solved a subtle bug where lifting RT allowed the ramp to coast currentThrottle back to neutral while the car was still physically rolling, meaning a subsequent LT press would find the "still moving forward" condition false and fall into the gentle coast path instead of braking hard.

Hall-effect floating-pin spurious interrupts. The US1881 was initially wired without a hardware pull-up resistor, relying solely on the ESP32's internal pull-up. The internal pull-up proved insufficiently strong against the capacitive coupling and switching noise from the adjacent motor phase wires, producing spurious interrupt triggers that inflated the pulse count and reported grossly incorrect speeds — in some cases reporting over 80 km/h while the wheel was stationary. The fix was a 12 kΩ external pull-up resistor from the signal line to 3.3 V, with the US1881's VDD at 5 V per its datasheet specification. This reduced the signal's noise susceptibility and eliminated spurious triggering entirely, confirmed by observing a stable zero pulse count with the wheel stationary and exactly four counts per manual revolution with the wheel turning.

Bluetooth pairing state after hardware swap. When the ESP32 was replaced with a new board during development, the Xbox Series X controller failed to connect. The controller was still bonded to the previous board's BLE address and advertised only to that address rather than entering open pairing mode. The solution was to force the controller back into open pairing mode by holding the pair button on the top edge for three seconds until the Xbox logo flashed rapidly, then resetting the new ESP32 so both devices were scanning simultaneously. This was also caused by the new board having a different MAC address from the one hardcoded in CONTROLLER_MAC, confirming that the library's MAC-filtering behaviour requires this pairing reset whenever the host board changes.

Steering inversion on first integration. On first bench test with the complete wiring, left stick input produced right wheel turn and vice versa. This was traced to the physical servo horn orientation relative to the steering linkage geometry: the servo was installed in a position where the PWM-to-angle mapping was mechanically reversed. Rather than dismounting and reinstalling the servo, the fix was a single compile-time flag (STEER_REVERSED = true) that swaps the map() output endpoints: map(raw, 0, 65535, SERVO_MAX_US, SERVO_MIN_US) instead of SERVO_MIN_US, SERVO_MAX_US. This confirmed correct left/right response on the bench in one upload cycle.

IMU axis mapping empirical confirmation. Rather than deriving the axis mapping analytically from the board's physical orientation, a dedicated standalone test sketch was written that printed plain-language labels (NOSE UP, LEAN LEFT, TURNING RIGHT) alongside raw AngleX/Y/Z values. The car was moved through three physical tests — flat and level, nose up, roll to one side — and the axis that changed in each case was recorded. This confirmed that the -AngleY axis corresponds to pitch (nose up

= positive after negation), AngleX corresponds to roll, and AngleZ is yaw. This empirical approach took approximately two minutes and provided ground-truth axis mapping that cannot be wrong due to a mounting-orientation assumption, which was particularly important given that the sensor is rotated approximately -90 degrees relative to the car's forward axis.

5. Testing and Validation

5.1 Test Plan

The testing strategy focused on validating the complete software pipeline of the project: Android local scoring, Android persistence, telemetry integration, Jetson API/WebSocket communication, backend upload ingestion, backend persistence, administrator web workflows, analytics, deployment behaviour, and monitoring visibility. The test plan was software-focused. AI model accuracy, mechanical performance, physical sensor calibration, and hardware manufacturing tests were outside the direct scope of this section, except where their outputs were consumed through software integration boundaries.

The main goal of the test plan was to confirm that the software subsystem could receive structured telemetry, process it into driver-reliability results, preserve completed sessions locally, upload completed sessions to the backend, store the uploaded records correctly, and display the results through protected administrator interfaces. Testing was therefore organized into unit testing, integration testing, system testing, deployment validation, monitoring validation, and user acceptance testing.

Unit testing was planned for isolated software logic that could be verified without requiring the full system to run. On the Android side, this included the Driver Reliability Score engine, continuous penalty calculations, event detection, escalation detection, timestamp handling, speed conversion, distance calculation, telemetry mapping, local repository behaviour, credential/session handling, and synchronization-related logic. The purpose of Android unit testing was to confirm that core behaviour remained correct even before connecting to the Jetson, backend server, or physical prototype.

Backend unit and service-level testing focused on validation services, mapper logic, upload request handling, duplicate-session handling, driver and vehicle reference checks, aggregate field updates, query services, analytics calculations, and security-related behaviour. These tests were intended to verify that backend business rules worked independently from the web interface. For example, upload validation should reject invalid required fields, analytics should calculate correct averages and rates, and protected services should not expose data without the correct access path.

Integration testing was planned around the interfaces between software components. The first integration area was the Android-to-Jetson connection. The Android client needed to confirm that it could reach the Jetson API, check vehicle state, request or claim authority, open a WebSocket telemetry stream, receive structured telemetry packets, map them into domain telemetry values, and release authority after the session ended. The Jetson-side service was tested through HTTP endpoints and WebSocket behaviour rather than by inspecting AI or mechanical internals.

The second integration area was Android-to-backend upload. Completed sessions created or stored on the Android client needed to be converted into backend upload DTOs and submitted to the authenticated backend endpoint. The test plan included successful upload cases, failed authentication cases, duplicate-session cases, invalid structure cases, warning-level ingestion cases, and retry

behaviour after upload failure. A successful integration test required the backend to store the session summary and its child artifacts, including safety events, escalation records, score-history points, and ingestion issue records where applicable.

The third integration area was backend-to-admin web. After data was stored, the administrator interface needed to display it correctly. Testing covered administrator login, protected route access, dashboard loading, driver list and driver detail pages, vehicle pages, session list, session detail page, driver analytics, fleet analytics, settings, and profile workflows. These tests were important because a backend can store data correctly but still fail to present it in a usable or secure way.

System testing was planned to validate the full end-to-end workflow. A complete software system test begins with the Jetson service running and exposing its API/WebSocket endpoints. The Android client connects to the Jetson service, receives telemetry, starts a driving session, updates the reliability score, records events and escalations, ends the session, stores the completed session locally, uploads the completed session to the backend, and verifies that the administrator can view the uploaded result on the web interface. This test confirms the complete software chain rather than only one isolated component.

Deployment validation was included because the backend server was deployed as a running service rather than remaining only as local code. The test plan included checking whether the Spring Boot backend starts successfully, whether the public admin website is reachable, whether administrator login works, whether protected routes remain protected, whether the database connection is active, whether the upload endpoint accepts authenticated API-client requests, and whether service logs show normal operation after restart. This validation is important because repository-level correctness does not automatically guarantee server-level reliability.

Monitoring validation was planned for the Grafana and Prometheus side of the software stack. The backend and server environment needed to expose operational metrics through Actuator, Micrometer, Prometheus registry support, node exporter, Prometheus, and Grafana dashboards. The purpose of this testing was not to validate the Driver Reliability Score itself, but to confirm that the deployed backend environment had runtime visibility. Monitoring validation included checking service availability, server metrics, dashboard loading, and whether abnormal backend or server behaviour could be detected through logs and dashboards.

Security testing was included across the backend and admin web components. Administrator pages should not be accessible without authenticated administrator login. API upload should not be accepted without valid API-client authentication. Driver, vehicle, session, dashboard, analytics, settings, and profile pages should remain protected. Passwords should not be stored as plain text. This testing was necessary because the system handles driver-related records and safety-related session history.

User acceptance testing was planned from the perspective of the expected users of the software. For the driver-side Android workflow, acceptance testing checks whether a user can log in, start a session, receive live score updates, end a session, and retain the completed session locally. For the administrator workflow, acceptance testing checks whether an administrator can log in, create or manage drivers and vehicles, review completed sessions, inspect session details, interpret score history and events, and view dashboard or analytics information. These tests focus on whether the system is understandable and usable, not only whether individual functions execute.

The test plan also recognized remaining validation limitations. Repository-supported tests and local workflow checks can verify a large part of the software implementation, but they do not fully replace live field testing with the Android device, Jetson service, deployed backend, public network

configuration, and prototype vehicle operating together for extended sessions. Therefore, the validation strategy distinguishes between confirmed repository-supported software tests and future full-system operational tests.

The testing strategy for the Road Monitoring subsystem was manual and scenario-based rather than automated unit testing, since the goal was to evaluate the system under realistic driving conditions.. Testing was carried out at four levels: module testing, performance evaluation, integration testing, and real-world testing on the RC car.

Module-level testing ensured that each component worked correctly on its own. The camera pipeline was checked to confirm stable frame capture at 1280×720 using GStreamer. The lane deviation estimator was tested by placing the camera at known positions on the track and verifying that the output values matched expected left, center, and right conditions. The TensorRT model was validated by checking that all required outputs were correctly produced after conversion. The output file /tmp/lane_deviation.txt was also verified to ensure it was updated every frame with valid values.

Performance testing compared the frame rate of the PyTorch and TensorRT versions to measure speed improvement. The deviation output was also evaluated by manually moving the camera across different lane positions and checking if the values responded correctly.

Integration testing confirmed that the deviation value generated by the inference pipeline could be correctly read by the bridge module (ai_inference.py) independently of the fleet server. This ensured that the communication between components worked correctly.

Finally, field testing was conducted in CARLA and on the physical RC setup under multiple scenarios. For each test, the simulator was launched, the map was loaded, and traffic and pedestrians were optionally spawned. Weather conditions such as clear, rainy, and night scenarios were applied using separate scripts. The system was then run while the vehicle was driven manually, and telemetry logs, console output, and debug images were recorded. These tests ensured that lane deviation, speed, acceleration, and obstacle detection worked correctly across different environments. As for testing on the psychical RC car prototype it validated real-time performance, system startup behavior, and the correctness of recorded video outputs during actual driving conditions.

5.2 Test Results

Test ID	Test Description	Expected Result	Actual Result	Status (Pass/Fail)
T-01	CARLA connection, vehicle spawn, autopilot (async)	Vehicle spawns and drives without errors	Vehicle spawned successfully; autopilot navigated normally	Pass
T-02	YOLOPv2 inference on CARLA frames	lane_deviation score rises noticeably when drifting	Lane Deviation score rises accurately in response to drifting.	Pass
T-03	Manual keyboard control with live deviation	Stable deviation output even with manual controlled driving.	Stable deviation output.	Pass
T-04	HSV mask on yellow tape	HSV mask on yellow tape	Both edges visible and consistent under indoor fluorescent lighting	Pass

T - 05	No-detection decay	Score should decay toward 0 after grace period when no tape visible	Score decayed at rate 0.80/frame after 3-frame grace period	Pass
T- 06	TensorRT FPS improvement	improvementTRT should exceed PyTorch baseline of 8-12 FPS	15-25 FPS achieved, average: 19 FPS	Pass
T- 07	Autostart on boot	Models should launch automatically.	Window opened and models are active without manual intervention within approximately 1 minute and 40 seconds after power on.	Pass
T-08	Fine-tuning YOLOPv2 on custom dataset	YOLOPv2 weights should accept gradient updates from 200-image dataset	TorchScript format confirmed to freeze weights; fine-tuning not possible without original training code	Fail
T-09	Face detector tested on driver-facing camera frames	Driver face should be detected and bounded by a face box	The face detector located the driver face and returned the bounding box for the next modules	Pass
T-10	Face detector tested when no face is visible	System should not force emotion, drowsiness, or head-pose predictions	The system avoided unreliable predictions when no valid face was detected	Pass
T-11	Drowsiness module tested with open eyes	EAR should remain above the closure threshold and drowsiness level should remain normal	The system classified the state as normal when the eyes were open	Pass
T-12	Drowsiness module tested with normal blinking	Short eye closure should not trigger a drowsiness event	Temporal persistence prevented normal blinking from being classified as drowsiness	Pass
T-13	Drowsiness module tested with sustained eye closure	Consecutive closed-eye frames should increase the drowsiness level	The system increased the drowsiness state after sustained eye closure	Pass
T-14	Drowsiness module tested using EAR and MAR features	Eye and mouth features should contribute to fatigue detection	The combined feature extraction supported the drowsiness detection process	Pass

T-15	CNN eye-state classifier tested	Model should classify eye state as open or closed	Eye-state classification supported the EAR-based drowsiness decision	Pass
T-16	Emotion model tested with eight-class output	Model should produce one of the original emotion classes	The model produced eight-class probabilities before operational mapping	Pass
T-17	Emotion mapping tested	Sadness should map to sad, anger should map to angry, and all other classes should map to neutral	The system correctly mapped the output into sad, angry, and neutral categories	Pass
T-18	Head-pose module tested with head turning	Yaw, pitch, and roll values should change according to head orientation	The module produced orientation values that changed with head movement	Pass
T-19	Full in-cabin AI pipeline tested	Face detection, drowsiness, emotion, and head pose should run in one pipeline	The module produced orientation values that changed with head movement	Pass
T-20	Processed AI output format tested	Output should contain driver-state indicators instead of raw video	The subsystem produced processed values such as drowsiness state, emotion label, head-pose status, face presence, FPS, and timestamp	Pass
T-21	ESP32 PWM brushless motor control via 60 A ESC	Controllable speed profile	34,000–39,000 RPM unloaded; stable in 40–60% limiter range	Pass
T-22	MPU6050 IMU acquisition (I2C)	Valid 6-axis data	Verified	Pass
T-23	Hall-effect wheel-speed pulse counting	Correct pulse-to-speed	Verified	Pass
T-24	UART telemetry packet integrity	No packet loss	No loss observed in steady state	Pass
T-25	Watchdog fail-safe on UART loss	Motor PWM cut	Verified	Pass
T-26	SD-card Black Box logging	Timestamped entries	Confirmed functional	Pass
T-27	Steering calibration (commanded vs physical)	≤ target error	[pending assembly]	Pending

T-28	Power stability under peak load	< 0.5 V fluctuation	[pending regulator integration]	Pending
T-29	Phase 1 debug APK build	The Android application compiles and packages successfully.	assembleDebug completed successfully in 18 seconds.	Pass
T-30	Phase 1 scoring engine unit tests	Rule-based scoring behavior is validated by unit tests.	ScoringEngineTest passed.	Pass
T-31	Phase 1 telemetry mapper unit test	Telemetry DTOs map into domain packets with expected field semantics.	TelemetryMapperTest passed.	Pass
T-32	Phase 1 sync manager unit tests	Upload, skip, and mark-synced behavior works as expected.	SyncManagerTest passed.	Pass
T-33	Phase 1 login credential policy tests	Login policy behavior aligns with the final accepted policy.	Policy test passed.	Pass
T-34	Phase 2 admin authentication	Only administrators access protected pages and APIs.	Login and authorization worked as expected.	Pass
T-35	Phase 2 driver and vehicle management	Admins manage driver and vehicle records.	Create, update, list, and detail flows worked.	Pass
T-36	Phase 2 session upload ingestion	Authenticated clients upload ended sessions.	Session summaries and child records were persisted.	Pass
T-37	Phase 2 session and analytics review	Admins view sessions, details, dashboard data, and analytics.	Dashboard, detail, fleet analytics, and driver analytics views worked.	Pass
T-38	Admin-only driver creation	Driver accounts are created through the backend/admin website rather than mobile self-signup.	Driver creation was handled through the admin-side backend flow.	Pass
T-39	Driver backend login from phone	The driver can log in from the Android application while connected to the internet.	The Android client authenticated the driver against the backend login boundary.	Pass
T-40	Hybrid thick/thin-client operation	Backend is used for identity and upload, while live	The phone performed local computation and prepared completed-	Pass

		scoring remains local.	session upload artifacts.	
T-41	Jetson local access-point connection	The Android app connects to the vehicle-side Jetson access point and communicates with the vehicle-control host.	The local vehicle communication path was implemented through the Jetson access-point/API host design.	Pass
T-42	WebSocket telemetry communication	Telemetry is sent and received through the WebSocket stream during the live-session flow.	The WebSocket telemetry boundary was implemented for live driving data exchange.	Pass
T-43	Lease and heartbeat recovery Vehicle	ownership should not stay locked forever after crash or disconnection.	The reservation, heartbeat, and lease-expiry model supports recovery from abnormal termination	Pass
T-44	Public backend/admin website deployment	The backend admin website is reachable through the configured public server address.	The website was deployed through the public IP/website link.	Pass
T-45	Grafana monitoring dashboard	The monitoring stack displays backend/server health information.	Grafana dashboard access and monitoring setup were completed.	Pass

Table 1: Test Results

5.3 Performance Evaluation

Success Criterion	Target Value	Achieved Value	Met? (Yes/No/Partial)
SC-1: TensorRT FPS improvement over PyTorch baseline	> 15 FPS sustained	15–34 FPS, average ~19 FPS	yes
SC-2: Autostart models on boot without manual intervention	Pipeline active within 3 minutes of power-on	Active within approximately 1 minute and 40 seconds	yes
SC-3: Single-edge fallback produces directionally correct score	Score reflects correct deviation when one edge occluded	Confirmed correct direction via drift direction memory and frame position heuristic	yes
SC-4: Telemetry schema compatibility with Android app	100% field compliance	All records written in correct JSON schema; verified via standalone telemetry test	yes
SC-5: Real-time telemetry output	Continuous WebSocket updates	20 Hz telemetry stream achieved	Yes

SC-6: Static-obstacle (parked car) detection via sensor fallback	Parked car classified as static_obstacle within range	Matched expected; confirmed via console and logs	Yes
SC-7: Drowsiness model accuracy	At least 90%	Approximately 92%	Yes
SC-8: Emotion model accuracy	At least 80%	Approximately 85%	Yes
SC-9: Face detection as first pipeline gate	Detect valid driver face before running dependent modules	Face detection successfully provided the face region for emotion, drowsiness, and head-pose modules	Yes
SC-10: Emotion output simplification	Convert eight-class emotion output into three operational categories	Sadness mapped to sad, anger mapped to angry, and all other classes mapped to neutral	Yes
SC-11: Blink false-positive reduction	Normal blinking should not trigger drowsiness	Consecutive-frame logic prevented short blinks from triggering drowsiness	Yes
SC-12: Privacy-aware AI output	Do not transfer raw video as the main AI output	AI subsystem produced processed driver-state indicators instead of raw camera frames	Yes
SC-13: Head-pose functional output	Estimate yaw, pitch, and roll from facial landmarks	Head-pose module produced orientation values used for attention monitoring	Yes
SC-14: Jetson integration readiness	Prepare AI outputs for Jetson-hosted API/WebSocket layer	AI outputs were structured for use by the software team's Jetson-hosted communication system	Yes
SC-15: Local scoring pipeline	Telemetry can be processed into a score result by domain logic.	Scoring engine implemented and ScoringEngineTest passed.	Yes
SC-16: Android local test status	Core local software tests pass.	76 local tests completed successfully.	Yes
SC-17: Android integration readiness	Client boundaries exist for telemetry, Jetson control, and backend sync.	Boundaries compile, but real Jetson/backend behavior was not fully verified from repository evidence.	Partial
SC-18: Secure administrator access	Admin pages and protected APIs require authenticated admin access.	Session-based administrator login and role-protected admin routes were implemented.	Yes
SC-19: Authenticated completed-session upload	Upload endpoint accepts only authenticated API-client requests.	API-client authentication and /api/upload/sessions ingestion were implemented.	Yes
SC-20: Persistent historical records	Backend stores sessions and child safety artifacts.	Session summaries, events, escalations, score-history points, drivers, vehicles, and ingestion issues were persisted.	Yes
SC-21: Administrative dashboard and analytics	Admins can inspect fleet and driver safety history.	Dashboard, fleet analytics, driver analytics, session review, driver pages, and vehicle pages were implemented.	Yes

SC-22: Vehicle-local communication design	The phone can use a vehicle-local Jetson access-point/API/WebSocket architecture for session communication.	Jetson-control API boundaries, WebSocket telemetry components, reservation/claim/release contracts, and heartbeat/lease behavior were implemented in the software architecture.	Yes
SC-23: Public backend/admin deployment	The Phase 2 backend/admin website is reachable through the configured public server address.	The backend/admin website was deployed using the public IP address and website link.	Yes
SC-24: Monitoring dashboard	Backend/server health can be observed through a monitoring dashboard.	Prometheus/Grafana monitoring was configured and a Grafana dashboard was available for operational visibility.	Yes
SC-25 — Transmission reliability (ESP32↔Jetson UART)	< 1% packet loss	No packet loss observed in bench conditions	Partial (bench-verified; field validation pending)
SC-26 — Power stability under peak motor load	< 0.5 V fluctuation during peak acceleration	Not yet measured; dedicated voltage regulator in procurement	Pending
SC-27 — Actuation response (servo/DC commands)	< 100 ms from microcontroller signal	Not yet measured; pending physical assembly	Pending
SC-28 — Sensor synchronization (IMU, Hall, telemetry)	Timestamp drift < 10 ms	SD-card Black Box logs timestamped entries correctly on bench; cross-stream drift not yet measured	Partial (logging verified; drift pending)
SC-29 — Vibration stability (sensor mounts)	Limit sensor displacement to prevent rolling-shutter distortion	Vibration-dampening features designed and confirmed structurally viable at CAD review; physical characterization pending	Partial (design-confirmed; physical pending)

Table 2: Performance Evaluation

5.4 User Feedback (if applicable)

[If user testing was conducted, present the feedback received and any changes made based on this feedback.]

6. Results and Discussion

6.1 Final Results

The final result of the in-cabin AI subsystem is a functioning perception pipeline that detects and interprets driver-state indicators from a driver-facing camera. The subsystem includes four main modules: face detection, drowsiness detection, emotion recognition, and head-pose estimation. These modules operate together as a single perception pipeline and produce processed outputs that support the Driver Reliability Score.

The face detection module successfully acts as the first stage of the pipeline. It detects the driver's face from camera frames and provides the face region needed by the emotion recognition module and the landmark-based modules. This result is important because drowsiness detection, emotion recognition, and head-pose estimation all depend on the correct localization of the driver's face.

The drowsiness module achieved approximately 92% accuracy. This result shows that the system can detect fatigue-related behavior with strong reliability at the prototype level. The module combines Eye Aspect Ratio, Mouth Aspect Ratio, CNN-based eye-state classification, and temporal persistence. The use of consecutive-frame counting improved the stability of the output because normal blinking was not immediately classified as drowsiness.

The emotion recognition module achieved approximately 85% accuracy. The original model supports eight emotion classes, but the final system uses only three operational categories: sad, angry, and neutral. Sadness and anger are kept as distress-related indicators, while the remaining emotion classes are mapped to neutral. This simplified output is more suitable for the Driver Reliability Score because it avoids overinterpreting facial expressions that are not directly needed by the prototype.

The head-pose module produced yaw, pitch, and roll values from facial landmarks. These outputs are used to estimate whether the driver's head orientation suggests reduced attention or distraction. Instead of reacting to a single frame, the module uses thresholds and consecutive-frame logic to reduce false distraction events caused by natural short head movements.

The Road Monitoring subsystem developed a real-time lane deviation system running on the NVIDIA Jetson Orin Nano. The system captures live frames from an IMX219 CSI camera, runs YOLOPv2 using a TensorRT FP16 engine, computes a calibrated lane deviation score using an HSV-based estimator, and outputs the result to a shared file every frame.

The main performance improvement came from TensorRT optimisation. The original PyTorch model ran at 8–12 FPS, while the TensorRT version achieved 15–25 FPS with an average of about 19 FPS, making the system suitable for real-time use. The output interface worked reliably, with the deviation file updated every frame and correctly read by the bridge module. The system also started automatically on boot without manual intervention. The figures below show how the lane deviation logic was behaving against the fake road built by the AI department.

Figure 4 detecting drowsiness and head position

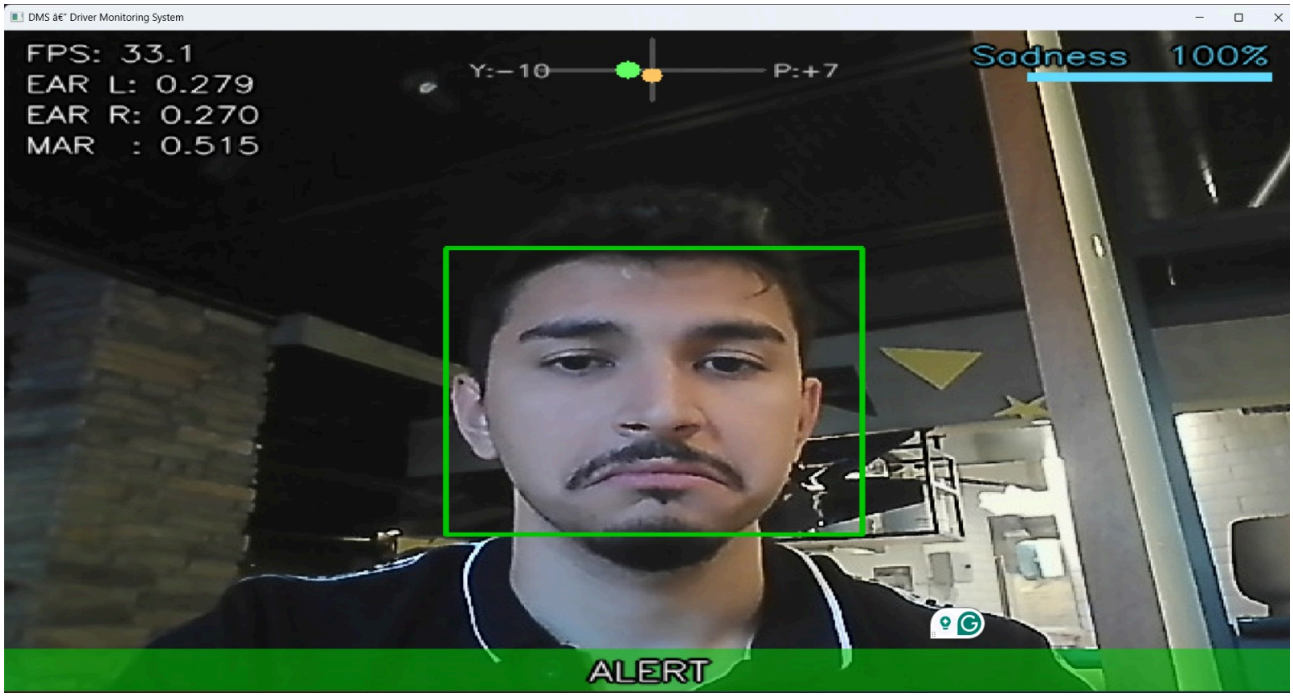

Figure 5 detecting sadness emotions

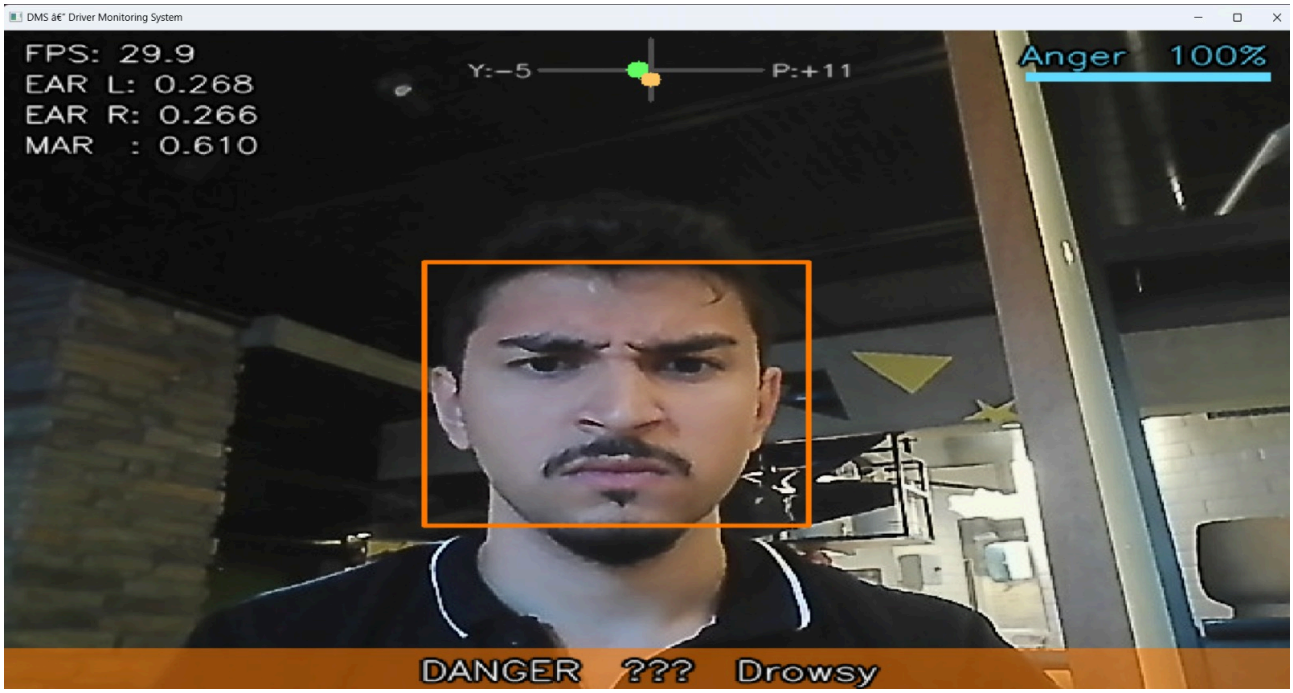

Figure 6 detecting angriness emotion

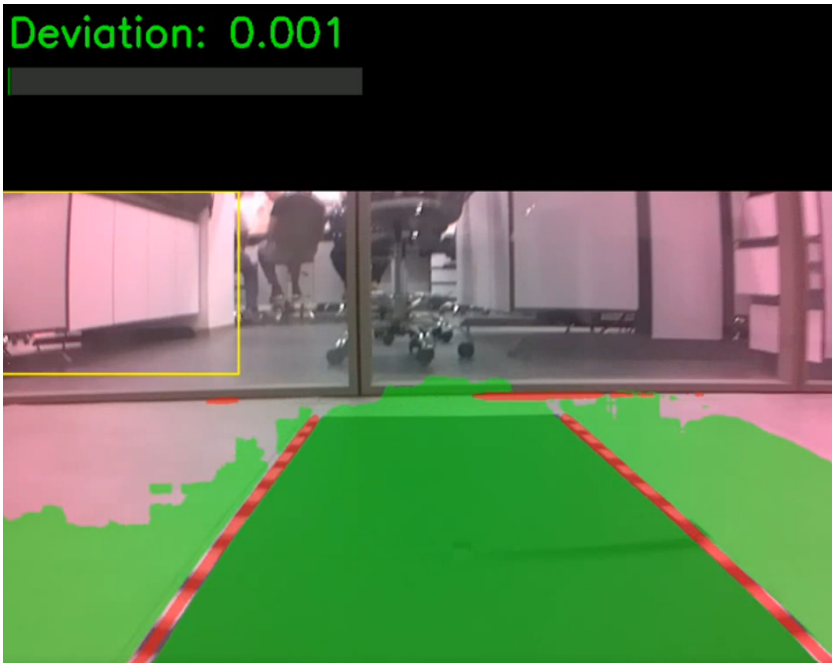

Figure 7: YOLOPv2 on Fake Road

As seen in **Figure 6.1** the lane deviation score is 0 when the car is moving at the center of the road.

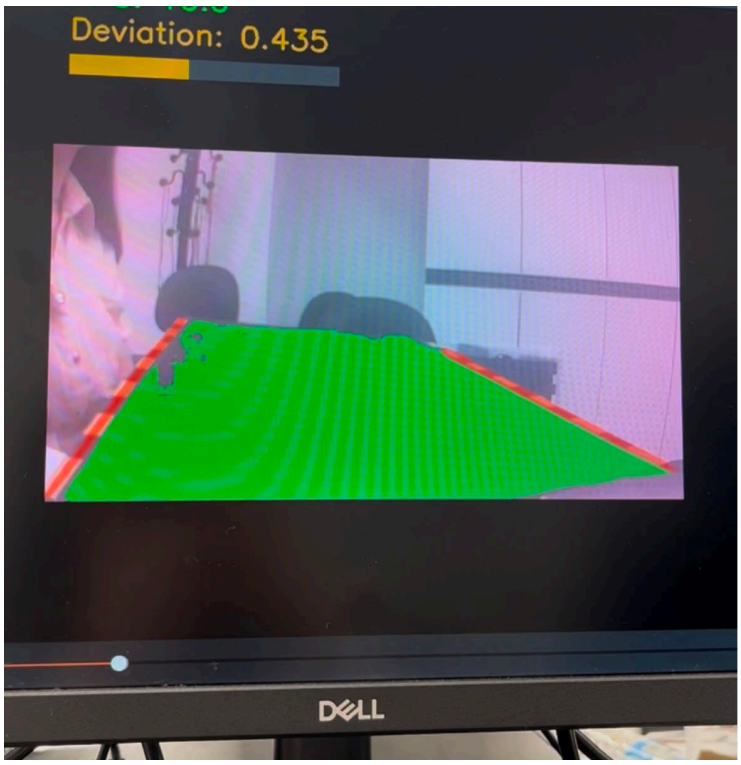

Figure 8: Lane Deviation Warning Score

Figure 6.2 shows a slightly deviated car lane highlighted in the colour yellow to show its severity.

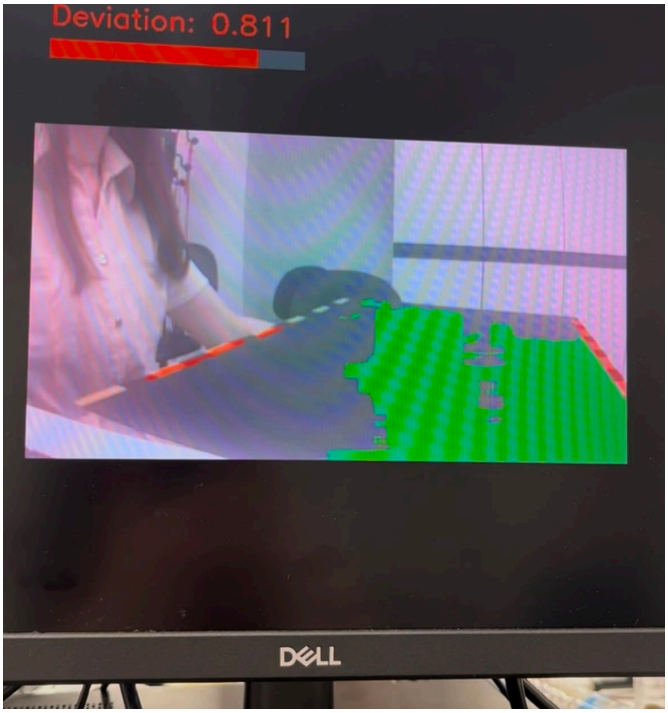

Figure 9: Severe Lane Deviation Score

Figure 6.3 shows a severely deviated car lane highlighted in the colour red to show its severity.

The completed CARLA module reliably produces real-time lane-deviation, speed, acceleration, and obstacle telemetry while driving the Dodge Charger in Town10HD_Opt. Evidence collected includes JSONL telemetry logs (one record per simulation tick), debug camera frames and lane-mask images saved at intervals, and screen recordings of the CARLA window with live console output for clear,

rainy/wet, and night scenarios, both with and without spawned traffic and pedestrians. A couple of examples can be observed below in figures …

Figure 10: YOLOPv2 on CARLA

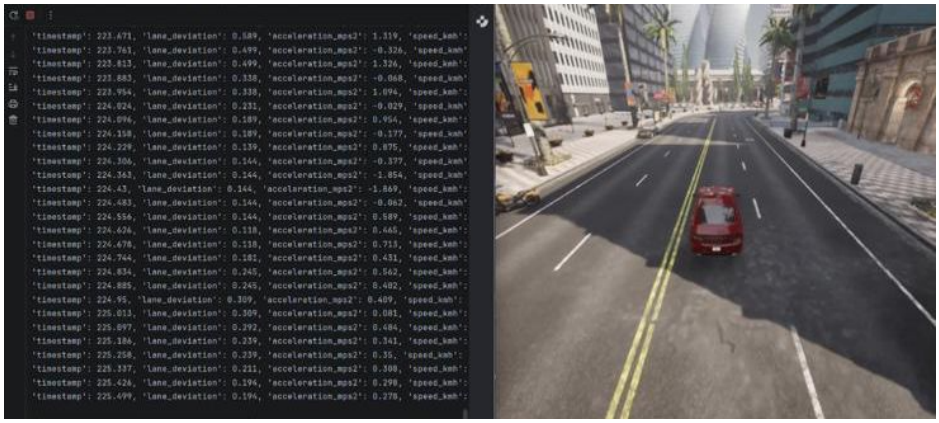

Figure 11: Lane Deviation CARLA Daytime

Figure 6.1.5

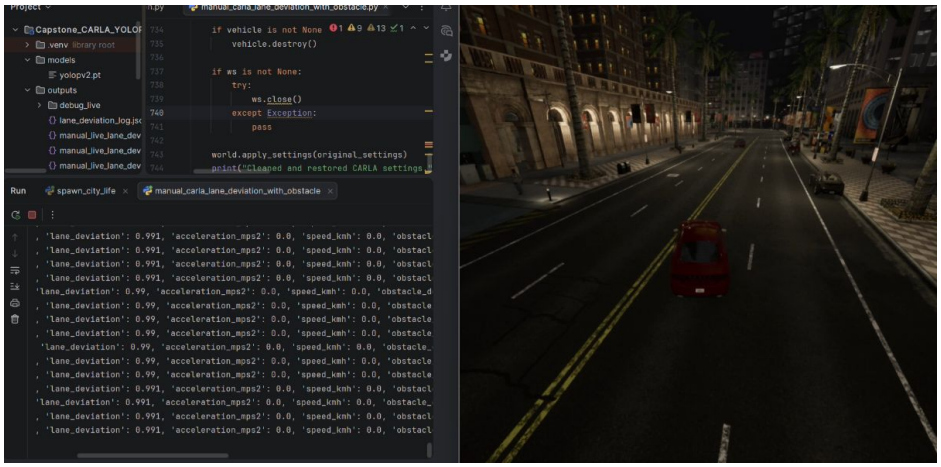

Figure 12: Lane Deviation CARLA Night-time

Figure 6.1.6

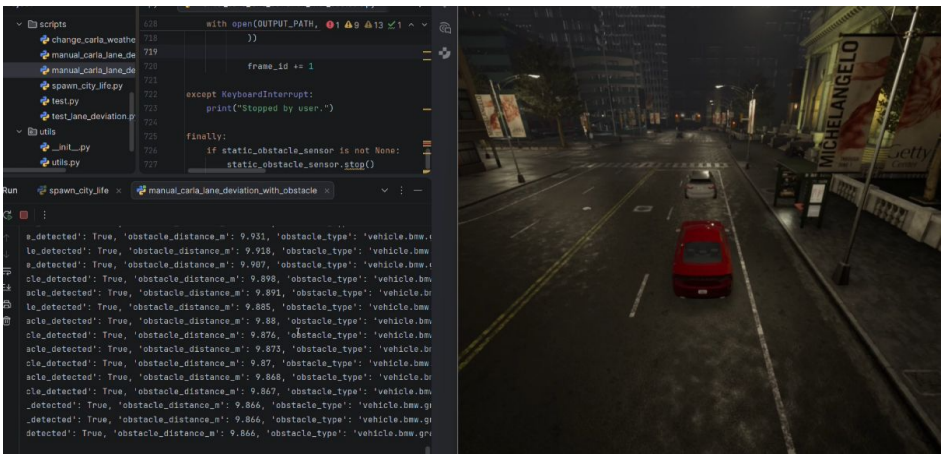

Figure 13: Obstacle Detection in CARLA

The final AI output is represented as processed driver-state indicators, including face presence, drowsiness level, emotion label, emotion confidence, distress event, head-pose status, FPS, and timestamp. There are also the final scores related to the road monitoring subsystem which include the lane deviation score along with the timestamps and fps. These outputs are transferred to the Jetson-hosted API and WebSocket system implemented by the software team.

The final result of the software subsystem is a working end-to-end driver monitoring software pipeline that consumes these AI outputs and converts them into scored, persistent, and administratively reviewable driver-reliability records. The completed software connects the Android thick-client application, Jetson-side API and WebSocket telemetry bridge, Spring Boot backend server, MySQL persistence layer, administrator web interface, and Grafana monitoring environment.

The Android client was completed as the live-session and local-scoring component of the system. It supports driver-side workflows, telemetry mapping, local scoring, local persistence, session history, backend upload preparation, and integration boundaries for Jetson and backend communication. The Driver Reliability Score is calculated locally using continuous penalties, event penalties, and escalation penalties. This allows the Android client to continue processing a session even when backend connectivity is unavailable.

Repository-supported Android validation showed that the Android client was buildable. The debug APK build completed successfully using the Gradle assembleDebug task in approximately 18 seconds in the inspected test run. This confirms that the Android project structure, Kotlin source files, dependencies, generated code, and build configuration were consistent enough to produce a working debug build.

The Android local test suite also completed successfully. A total of 76 local tests passed during the inspected repository test run. These tests covered important software behaviours such as scoring logic, telemetry processing, synchronization-related logic, credential and session handling, and related domain behaviour. This result supports the claim that the core Android software logic was not only implemented but also validated through automated local tests.

The scoring pipeline was one of the main final outcomes of the Android implementation. The scoring engine processes telemetry into a bounded score between 0 and 100. It applies smoothing to reduce noise, uses bounded time steps to avoid unfair penalties from abnormal timestamp gaps, detects high-fatigue, lane-departure, harsh-braking, and emotional-distress events, and records escalation patterns such as repeated harsh braking or combined fatigue and lane deviation. The scoring result is stored with supporting artifacts rather than only as a final number.

The local persistence result was also completed on the Android side. Room and SQLite are used to preserve drivers, vehicles, session summaries, events, escalations, score-history points, and synchronization metadata. This means that completed driving sessions can remain on the Android device if backend upload is not immediately possible. The session can later be uploaded when backend connectivity is available. This result satisfies the offline-first requirement of the software subsystem.

The Jetson-side software result is an API and WebSocket bridge for integration with the Android client. The Jetson service exposes health and vehicle-state endpoints, authority-management endpoints, and a telemetry WebSocket endpoint. This allows the Android client to check whether the vehicle is available, claim usage authority, receive live telemetry, maintain heartbeat and state awareness, and release authority when the session ends. The Jetson-side software therefore acts as a controlled telemetry and vehicle-state bridge rather than as an unmanaged script.

The Jetson integration result showed that manual configuration of the local Jetson base URL allowed the Android client to communicate with the Jetson service. Through this path, the software system could access the Jetson-side API and telemetry WebSocket stream. Automatic discovery remained a refinement area, but the direct API and WebSocket integration path demonstrated the required software communication boundary for the prototype.

The backend server was completed as the historical administration and upload-ingestion component. The backend was implemented using Java, Spring Boot, Spring Security, Spring MVC, Thymeleaf, Spring Data JPA, Hibernate, and MySQL. It provides authenticated administrator access, protected web pages, authenticated API-client upload, session persistence, driver management, vehicle management, session review, dashboard metrics, driver analytics, and fleet analytics.

The completed-session upload endpoint was implemented and validated as a protected machine-to-machine API. The endpoint accepts completed-session uploads only from authenticated API clients. Uploaded sessions are validated, mapped, and persisted into backend records. The backend stores the session summary and child artifacts, including safety events, escalation records, score-history points, and ingestion issue records where applicable.

The backend preserves the uploaded session result instead of silently recalculating it. This is a significant final result because it preserves the responsibility boundary between Android and backend. The Android client produces the completed scored session using the live telemetry sequence. The backend stores the completed session as historical truth and uses it for review and analytics. This prevents score differences from appearing after upload due to backend-side recomputation.

The administrator web interface was completed as the human review layer of the software system. The admin web pages support login, dashboard review, driver management, vehicle management, session lists, session details, driver analytics, fleet analytics, settings, and profile workflows. These pages allow administrators to inspect completed sessions, final scores, penalty totals, event counts, escalation counts, score-history information, validity status, and upload-processing status.

The backend security result includes protected administrator routes and separated API-client upload authentication. Administrator pages require authenticated administrator access, while completed-session upload requires authenticated API-client access. This separation prevents upload permission from being treated as full administrator permission. Passwords are stored using hashing rather than plain text, which improves the security baseline of the backend system.

The backend analytics result includes average final score, total fleet distance, daily score trends, event-type counts, escalation-type counts, driver status counts, vehicle status counts, session status counts, invalid-session rate, warning-session rate, completed-session rate, active-driver share, and busy-vehicle share. These analytics allow the backend to support fleet-level and driver-level review rather than only storing raw session records.

The deployed backend result showed that the Spring Boot server could run as a public backend and admin website. Administrator login and driver-related backend access were verified during deployment checks. The server-side implementation therefore moved beyond local code execution and was deployed as a running service with database-backed functionality.

Grafana and monitoring support were also completed as part of the software result. The backend and server environment was connected with monitoring tools including Spring Boot Actuator, Micrometer and Prometheus support, Prometheus, node exporter, and Grafana dashboards. This provides runtime visibility into server behaviour and supports operational troubleshooting. The monitoring layer does not replace functional testing, but it improves the maintainability of the deployed backend environment.

The final integrated workflow can be summarized as follows. The AI pipeline on the Jetson Orin Nano produces structured driver-state indicators from the driver-facing camera. The Jetson-side service exposes these through an API and WebSocket telemetry stream. The Android client connects to the telemetry source, processes live packets, updates the Driver Reliability Score, records events and escalations, stores the completed session locally, and prepares an upload payload. The backend authenticates the upload, validates and stores the session in MySQL, and exposes the result through protected administrator pages and analytics dashboards. Grafana provides operational visibility for the deployed backend environment.

Overall, the final results demonstrate that both subsystems achieved their main objectives. The AI pipeline delivers reliable driver-state indicators from edge hardware. The software pipeline converts those indicators into explainable, persistent, and administratively reviewable driver-reliability records. The remaining limitations across both subsystems are mainly related to extended real-world testing, automatic Jetson discovery refinement, production hardening, and longer-duration field validation with the full physical prototype.

The Mechatronics subsystem delivered a complete, bench-validated vehicle platform consisting of real-time ESP32 firmware, a sensor suite, and a structured telemetry interface to the Jetson Orin Nano. The firmware controls a 60 A brushless ESC and steering servo from a Bluetooth Xbox Series X controller, with all actuation logic running non-blocking so that controller latency and steering response are never degraded by sensor reads or telemetry transmission.

A safe-start gate prevents the motor from activating until both control triggers are confirmed at neutral after the controller connects, and a controller-disconnect fail-safe returns the vehicle to neutral if the Bluetooth link is lost beyond a defined timeout. Because the ESC fitted to the platform operates in single-click forward/reverse mode rather than forward/brake mode, braking was implemented by commanding reverse against forward motion, with brake depth made proportional to trigger pressure and applied instantly rather than through the normal acceleration ramp. This was validated on the bench by confirming the throttle reading snapped directly to the commanded value on the same control loop as the trigger press, independent of the ramp state.

The MPU6050 IMU was integrated over I2C with gyroscope offset calibration performed at startup and accelerometer resting-offset correction applied in firmware, producing a zero-biased, correctly signed fore/aft acceleration signal. Axis mapping was confirmed empirically with a dedicated standalone test that printed plain-language motion labels against raw sensor values, rather than assumed from the sensor's physical mounting orientation. Pitch and roll outputs proved reliable and drift-free, as expected from gravity-referenced accelerometer data; yaw, derived solely from gyroscope integration in the absence of a magnetometer, was confirmed to drift at roughly 2–5 degrees per minute uncalibrated and below 0.5 degrees per minute after calibration, consistent with the sensor's known limitations and acceptable for the platform's short-duration runs.

Wheel speed is measured using a US1881 Hall-effect sensor with interrupt-driven pulse counting, converting pulse counts to a linear speed estimate in km/h. An external pull-up resistor was added

after bench testing revealed that the ESP32's internal pull-up alone was insufficient against switching noise from the nearby motor phase wires, which had been producing spurious interrupts and grossly inflated speed readings. After the fix, pulse counts were confirmed accurate against manual wheel-revolution counts, and a timed hand-spin test confirmed the resulting speed output matched a hand-calculated reference value.

All of this is packaged into a JSON telemetry schema — speed, steering angle, acceleration, obstacle distance, turn-signal state, ignition state, and stationary state — transmitted at 20 Hz over USB serial to the Jetson, matching the contract shared with the AI and Software departments. This schema was agreed early as a fixed integration boundary, allowing the Software team to develop its telemetry mapper and scoring logic independently of the firmware implementation. A sustained bench run produced roughly 12,000 consecutive packets at 20 Hz with no parse errors on the Jetson side, and obstacle distance is currently transmitted as a documented placeholder constant pending a ranging sensor in a future iteration.

Beyond the vehicle electronics, the Mechatronics subsystem also carried out the full bring-up of the shared Jetson Orin Nano platform — flashing JetPack, configuring the IMX219 camera pipeline, handing off a ready-to-develop platform to the AI and Software teams ahead of their own implementation work.

A custom chassis was designed in SolidWorks and fabricated via FDM printing in PLA+ for structural components and TPU for vibration-dampened sensor mounts, housing the Jetson, ESP32, IMU, Hall sensor, motor/ESC, and battery. Because chassis printing sat on the critical path, electronics and firmware development were deliberately decoupled from fabrication and fully bench-validated in isolation, so final assembly proceeds against components with an already-proven baseline. The main outstanding hardware item is a ground-reference noise coupling between the steering servo and the ESC signal line, characterized during bench testing as a brief reverse twitch under hard steering input; the root cause was diagnosed and a hardware fix (relocating the servo's ground return to the ESC's BEC ground) was identified but not yet implemented at time of writing.

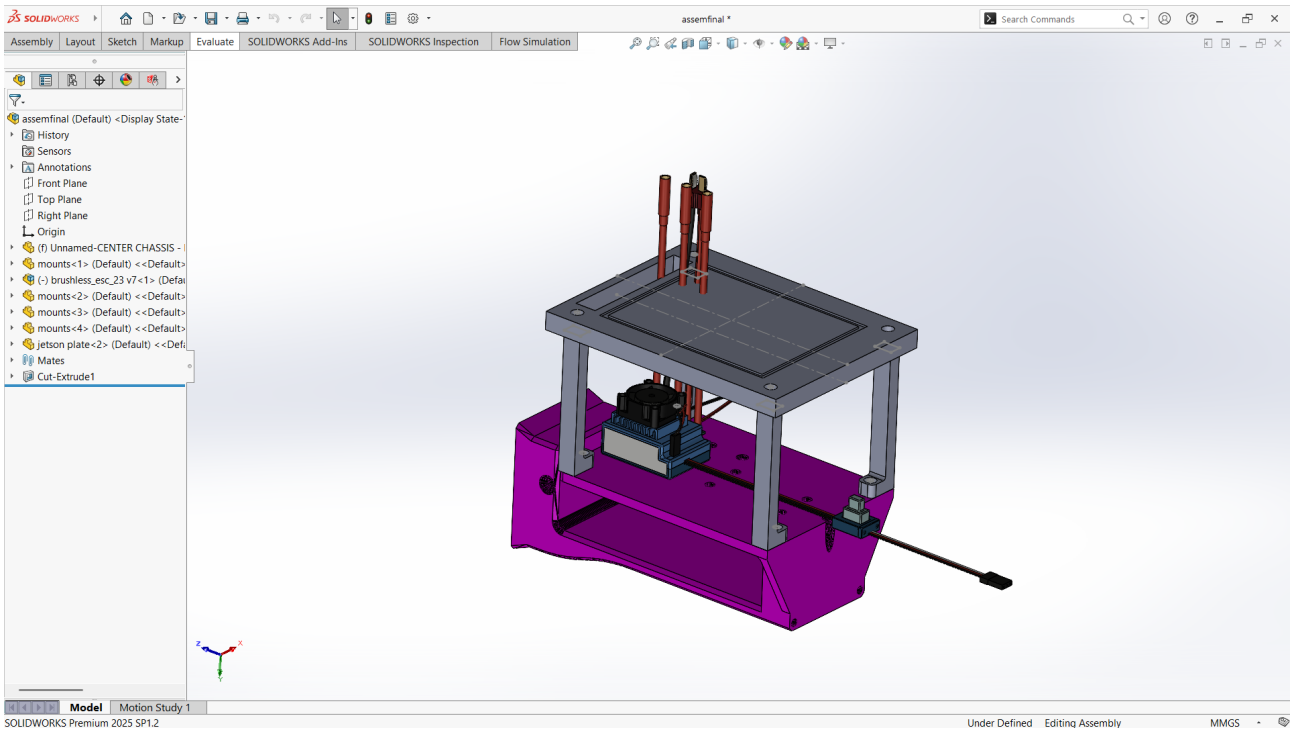

Figure 14 center chassis and jetson mount design assembly

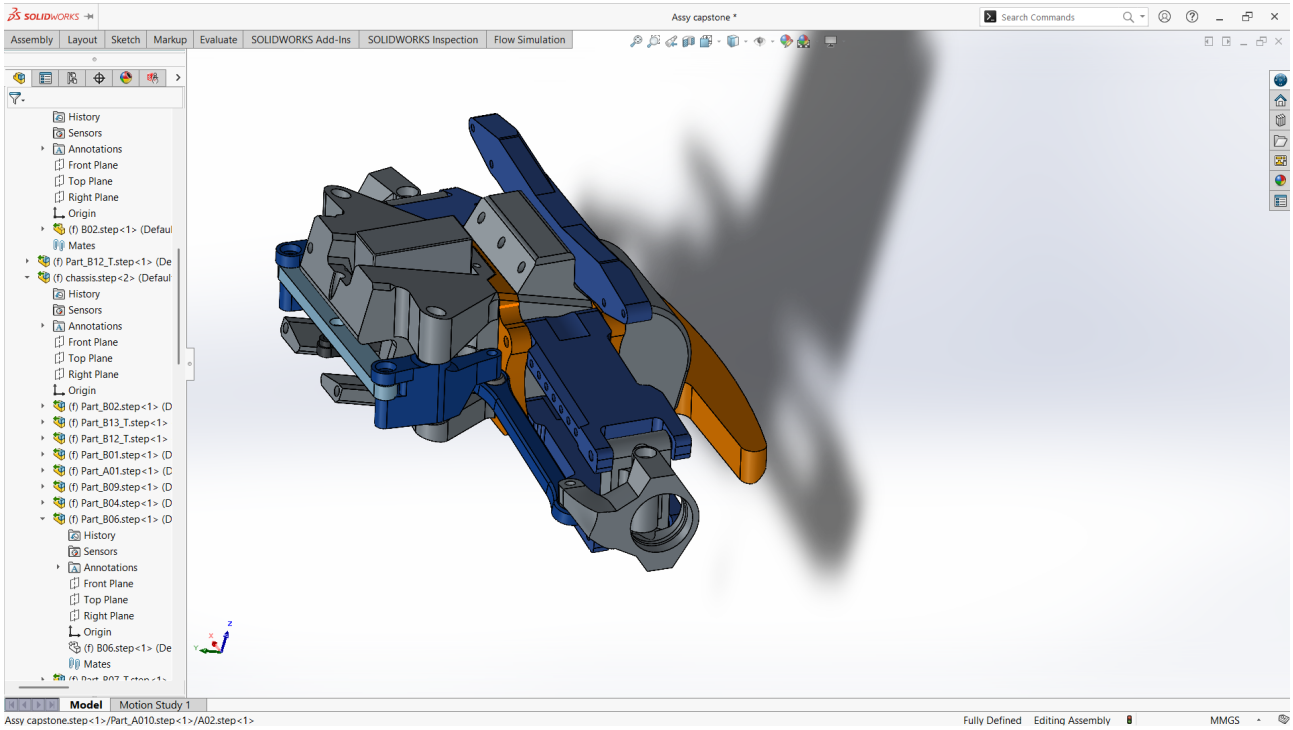

Figure 15 Front end steering system hyprid Assem

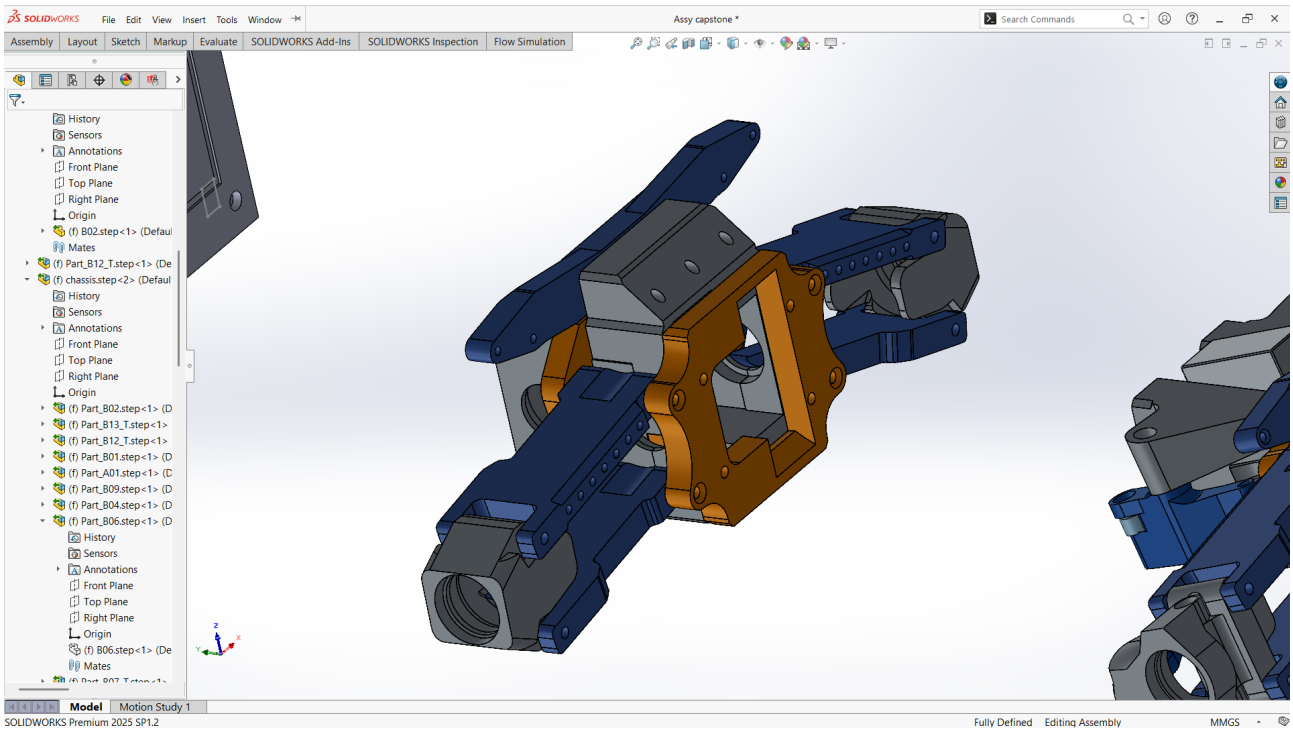

Figure 16 Rear End Assembly modified

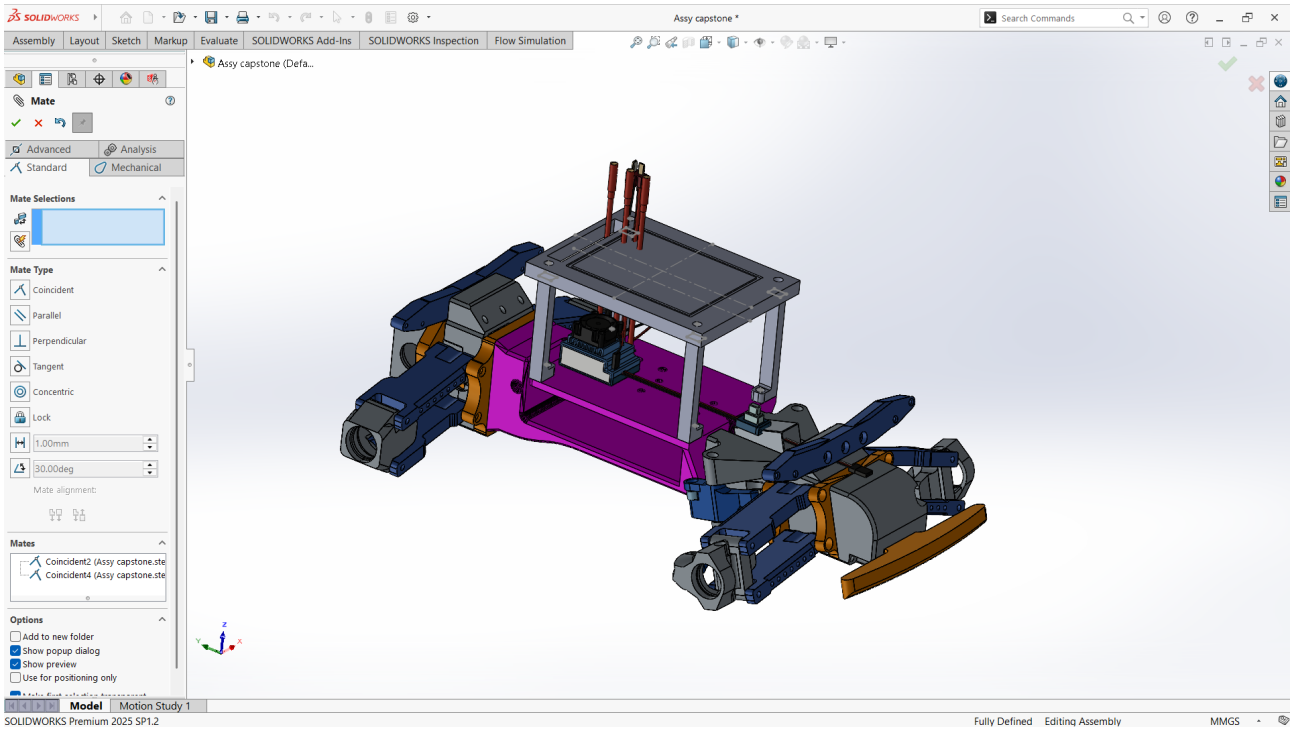

Figure 17 full assembly

6.2 Analysis and Discussion

The results show that the in-cabin AI subsystem achieved its main technical purpose: converting camera frames into meaningful driver-state information. The system does not simply run one model. Instead, it combines multiple complementary modules to make the driver-monitoring output more stable and useful for scoring.

The face detector is important because it works as a reliability gate. If the face is not detected, the system avoids forcing predictions from the other modules. This improves reliability because emotion recognition, drowsiness detection, and head-pose estimation should not run on invalid or missing face regions. This design choice reduces the chance of producing misleading outputs.

The drowsiness module produced the strongest quantitative result, with an approximate accuracy of 92%. This is significant because drowsiness is one of the most safety-critical indicators in the project. The combination of EAR, MAR, CNN-based eye-state classification, and temporal persistence improved the module compared with a simple frame-by-frame classifier. EAR provides an interpretable geometric feature, the CNN model provides additional visual classification, and temporal persistence reduces false positives from normal blinking.

The emotion module achieved approximately 85% accuracy, which is acceptable for a supporting driver-state indicator. However, emotion recognition is more sensitive than drowsiness detection because facial expressions can be subtle, lighting-dependent, and culturally variable. For this reason, the system does not use the full eight-class emotion output directly. Instead, the output is mapped into sad, angry, and neutral. This is a reasonable design decision because the prototype does not require detailed psychological interpretation. The module only needs to detect distress-related emotional states that may contribute to the reliability score.

The head-pose module adds another layer of interpretation by estimating whether the driver's head is oriented toward or away from the expected driving direction. This is useful because drowsiness and emotion alone do not fully describe attention. A driver may not be drowsy but may still be distracted if

the head remains turned away for a sustained period. The use of yaw, pitch, roll, and consecutive-frame counting helps the system distinguish between normal head movement and sustained distraction.

A key strength of the AI subsystem is the use of processed outputs instead of raw video transfer. This improves privacy and makes software integration easier. The AI subsystem provides simple values and event flags, while the software subsystem handles communication, database logic, scoring, and application display. This separation makes the full system more modular and easier to maintain.

The results also show that the Road Monitoring pipeline successfully achieves its main goal of producing a stable, real-time lane deviation score on a Jetson-based system. Several design choices were important in reaching this outcome.

A key decision was separating the lane deviation estimator from YOLOPv2's segmentation output. While YOLOPv2 works and produces visible lane overlays on the test track, its segmentation is not precise enough for accurate pixel-level deviation measurement. The custom HSV-based estimator solves this by working directly on the track's color information, making the system more stable, deterministic, and easy to tune for the specific environment.

TensorRT optimisation significantly improved performance, but the conversion process was more difficult than expected due to an unsupported ONNX operation (SequenceConstruct). This was resolved by modifying the ONNX graph to remove the unsupported node and connect the outputs directly, allowing the model to run correctly in TensorRT without losing functionality.

Finally, the attempt to fine-tune YOLOPv2 was limited by the use of a TorchScript model, which is meant for deployment and does not support training. If the original training code had been available, fine-tuning on the custom dataset could have improved accuracy on the yellow tape track. This is noted as a limitation and a direction for future work.

As for CARLA, Lane-deviation estimation works well because the histogram-based peak search and temporal smoothing make it tolerant of noisy single-frame segmentation, and the keyboard-start flag avoids the unrealistic non-zero deviation that occurred at spawn. The hybrid obstacle approach successfully separates moving traffic/pedestrians from parked cars while suppressing false positives from street furniture, which a single obstacle sensor could not do. The main limitation is that running YOLOPv2 every frame was too slow for smooth simulation, so inference is throttled to every 8 frames; this is an acceptable trade-off for testing purposes but would need to be revisited for a real-time deployed system. An earlier risk-scoring feature was removed because it added complexity without improving the reliability of the core telemetry.

Some AI limitations remain. The drowsiness and emotion accuracies were reported, but separate quantitative accuracy values were not reported for face detection and head-pose estimation. These modules were validated functionally, but future testing should measure them under different lighting conditions, camera positions, face angles, and driver appearances. The system may also be affected by glasses, shadows, low camera quality, or partial occlusion of the face.

The final software results similarly show that the project achieved its main objective of building an end-to-end software foundation for fleet driver monitoring. The completed implementation connects Android local scoring, Jetson API and WebSocket telemetry access, backend upload ingestion, MySQL persistence, administrator web review, analytics, and Grafana monitoring. This means that the project moved beyond a single prototype screen and produced a complete software workflow from telemetry intake to administrator-facing historical review.

The Android client achieved its main objective as the live-session component. The application can be built successfully, and the local test results support the correctness of important client-side logic. The Android side was able to implement the Driver Reliability Score as local deterministic domain logic,

which was an important design goal. This worked well because live scoring should remain close to the telemetry stream and should not depend on the backend being reachable during the active driving session. The offline-first approach also worked well because completed sessions can be preserved locally before backend synchronization.

The Driver Reliability Score design worked well from an explainability perspective. Instead of producing only a final number, the scoring pipeline separates continuous penalties, event penalties, and escalation penalties. This makes the result easier to interpret because the final score can be connected to fatigue risk, lane deviation, harsh braking, emotional-risk events, repeated unsafe behaviour, and combined risk patterns. For an administrative fleet system, this is more useful than a score that cannot be explained.

The backend achieved its main objective as the historical administration component. It provides authenticated administrator access, protected routes, driver and vehicle management, completed-session upload ingestion, session review, driver analytics, fleet analytics, and dashboard metrics. This worked well because the backend was designed as a layered Spring Boot system with separate controllers, services, repositories, mappers, validation logic, query services, and view models. The result is easier to maintain than a backend where security, persistence, validation, and page rendering are mixed together.

The completed-session upload design also worked well. The backend receives completed sessions from an authenticated API client, validates the uploaded structure, stores the session summary, stores child records, records ingestion issues where applicable, and updates aggregate information. This supports the project objective of creating a reliable connection between Android session production and backend historical storage. The backend also preserves the uploaded Android score instead of recomputing it, which protects the historical meaning of each completed session.

The Jetson-side API and WebSocket bridge achieved the basic integration objective. The Jetson software provides health, vehicle-state, authority-control, and telemetry-streaming endpoints. This allows the Android client to treat the Jetson as a structured software service rather than as an uncontrolled data source. The WebSocket approach is suitable for live telemetry because it supports continuous updates without requiring repeated manual HTTP polling.

The admin web interface worked well as the review layer of the system. The administrator can access protected pages for dashboard review, driver management, vehicle management, session lists, session details, driver analytics, and fleet analytics. This is important because the software project is not only about generating a score but also about making the score reviewable. The backend pages allow uploaded sessions to become understandable records for administrators.

Grafana and monitoring support worked well as an operational visibility layer. Monitoring does not prove business correctness by itself, but it improves the maintainability of the deployed backend. The inclusion of Actuator, Prometheus-related support, node exporter, and Grafana gives the server environment a way to expose runtime behaviour and support troubleshooting. This is especially important because the backend was deployed as a running service rather than only remaining as local development code.

The main result that did not fully work as expected was automatic Jetson discovery. Manual configuration of the Jetson base URL allowed the Android client to communicate with the Jetson API and WebSocket service, but automatic discovery still requires refinement. The likely reason is that discovering the local wireless network is not enough; the selected vehicle candidate must also correctly propagate the Jetson host address, HTTP base URL, and WebSocket URL into the Android client. This does not invalidate the integration architecture, but it identifies a remaining usability and automation gap.

Another limitation is that repository-supported validation does not fully replace long-duration real-world system testing. Android builds and local tests provide strong evidence for client-side software correctness, and backend workflow checks provide evidence for server-side correctness. However, extended operation with the Android device, Jetson service, deployed backend, and physical prototype running together under realistic conditions remains necessary. This is a normal limitation for a capstone prototype because live telemetry, wireless network behaviour, timing irregularities, and deployment recovery can expose issues that unit tests cannot fully simulate.

Some backend production-readiness items also remain as future work. The implemented backend supports authentication, upload ingestion, persistence, administration, analytics, deployment, and monitoring, but a production system would require stronger operational hardening. This includes enforcing HTTPS, moving secrets out of source code, managing database migrations through migration tooling, defining backup and recovery procedures, improving service restart policies, and performing longer security validation. These limitations are related to deployment maturity rather than to the core software architecture.

Compared with typical fleet telematics systems, this project follows a similar general pattern by collecting driving-related data and presenting it through an administrative review layer. However, the project differs by emphasizing an Android thick-client approach for live scoring, keeping the scoring local and sending only completed sessions to the backend afterward. This makes the system more resilient during connectivity interruptions and better aligned with offline-first mobile operation. It also differs from typical driver-monitoring approaches that focus mainly on perception outputs, because this implementation does not stop at detecting drowsiness, lane deviation, or emotion-related indicators. Instead, it converts those indicators into a structured reliability score, supporting event records, escalation records, score-history points, and backend analytics, covering the full path from telemetry interpretation to administrative decision support.

Compared with conventional backend dashboard systems, this project includes a clearer separation between score production and score review. The backend is not treated as the live scoring engine. Instead, it stores completed session results produced by the Android client and then provides analytics over those stored records. This avoids silent changes to historical session outcomes and keeps the source of the score traceable.

The main strength of the overall system is the completeness of the combined pipeline. The AI subsystem delivers reliable driver-state indicators from edge hardware. The software pipeline converts those indicators into explainable, persistent, and administratively reviewable driver-reliability records, covering Android scoring, local persistence, Jetson telemetry integration, backend upload ingestion, admin web pages, analytics, deployment, and monitoring. The main weaknesses are that some AI modules still require broader quantitative validation, automatic Jetson discovery needs refinement, and long-duration live telemetry sessions and production deployment hardening remain as future work. Overall, the results show that the core architecture across both subsystems is successful and that the remaining work is mainly refinement, hardening, and extended real-world validation.

Evaluated against its stated objectives, the Mechatronics subsystem met its core functional goals — real-time actuation, sensor integration, telemetry packaging, and a working fail-safe architecture — while several secondary objectives remain at a "identified but not yet closed" state rather than fully resolved. This distinction matters for an honest assessment of the subsystem's maturity, and it is worth discussing why the gaps fell where they did rather than treating the result as uniformly complete.

The strongest result is the safety and control architecture itself. The safe-start gate, the proportional instant-brake logic, and the controller-disconnect fail-safe were not afterthoughts added once the car was driving — they were treated as preconditions in the bench-first methodology, and this shows in how the system behaves at its edges rather than just in normal operation. The decision to make

braking depth track trigger pressure directly, rather than depending on the ESC's commanded throttle history, is a good example of iterating past a plausible-but-wrong first implementation: an earlier version correctly handled the case where the driver releases the throttle and immediately brakes, but failed silently in the much more common case where the throttle is released and the brake is applied a moment later, because by then the ramp logic had already coasted the command back to neutral and the "still moving forward" condition no longer held. Catching this through direct bench observation, rather than assuming the first working version was correct, is the kind of result that is easy to undervalue in a report but materially affects whether the platform is safe to operate near people.

The most significant unresolved limitation is the absence of a true active brake. This is not a firmware shortcoming — it is a consequence of the ESC's single-click forward/reverse hardware mode, which provides no electrical braking circuit distinct from commanding reverse. The firmware response (proportional reverse-as-brake) is the correct engineering answer given that constraint, but it means the platform's stopping behavior is fundamentally bounded by the ESC's reverse current limit rather than a dedicated brake, and braking force cannot be made stronger than the motor's reverse authority allows without changing the ESC's configured mode. This is a hardware ceiling, not a tuning problem, and is reported as such rather than implied to be solvable in firmware.

The steering-induced ground noise issue is a useful case study in distinguishing symptom from cause. The fault presented identically to several different hypotheses in sequence — code logic, ESC programming, signal corruption — and each was tested and ruled out empirically before the actual cause (a shared ground return path between the servo and the ESP32, rather than the servo and the ESC) was isolated. The diagnostic value of that process exceeds its immediate cost: the firmware-level mitigation (holding idle at true neutral rather than one microsecond into the reverse band) reduced the frequency of the fault without eliminating its root cause, and the documented hardware fix is low-risk and well-specified for the next assembly pass. Reporting this as resolved would overstate the current state of the platform; reporting it as undiagnosed would understate the engineering work already done. "Characterized and mitigated, fix identified but not yet implemented" is the accurate middle ground, and it is presented that way throughout this report.

The IMU and Hall-effect sensing results illustrate a broader theme in the subsystem: where physics imposes a hard limit, the firmware works within it rather than against it, and where a result could plausibly be assumed, it was instead measured. Yaw drift on the MPU6050 is a known consequence of using gyroscope integration without a magnetometer, and no amount of clever filtering removes it — the calibration step reduces residual bias but does not produce a stable heading, and the report does not claim otherwise. Conversely, the IMU axis mapping was not assumed from the sensor's described mounting orientation; it was determined by physically moving the car and reading the sensor's actual response, which is the more defensible approach given that a verbal description of "rotated roughly -90 degrees" turned out to require an axis swap and a sign flip that would have been easy to get wrong by inspection alone. Similarly, the Hall-effect sensor's inability to distinguish forward from reverse motion is reported as a known telemetry limitation (reverse driving registers as positive speed) rather than worked around with a heuristic that could silently misclassify vehicle state.

The decision to treat the ESP32 as the sole real-time control authority, with the Jetson receiving telemetry only and never issuing actuation commands, was a deliberate architectural choice rather than a constraint discovered after the fact. This matters for the project's broader safety story: whatever latency or scheduling variability exists on the non-real-time Jetson host — running AI inference, the FastAPI server, and WebSocket relay concurrently — has no path back into vehicle control. The cost of this choice is that the platform cannot currently support AI-driven actuation (e.g., autonomous braking from a perception result), which was out of scope for this iteration but is a natural next step once the architecture's read-only telemetry boundary is deliberately reopened for control.

Several limitations are economic and logistical rather than technical, and are worth discussing separately because they shaped engineering decisions throughout. Component sourcing within Turkey at the required specification and budget was a recurring constraint, and is visible in two substitutions:

the TVS diodes specified for the power protection circuit were unavailable locally and were replaced with back-to-back Zener diode pairs, and the LTC3780 buck converter was replaced with the locally available XL4016. Both substitutions were validated on the bench before acceptance, so the substitutions did not compromise the bench-first validation discipline — but they are a reminder that the platform's design space was constrained by what could actually be procured in the project timeline, not only by what was technically optimal.

Taken together, the Mechatronics subsystem's results support its intended role as a physical testbed: a platform whose behavior is well-characterized, whose failure modes are documented rather than hidden, and whose telemetry can be trusted by the AI and Software departments because each field's derivation and limitations are explicitly known. The subsystem's remaining gaps — the active-brake ceiling, the servo ground fix, obstacle ranging, and yaw stability — are not surprises discovered late; they were identified during the same bench-validation process that produced the subsystem's working results, and are reported with the same level of specificity.

6.3 Comparison with Original Objectives

The Road Monitoring sub-team met most of its objectives. The IMX219 CSI camera was successfully integrated with the Jetson Orin Nano using a working GStreamer pipeline. YOLOPv2 was deployed successfully and produced segmentation overlays on live camera frames. The HSV lane deviation estimator was fully implemented, calibrated, and tested across left, centre, and right positions. It outputs a normalised deviation score between 0 and 1 with smoothing, a single-edge fallback, and a grace period for missing detections. The result is written to /tmp/lane_deviation.txt every frame and correctly read by the bridge module.

TensorRT optimisation achieved 15–25 FPS, exceeding the 15 FPS target. The conversion required resolving an ONNX compatibility issue by modifying the model graph, which was a key technical challenge. The physical test track was built and used successfully for calibration and real-world testing.

The only objective not fully achieved was YOLOPv2 fine-tuning. A 200-image dataset was prepared, but training was not possible due to the TorchScript format not supporting weight updates. This was partially addressed by the HSV-based estimator, which provides reliable deviation scoring without retraining.

The in-cabin AI subsystem met all of its main objectives. The face-detection module successfully located the driver's face and provided the face region required by all other modules. The drowsiness detection module was implemented using EAR, MAR, CNN eye-state classification, and temporal persistence, achieving approximately 92% accuracy. Consecutive-frame logic was used to prevent short blinks from immediately triggering drowsiness events, reducing false positives caused by normal blinking. The emotion recognition module was integrated and used to detect driver emotional state, with the eight-class model output successfully mapped into three operational categories: sad, angry, and neutral, achieving approximately 85% accuracy. The head-pose estimation module produced yaw, pitch, and roll values from facial landmarks and was fully integrated into the pipeline. The subsystem produced processed driver-state indicators suitable for the Jetson-hosted API and WebSocket layer, and the system uses processed values and event indicators instead of raw camera frames, satisfying the privacy-aware integration objective.

The strongest result is the drowsiness module, which achieved approximately 92% accuracy and provides an important safety-related signal. The emotion module achieved approximately 85% accuracy and was simplified into practical categories for driver-risk interpretation. Face detection and

head-pose estimation were successfully integrated as functional modules, although future work should include more detailed quantitative testing for these components.

The original software objective was to build a thick-client Android application capable of supporting driver monitoring, local reliability scoring, and session handling. During implementation, the objective expanded into a complete software pipeline that includes the Android client, Jetson-side API and WebSocket telemetry bridge, Spring Boot backend server, MySQL persistence, administrator web interface, upload ingestion, analytics, deployment support, and Grafana monitoring. This expansion did not replace the original objective; it extended it so that the system could support both live driver-side operation and backend administrator review.

The Android thick-client objective was fully met. The Android client was implemented using Kotlin, Jetpack Compose, ViewModels, repositories, Room persistence, telemetry mapping, scoring logic, backend DTOs, and integration boundaries. The application builds successfully, and repository-supported Android tests passed. This confirms that the Android project reached a functional software state rather than remaining only as a design proposal.

The local Driver Reliability Score objective was fully met. The scoring engine was implemented as deterministic domain logic and calculates a score bounded between 0 and 100. The score is based on continuous penalties, event penalties, and escalation penalties. The system also records supporting artifacts such as score-history points, safety events, and escalation records. This meets the objective of producing an explainable driver reliability score rather than a vague or untraceable result.

The telemetry-processing objective was mostly met. The Android client can process structured telemetry fields such as timestamp, speed, acceleration, drowsiness score, emotional-risk value, lane deviation, and turn-signal state. The software performs mapping, unit conversion, timestamp handling, smoothing, and scoring based on these values. This objective is marked as mostly met rather than fully field-proven because extended real-world testing with continuous live telemetry from the complete prototype still requires more validation.

The offline-first local persistence objective was fully met. Room and SQLite were used to store completed sessions, driver records, vehicles, safety events, escalation records, score-history records, and synchronization metadata. This allows the Android client to preserve completed sessions locally even when backend connectivity is unavailable. The design directly supports the requirement that live scoring and session preservation should not depend on constant internet access.

The Jetson software integration objective was partially met. The Jetson-side service was implemented as an API and WebSocket bridge with health and status endpoints, vehicle-state access, authority-control operations, and telemetry streaming. Manual Android configuration to the Jetson base URL demonstrated the required communication path. However, automatic discovery and seamless propagation of the selected Jetson HTTP and WebSocket URLs still require refinement. Therefore, the API and WebSocket bridge exists and works as a software boundary, but the discovery workflow is not fully complete.

The backend server objective was fully met. A Spring Boot backend-admin system was implemented using Java, Spring Security, Spring MVC, Thymeleaf, Spring Data JPA, Hibernate, MySQL, validation services, upload services, query services, and administrator page controllers. The backend supports authentication, persistent storage, driver management, vehicle management, session review, analytics, and protected administrator access. This exceeded the original Android-only scope and became a major part of the final software contribution.

The completed-session upload objective was fully met. The backend provides an authenticated API-client upload endpoint for completed driving sessions. The upload workflow validates the request, checks references, handles duplicate session behaviour, stores the session summary, persists child

records, records ingestion issues where applicable, and returns a structured response. This completed the connection between Android session production and backend historical storage.

The administrator web objective was fully met. The backend includes protected administrator pages for login, dashboard review, drivers, driver creation, driver details, driver analytics, vehicles, vehicle creation and editing, sessions, session details, fleet analytics, settings, and profile. These pages allow administrators to review completed sessions and inspect the supporting evidence behind reliability scores.

The backend analytics objective was fully met. The backend calculates dashboard and analytics values such as average final score, total distance, daily score trends, event-type counts, escalation-type counts, driver status counts, vehicle status counts, session validity counts, invalid-session rate, warning-session rate, completed-session rate, active-driver share, and busy-vehicle share. These analytics allow the backend to act as a review and decision-support layer rather than only as a database.

The monitoring and operational visibility objective was fully met at the prototype level. Spring Boot Actuator, Micrometer and Prometheus support, node exporter, Prometheus, and Grafana were used to provide runtime visibility for the deployed backend environment. This supports troubleshooting and server observation. However, production-grade observability would still require stronger alerting, backup monitoring, and longer operational testing.

The integration objective was partially met. The project successfully defined and implemented the main software integration boundaries between the Jetson service, Android client, backend upload API, admin web system, database, and monitoring stack. The final architecture demonstrates the intended end-to-end software path. However, the full integrated system still needs longer live testing with Android, Jetson, backend server, and the physical prototype operating together over extended driving sessions. For this reason, integration is considered implemented at the software-contract level but only partially validated at the full operational level.

The security objective was fully met at prototype level. Administrator routes are protected by authenticated administrator access, and completed-session upload is protected by API-client authentication. Passwords are stored using hashing rather than plain text. The system separates administrator access from machine upload access. Production use would still require HTTPS enforcement, stronger secret management, stricter deployment policies, and more security testing.

The production-readiness objective was partially met. The backend was deployed as a running service and connected to monitoring, which shows that the software moved beyond local-only execution. However, production hardening remains incomplete. Required future work includes managed database migrations, HTTPS enforcement, backup and recovery procedures, stronger secret storage, service restart testing, and longer reliability testing.

No core objective across either subsystem was completely unmet. All AI perception objectives were fully achieved. The main Android, backend, admin web, upload, persistence, scoring, Jetson API and WebSocket, analytics, and monitoring software components were implemented. The objectives marked as partially met are mainly those requiring extended real-world validation, automatic discovery refinement, or production hardening. Overall, the achieved outcomes satisfy the main capstone objectives across both the AI, mechattonics and software subsystems and provide a complete foundation for future improvement.

Mechatronics:

Objective (proposal)	Outcome	Met?
Custom 3D-printed chassis housing Jetson/ESP32/sensors/power	CAD complete; printing ~85%	Partial
ESP32 firmware: motor (via ESC), servo, IMU, Hall-effect, UART	All modules bench-verified	Yes (bench)
High-speed bidirectional UART link to Jetson	Established and verified, no loss	Yes (bench)
Black-Box SD-card logging	Configured and verified	Yes (bench)
Physical track testing on varied surfaces	assembled	Yes
RC-baseline analysis	Dropped — direct CAD design	Re-scoped
LiDAR (RPLIDAR A1M8)	Removed; reassigned to AI camera	Re-scoped
DC motor + motor-driver IC	Replaced by brushless + 60 A ESC	Re-scoped

Figure 6.3.1 Mechatronics Objective Comparision

7. Project Management Summary

7.1 Work Breakdown and Schedule

The overall AI system was developed using a task-based and phase-based schedule, split between the in-cabin AI subsystem and the Road Monitoring subsystem. The in-cabin AI work was divided into four main modules: face detection, drowsiness detection, emotion recognition, and head-pose estimation. Halit Barboti and Emad Alden Aleassa were responsible for the main implementation and testing of these modules.

The Road Monitoring system was first validated using the CARLA simulator, where YOLOPv2 was deployed in a controlled driving environment to perform lane detection. Lane information was used to compute lane deviation and verify lane-drift logic before moving to the Jetson Orin Nano and RC car implementation.

Both subsystems followed iterative development cycles. The in-cabin AI followed a structured sequence: requirement analysis, face detection and landmark pipeline development, drowsiness detection, emotion recognition, head-pose estimation, and final integration and testing. The Road Monitoring system followed six phases covering camera integration, YOLOPv2 deployment, HSV-based lane deviation estimation, output interface and automation, TensorRT optimisation, and final calibration and testing.

In the Road Monitoring pipeline, early work focused on Jetson setup, JetPack 6.1 configuration, and IMX219 CSI camera integration using GStreamer. YOLOPv2 was then deployed in TorchScript format and verified to produce segmentation outputs, while a 200-image dataset of the physical track was collected and annotated. Fine-tuning was attempted but was not possible due to TorchScript weight freezing.

A custom HSV-based lane deviation estimator was then developed to improve accuracy. This included ROI cropping, histogram-based lane detection, peak grouping, deviation calculation, and stability improvements such as smoothing and fallback handling. Output handling and automation were implemented using a file-based interface, a bridge module, and autostart configuration on boot.

TensorRT optimisation was applied to improve performance, converting the model from TorchScript to ONNX and then to a FP16 engine. This required resolving compatibility issues, including removing the unsupported SequenceConstruct node and correcting input resolution settings. Final calibration and testing were performed on the RC car, including parameter tuning for stable lane-centre detection.

Overall, both subsystems followed their planned development structure with minor delays due to tuning and integration challenges. The Road Monitoring system achieved all major objectives except YOLOPv2 fine-tuning, which was blocked by format limitations. Additional time in both subsystems was required for threshold tuning, output alignment, and integration with the Jetson-based API and communication pipeline.

The software team followed a parallel iterative work breakdown across two phases. Phase 1 covered the Android client and was divided into domain scoring, local persistence, telemetry and network boundaries, orchestration, synchronization, ViewModels, UI screens, localization, mock and debug support, and testing. Phase 2 covered the Spring Boot backend-admin system and was divided into backend project setup, domain and persistence modeling, repositories and mappers, command services, query services, validation, security, REST APIs, admin pages, analytics, seed data, monitoring, and tests. Both phases progressed iteratively, with each work area building on the previous one before moving to the next layer of functionality.

Figure 18: Gnatt Chart

7.2 Individual Contributions

Team Member	Department	Key Contributions	Overall Effort (%)
Halit Barboti	Artificial Intelligence	Shared responsibility for the in-cabin AI subsystem; contributed to face-detection pipeline, camera-frame processing, drowsiness module support, emotion and head-pose integration, AI output preparation for Jetson-hosted communication, testing, and documentation.	25%
Emad Alden Ahmad Hasan Aleassa	Artificial Intelligence	Shared responsibility for the in-cabin AI subsystem; contributed to drowsiness detection logic, EAR/MAR-based fatigue analysis, CNN eye-state classification support, emotion model integration, head-pose module support, threshold tuning, testing, and documentation.	25%
Joud Wardeh	Artificial Intelligence Engineering	Shared responsibility of the Road Monitoring subsystem CARLA Environment Setup YOLOPv2 set up on CARLA CARLA lane deviation logic Speed/acceleration and hybrid obstacle detection Different Weather simulations on CARLA Setting up imx219 camera on jetson Built fake road	25%
Rama Ayman Moh'd Ghazi Al-Tamimi	Artificial Intelligence Engineering	Shared responsibility of the Road Monitoring subsystem Lane Deviation Logic on Jetson YOLOPv2 set up on CARLA AI interface module for integration YOLOPv2 deployment on Jetson TensorRT conversion Built fake road Dataset collection and annotation for potential fine-tuning.	25%
Zaid Khalid Jasim Jasim	Software Engineering	Phase 1 Android implementation, client architecture, UI/ViewModel	50%

		work, telemetry/backend integration boundaries, testing support, and Phase 2 backend/API/persistence support	
Mohammed Omar Mosleh Mosleh	Software Engineering	Phase 2 backend/admin implementation, upload ingestion and validation review, admin dashboard/report preparation, integration review, and Phase 1 support in app/security/session/mock infrastructure	50%
Ashraf Atheer Ali Al-Attabi	Mechatronics Engineering	• Component procurement • ESP32 firmware — brushless motor control (60 A ESC) • ESP32 firmware — servo steering • ESP32 firmware — IMU acquisition (I^2C, MPU6050) • ESP32 firmware — Hall-effect speed sensing • ESP32 firmware — watchdog / fail-safe logic • ESP32 firmware — watchdog / fail-safe logic • UART link (firmware side) • Configure the imx219 camera • Flashing the jetosn and uploading the op system on the jetson Bench-level integration testing	50%
Karam Zuhier Thaher Qushtom	Mechatronics Engineering	• Component procurement • SolidWorks chassis design • Vibration-dampening sensor mounts • UART telemetry / JSON schema alignment	50%

		• Jetson power-supply design • Bench-level integration testing • ESP32 firmware — IMU acquisition (I^2C, MPU6050) • ESP32 firmware — brushless motor control (60 A ESC) • Configure the imx219 camera • Flashing the jetosn and uploading the op system on the jetson	

Table 3: Individual Contributions

7.3 Inter-Departmental Collaboration Assessment

The project required collaboration between the Software, AI, and Mechatronics departments because the final system depended on more than one technical discipline. The in-cabin AI subsystem required the closest collaboration between the Artificial Intelligence and Software departments. The AI team was responsible for generating processed driver-state indicators, while the Software Department was responsible for hosting the WebSocket and API communication layer on the NVIDIA Jetson Orin Nano and integrating the received data into the Driver Reliability Score and application interface. More broadly, the Software Department was also responsible for Android development, backend server development, Jetson API and WebSocket integration, administrator web development, database persistence, monitoring, and system-level integration. The AI and Mechatronics departments contributed the perception and physical vehicle sides of the prototype, while the Software Department converted their outputs into usable application workflows, scoring results, storage records, and administrator analytics.

This separation of responsibilities improved the structure of the project. The AI team focused on detection accuracy, module reliability, and meaningful driver-state outputs. The software team focused on communication, data transfer, application logic, scoring implementation, and user-facing display. As a result, the AI subsystem did not need to manage the full software communication stack, and the software subsystem did not need to process raw camera frames.

The joint-department approach worked well because each department contributed a necessary part of the complete system. The AI side provided behavioural-risk indicators such as drowsiness, emotional-risk value, and lane-deviation information. The Mechatronics side provided vehicle-related values and physical prototype behaviour. The Software Department then consumed these values through structured interfaces and used them for Android scoring, session recording, backend upload, and administrator review. This created a clearer complete system than would have been possible if each department worked only on an isolated demonstration.

The collaboration was most important during the definition of AI outputs. The AI subsystem needed to provide values that were simple enough for the software system to consume, such as face presence, drowsiness level, emotion label, distress event, head-pose status, FPS, and timestamp. This required the AI outputs to be structured as processed indicators rather than raw model outputs. One challenge in this collaboration was ensuring that the AI outputs matched the expected data format of the software-side Jetson server. This was handled by keeping the output simple and consistent, allowing

the software team to receive driver-state indicators through the Jetson-hosted API and WebSocket layer and use them in the Driver Reliability Score.

The most effective part of the broader collaboration was the use of integration boundaries. Instead of requiring the Android application to understand the internal details of AI inference or mechanical sensor acquisition, the system was designed around structured telemetry fields. This allowed the software side to focus on timestamp handling, telemetry mapping, scoring, persistence, upload contracts, backend validation, and analytics. At the same time, AI and Mechatronics could continue developing their own internal components as long as the agreed telemetry output remained compatible.

The Jetson-side integration also became an important collaboration point. The Jetson service acted as the software bridge between external telemetry sources and the Android client. By exposing API endpoints and a WebSocket telemetry stream, the Jetson software allowed the Android client to interact with the prototype through defined software operations rather than through direct hardware access. This helped separate live telemetry delivery, authority and state handling, Android session logic, and backend administration into a more controlled system architecture.

The backend-admin system improved inter-departmental collaboration by providing a final destination for completed software results. Once the Android client produced scored sessions, the backend stored them and made them visible through administrator pages and analytics. This helped demonstrate the value of the integrated system because the output of AI, Mechatronics, Android, and Jetson integration could become a permanent backend record rather than a temporary live display.

One challenge in the collaboration was that interface expectations changed during implementation. Values such as lane deviation, speed, acceleration, and telemetry timing needed clear interpretation before they could be used reliably in scoring. For example, lane deviation may be interpreted as a normalized AI value or as a physical measurement, and speed may need conversion before braking and distance calculations. The solution was to document the software assumptions and place normalization logic inside the Android telemetry-mapping and scoring layers.

Another challenge was that integration timing did not always match individual department progress. The software side needed stable telemetry contracts before finalizing scoring and backend upload structures, while AI and Mechatronics outputs were still evolving. This created the need for mock data, manual configuration, and contract-based testing. This approach allowed the Android and backend components to progress even when the full physical integration was not continuously available.

A further challenge was validating the complete end-to-end system. Repository tests can validate Android scoring, backend upload, security, persistence, and analytics, but they cannot fully replace long-duration integrated testing with the Android device, Jetson service, backend server, and physical prototype running together. The project therefore achieved strong software-level integration, but full operational validation remains an area for improvement.

From the Mechatronics side, inter-departmental collaboration centred on two relationships: supplying vehicle telemetry to the Software department through the agreed JSON contract, and provisioning the shared Jetson Orin Nano platform that both the AI and Software departments built on top of. Both relationships required the Mechatronics team to think of its outputs as a stable interface rather than an internal implementation detail, which shaped several firmware decisions that would otherwise have been made differently.

The telemetry contract with Software was agreed early, at the level of individual field names, units, and types speedKmh, steeringAngleDeg, accelerationMps2, turnSignal, ignitionOn, stationary rather than left to be inferred from whatever the firmware happened to output. This had a direct, practical benefit during development: because the schema was fixed, the Software team could build and test its telemetry mapper and scoring logic against mock or replayed JSON packets without waiting for the

physical vehicle to be available, while the Mechatronics team could continue iterating on sensor calibration, ESC behaviour, and IMU axis mapping without those changes ever touching the field names or packet structure Software depended on. This mirrors the same lesson the AI–Software collaboration reached independently: agreeing the exact contract before either side commits to implementation removes an entire category of late integration bugs.

A concrete example of where this boundary mattered was the obstacleDistanceMeters field. The ranging sensor originally planned for this field (RPLIDAR A1M8) was removed from scope after it proved unavailable locally within budget, partway through development. Because the field had already been defined as part of the contract, the Mechatronics team could substitute a documented placeholder constant without requiring any change on the Software side the scoring engine consumes the field exactly as it would a real measurement, and the substitution is recorded as a known limitation rather than surfacing as an integration break.

A second, less visible but equally important collaboration point was Jetson Orin Nano bring-up. Before any AI inference or Jetson-side software could run, the device needed to be flashed, networked, and have its camera and serial interfaces validated work that does not belong naturally to either the AI or Software department's core focus, but blocks both if left undone. The Mechatronics team carried out this bring-up directly: flashing JetPack, validating the IMX219 camera pipeline, provisioning a Jetson-compatible (aarch64) Python/PyTorch/TensorRT environment, configuring USB serial port permissions for the telemetry stream, and setting up the FastAPI server to launch automatically on boot. Handling this centrally meant the AI and Software teams received a platform that was already operational rather than each needing to independently solve Jetson-specific provisioning problems alongside their own development work.

One collaboration challenge specific to the Mechatronics boundary was keeping the serial link to the Jetson strictly machine-readable. During bench development, human-readable diagnostic output on the serial port was essential for debugging but the same port, once connected to the Jetson, needed to carry only clean JSON, since the FastAPI server's parser would raise an exception on any non-JSON line and silently interrupt the telemetry stream. This was resolved with a single compile-time flag gating all diagnostic output, but it only became a known requirement after coordinating with the Software team on exactly how the Jetson-side parser would behave on malformed input — a detail that would not have surfaced from the Mechatronics side alone, since the firmware behaved correctly in isolation on the bench.

A further challenge was that some Mechatronics-side hardware limitations only became relevant once Software began designing scoring logic around the telemetry fields. The Hall-effect sensor's inability to distinguish forward from reverse motion, for instance, matters far more to a department building braking-aggression penalties from speed and deceleration data than it does to the Mechatronics team validating that the sensor reports a sane magnitude. Surfacing this kind of field-level limitation explicitly rather than letting Software discover it empirically during scoring validation required treating documentation of sensor caveats as part of the deliverable, not an afterthought to it.

Looking back, the collaboration would have benefited from agreeing the telemetry contract slightly earlier than it was, specifically around failure and edge-case behaviour rather than only the happy-path field definitions — for example, what a consuming system should assume when stationary is true but speedKmh has not yet decayed to exactly zero, or how a brief Bluetooth dropout should be reflected in the telemetry stream versus silently resolved at the firmware level. These questions were resolved adequately during development, but earlier joint specification of edge-case behaviour, alongside the field-level contract that was already agreed, would have reduced the small amount of back-and-forth that occurred once Software began building scoring logic against real (rather than mocked) telemetry.

The collaboration could be improved by defining the telemetry contract earlier and freezing it before major implementation. A formal shared document should define each field name, unit, range, update

frequency, meaning, and failure behaviour. This would reduce ambiguity between departments and prevent late changes from affecting Android mapping, scoring thresholds, backend DTOs, and analytics. This mirrors the observation from the AI and Software collaboration, where an earlier definition of the final communication contract, including exact field names, data types, update rate, and error-handling rules, would have further reduced integration friction.

The collaboration could also be improved by using more scheduled integration checkpoints. Instead of testing department outputs only near the end, the teams should regularly run short integration sessions where Jetson telemetry, Android scoring, backend upload, admin web review, and Grafana monitoring are checked together. This would expose contract mismatches, network issues, timestamp problems, and upload errors earlier.

Overall, the joint-department approach was effective because the project required perception input, vehicle telemetry, mobile processing, backend administration, and monitoring to work together. The Software Department's contribution was to connect these pieces into a usable system pipeline. The main lesson is that multidisciplinary projects require strict interface contracts, frequent integration testing, and clear ownership of responsibilities. When these are present, each department can work independently while still contributing to one complete system.

7.4 Budget and Resources

The proposal budgeted ৳15,960 from the university fund, dominated by the RPLIDAR A1M8 (~৳8,566) and an RC chassis (৳3,500), neither of which was purchased after the LiDAR was dropped and the team moved to a custom 3D-printed chassis. The reconciled actuals from the project ledger are:

Components purchased against the university budget (with recorded prices and receipts):

- Servo MG995 — ৳1,489.00 — receipt SE02026000000018 (Karam)
- Filament 1 kg (PLA+) — ৳600.00 — AAA2026000000432 (Karam)
- LiPo battery + battery tester — ৳2,100.00 — RBT2026000000358 (Karam)
- XT60 connectors — ৳431.00 — BOM2026000008243 (Karam)
- TPU filament — ৳936.00 — CEF2026000006790 (Karam)
- Camera 200 mm V2 cable — ৳2,075.00 — ROBOTISTAN BOM2026000004814 (Ashraf)
- 100 resistors — ৳87.40 — TRENDYOL TYE2026000000049 (Ashraf)
- ESP32 — ৳462.00 — TRENDYOL TYE2026000000257 (Ashraf)
- Micro SD card 64 GB — ৳1,020.00 — GIB2026000000015 (Ashraf)
- 6802 bearings — ৳499.90 — TRN2026000000308 (Ashraf)
- Metal servo attachment — ৳149.90 — CDA2026000001066 (Ashraf)
- 2× 1 m M3 threaded rods — ৳594.00 — ECF2026000000104 (Ashraf)
- M4 nylon nut ×10 — ৳82.80 — TYE2026000000017 (Ashraf)
- Motor D3542 1000KV — ৳1,948.80 — RLE2026000004712 (Ashraf)
- M4 12 mm socket head — ৳202.60 — BOM2026000007575 (Ashraf)

- Wheels and soldering board — 1,800 (Karam)
- Servo motor 30kg—1,040 (Ashraf)

Components purchased but without an individual recorded price (bundled or low-cost):

- Breadboard ×2, ESC 30A, micro SD card reader, small gears set, disc magnets, IMU MPU6050, Hall-effect sensor, DC barrel jack connector (Karam)
- Jumper wires (M-M, F-F, M-F), mini breadboard ×2, L298N motor driver, IMX219 170° camera, 220 Ω resistors, M3 nylon nut ×10 (Ashraf)

Components purchased personally (outside the university budget — "no uni bill"):

- 80 mm suspension — ৳756.00 (Ashraf)
- 60 A ESC — ৳1,065.00 (Karam)
- Screws — ৳815.00 (Karam)
- Boards for the fake road — 800 (Rama)

Not purchased / substituted / considered:

- LiPo charger + LiPo bag — not bought (found in the university lab)
- RC transmitter + receiver — not bought (substituted with a console / PS4 controller)
- Raspberry Pi Zero 2 W (~৳953, Robotistan) — considered, then replaced with the ESP32-CAM / webcam approach

Reconciled totals:

- Total spent (all sources): ৳21,914
- Spent from the university budget: ৳15,478.40
- Remaining from the university budget: ৳521
- Spent outside the university budget (personal): ৳6,436.00
- Karam's total spend: ৳8,936.00
- Ashraf's total spend: ৳10,418.40
- Rama's total spend : ৳800

The ai department used open-source tools including Python, PyTorch, OpenCV, ONNX Runtime, YOLOPv2, and CARLA. Hardware resources included the NVIDIA Jetson Orin Nano and the RC-car prototype.

The software resources used were primarily development tools and open-source frameworks: Android Studio, Gradle, Kotlin, Jetpack Compose, Room, Retrofit, OkHttp, Java, Spring Boot, Spring Security, Spring Data JPA, MySQL, Thymeleaf, Actuator, Micrometer, JUnit, Mockito, Git, and supporting development environments. No separate paid software budget was made.

8. Ethical, Safety, and Sustainability Considerations

The project raises important ethical, safety, and sustainability considerations across both the AI and software subsystems. The in-cabin AI subsystem uses a driver-facing camera to analyze facial behavior, while the software subsystem handles driver-related data, scoring outputs, session history, and administrator access. Together, these design choices require careful attention to privacy, fairness, system safety, and long-term maintainability.

Ethics

The main ethical concern on the AI side is driver privacy. The in-cabin camera captures the driver's face, which is sensitive personal data. To reduce this risk, the AI subsystem is designed to process camera frames locally and provide only processed driver-state indicators. The system does not require raw video to be transferred as the main AI output. Instead, it produces values such as face presence, drowsiness level, emotion category, head-pose status, FPS, and timestamp. This design follows the principle of data minimization because only the information needed for driver monitoring is shared with the rest of the system.

Another ethical consideration is the limitation of emotion recognition. Facial emotion recognition is not always fully reliable because expressions can vary between individuals, cultures, lighting conditions, and camera angles. For this reason, the project does not use the emotion model to make a psychological judgment about the driver. The original emotion model has eight classes, but the operational system uses only sad, angry, and neutral. All other emotions are mapped to neutral. This reduces the risk of overinterpreting facial expressions and keeps the emotion module as a supporting indicator rather than a final decision-making authority.

The system also avoids driver identity recognition. The purpose of the face detector is only to locate the driver's face region so that drowsiness detection, emotion recognition, and head-pose estimation can operate correctly. The system is not designed to identify who the driver is, verify identity, or store biometric identity records. This distinction is important because the project's goal is safety monitoring, not surveillance or biometric authentication.

On the software side, a major ethical concern is that the system produces a Driver Reliability Score that could influence how a driver is interpreted by an administrator. Therefore, the score must be treated as a decision-support indicator rather than as an absolute judgment of a person's ability, behaviour, or employment suitability. The software system stores driver accounts, vehicle assignments, completed sessions, final scores, penalty totals, events, escalations, score-history points, timestamps, distances, and upload-processing information. These records can reveal behavioural patterns and safety-related information about individual drivers. The backend therefore protects administrator pages through authenticated access, and completed-session upload is protected through authenticated API-client access. This reduces the risk of unauthorized access to driver records.

The software design also avoids unnecessary storage of raw sensitive data. The Android client and backend operate on structured telemetry values and scoring artifacts rather than storing raw facial video or unnecessary biometric recordings. AI-related values such as drowsiness, emotional-risk, and lane-deviation indicators are consumed as numerical outputs through the telemetry boundary. This supports data minimization because the backend only needs completed-session records, scoring evidence, and analytics values for administrator review.

Explainability is another ethical consideration. A reliability score without explanation could be misleading or unfair. To address this, the scoring model separates continuous penalties, event penalties, and escalation penalties. The system stores supporting evidence such as score-history points, detected events, escalation records, validity status, and upload-processing status. This allows an administrator to review why a score changed rather than relying only on a final number. The purpose is to make the scoring process auditable and less arbitrary.

Bias and misinterpretation are also relevant. Drowsiness, emotional-risk, and lane-deviation indicators may be affected by camera conditions, sensor noise, lighting, driver posture, temporary occlusion, or telemetry inconsistency. The software cannot assume that every risk indicator is perfectly accurate. For this reason, the Driver Reliability Score is treated as a software-generated indicator based on available telemetry, not as a definitive human judgment. Any real deployment should include human review, appeal mechanisms, and clear policy rules before using score data for disciplinary or employment decisions.

Safety

From a safety perspective, the in-cabin AI subsystem is designed as a support system rather than a direct vehicle-control system. It does not control steering, braking, acceleration, or any physical vehicle actuator. Its role is to provide driver-state indicators to the software scoring layer. This reduces safety risk because the AI module does not directly take control of the vehicle based on uncertain predictions.

False positives and false negatives are important safety concerns. A false positive may incorrectly classify a driver as drowsy, distressed, or distracted, while a false negative may fail to detect an actual risky state. To reduce false positives, the drowsiness and head-pose modules use temporal persistence instead of reacting to a single frame. Normal blinking is not immediately treated as drowsiness because the eyes must remain closed for multiple consecutive frames before a drowsiness event is produced. Similarly, natural short head movements are not immediately classified as distraction unless the abnormal orientation persists.

The system is also designed to fail more safely when the face is not detected. If the face detector does not find a valid driver face, the dependent modules do not force emotion, drowsiness, or head-pose predictions. This prevents the system from producing misleading outputs when the input frame is unreliable.

Another important thing to factor in when it comes to safety is the simulation done inside CARLA. Testing entirely in CARLA simulation avoided any real-world risk while still exercising rainy, foggy, and night conditions that would be dangerous to test physically. No human subjects or personal data were involved; all data is synthetic telemetry generated by the simulator.

On the software side, the Android client is designed to support live session scoring without distracting the driver. The user interface should avoid unnecessary interaction during driving and should prioritize clear, minimal presentation. Safety-related review should mainly occur after the session through the backend administrator interface rather than requiring the driver to interact with complex screens during active vehicle operation.

The scoring logic also includes safety-oriented protections against misleading calculations. Invalid or non-increasing timestamps do not create continuous penalty growth. Excessive time gaps are capped so that a network or timing irregularity does not create an unfair penalty. Telemetry smoothing is used to reduce overreaction to single-packet noise. These choices improve the safety and fairness of the software output because the score is less likely to change sharply due to technical noise alone.

Integration safety was addressed through the Jetson-side API and authority-control workflow. The Android client does not treat the vehicle telemetry source as an uncontrolled stream. The Jetson-side software exposes health, vehicle-state, authority, heartbeat, and release operations. This helps prevent unclear ownership of an active session and reduces the risk of multiple clients treating the same vehicle as available at the same time.

Backend safety is mainly related to data integrity and administrative access. The backend validates completed-session uploads, checks driver and vehicle references, handles duplicate session identifiers, and stores session summaries with their child artifacts transactionally. This reduces the risk of

incomplete or inconsistent historical records. The backend also preserves the uploaded Android score instead of silently recalculating it, which prevents historical session results from changing unexpectedly after upload.

Security is an important part of both ethics and safety. The backend separates administrator authentication from API-client upload authentication. Passwords are stored using hashing rather than plain text. However, the system remains a prototype and would require stronger production hardening before real deployment. Future work should include enforced HTTPS, stronger secret management, formal database migration tooling, backup and recovery procedures, access logging, stricter role management, and additional security testing.

Monitoring through Prometheus and Grafana contributes to operational safety. A backend server can fail, restart, lose database access, or show abnormal resource behaviour. Monitoring gives maintainers visibility into the deployed environment and supports troubleshooting. This does not remove the need for functional testing, but it improves the ability to detect and investigate server-side issues during operation.

Sustainability

From a sustainability perspective, the AI subsystem uses edge computing on the NVIDIA Jetson Orin Nano instead of sending raw video continuously to a cloud server. Local processing reduces network dependency, limits unnecessary data transfer, and reduces cloud-computing energy usage. It also improves privacy because sensitive camera frames remain inside the local system.

The project also uses open-source and widely available software tools such as Python, OpenCV, ONNX Runtime, NumPy, and TensorRT-supported deployment. This reduces the need for expensive proprietary software and makes the system easier to reproduce, maintain, and improve. The use of a standard driver-facing camera and edge AI hardware also supports practical sustainability because the subsystem does not require complex or highly specialized sensing equipment at the prototype stage.

Simulation-based testing reduces the need for physical test drives, lowering fuel use and emissions associated with road testing. One design consideration was avoiding misclassification of obstacles (e.g., treating a pedestrian or vehicle as "no obstacle"), since in a real deployment that would be a safety-critical failure; the hybrid detection approach and keyword filtering were tuned specifically to reduce both false negatives and false positives.

On the software side, the layered Android and backend architectures support maintainability by separating UI, domain logic, data persistence, networking, validation, security, query services, and view models. This reduces the cost of future changes because a scoring change, UI change, database change, or telemetry-contract change can be handled in the relevant layer rather than across the entire system.

The offline-first Android design also supports sustainability from an operational perspective. It reduces unnecessary dependence on constant backend communication during live sessions. Instead of streaming every raw packet to the public backend, the Android client processes the session locally and uploads completed-session artifacts after the drive. This reduces network usage and makes the system more resilient in environments with unstable connectivity.

From an environmental perspective, the project is a prototype and does not directly measure environmental impact. However, local scoring, structured uploads, and selective backend storage avoid unnecessary raw-data transfer and reduce the amount of data stored centrally. The backend monitoring stack also helps identify abnormal server behaviour that could waste resources. Future improvements could include more efficient data-retention policies, automatic cleanup of unnecessary logs, optimized database indexes, and clear archival rules for old sessions.

Overall, the project addresses ethical, safety, and sustainability concerns through local edge processing, data minimization, avoidance of identity recognition, simplified emotion output, temporal filtering, protected access, explainable scoring, bounded and smoothed telemetry processing, controlled Jetson integration, transactional backend persistence, monitoring visibility, and maintainable layered design. The remaining limitations are mainly related to real-world deployment: the system should not be used for high-stakes driver evaluation without additional validation, stronger security controls, human review procedures, and clear organizational policies.

9. Conclusions

9.1 Summary of Achievements

This project developed a complete driver monitoring system for the Smart Car and Driver Monitoring App for Commercial Fleet Management. The AI subsystem, implemented by Halit Barboti and Emad Alden Aleassa, analyzed driver-facing camera input and produced processed driver-state indicators to support the Driver Reliability Score. The software subsystem delivered the complete pipeline from telemetry intake to administrator-facing historical review, including the Android thick-client application, Driver Reliability Score engine, local Room and SQLite persistence, Jetson-side API and WebSocket integration bridge, Spring Boot backend server, MySQL historical database, administrator web interface, completed-session upload API, backend analytics, deployment support, and Grafana monitoring.

The Road Monitoring subsystem achieved a complete end-to-end pipeline from simulation to real hardware deployment. YOLOPv2 was first validated in CARLA, where it successfully detected lane structures under different weather and traffic conditions. This allowed early testing of lane deviation logic before moving to physical deployment.

On the Jetson Orin Nano, YOLOPv2 was successfully deployed using TensorRT FP16 optimisation, achieving real-time performance between 15–25 FPS. A custom lane deviation system was implemented to convert segmentation outputs into a stable, normalised deviation score in the range of [0.0, 1.0].

The system was further validated on a physical RC car setup using an IMX219 CSI camera mounted in a downward-facing configuration over a controlled cardboard track. The pipeline successfully produced stable lane deviation outputs in real time.

Successfully delivered a working CARLA-based simulation that drives an ego vehicle, runs YOLOPv2 lane segmentation, computes a stable real-time lane-deviation value, calculates speed and acceleration, and detects both moving and static obstacles using a hybrid detection approach. The system was validated under multiple weather and lighting conditions, with JSONL logs and debug images as evidence, providing the Road Monitoring AI module with realistic, varied test data without any real-world driving risk.

A major engineering achievement was the TensorRT conversion process, which required resolving ONNX compatibility issues including removal of the unsupported SequenceConstruct node and rebuilding the computation graph for static execution.

The drowsiness detection module was one of the strongest AI achievements. It combined Eye Aspect Ratio, Mouth Aspect Ratio, CNN-based eye-state classification, and temporal persistence. This design allowed the system to distinguish normal blinking from sustained eye closure. The drowsiness model achieved approximately 92% accuracy, which shows that it can provide a strong safety-related indicator for the prototype.

The emotion recognition module was also successfully integrated. The original model supports eight emotion classes, but the final operational system uses only three categories: sad, angry, and neutral. Sadness and anger are treated as distress-related indicators, while all other classes are mapped to neutral. This design reduced unnecessary emotional overinterpretation and made the emotion output more suitable for driver-risk scoring. The emotion model achieved approximately 85% accuracy. The head-pose estimation module completed the AI pipeline by producing yaw, pitch, and roll values from facial landmarks, using thresholds and consecutive-frame logic to distinguish sustained distraction from natural short head movements. Together, these AI modules produced structured driver-state indicators that could be transferred to the Jetson-hosted communication layer without requiring raw video transfer.

On the software side, one of the most significant achievements was the implementation of the Android client as a local-first driving-session application. The Android side is responsible for receiving structured telemetry, mapping it into domain-level values, calculating the Driver Reliability Score, detecting events, detecting escalation patterns, storing score-history points, preserving completed sessions locally, and preparing backend upload payloads. This design allows the client to continue scoring and storing sessions even when backend connectivity is unavailable.

The Driver Reliability Score model was another major achievement. The score is bounded between 0 and 100 and is calculated using continuous penalties, event penalties, and escalation penalties. Continuous penalties represent sustained risk signals such as fatigue, lane deviation, and braking aggression. Event penalties represent specific unsafe behaviours such as high fatigue, lane departure, harsh braking, and emotional distress. Escalation penalties represent repeated or combined risk patterns. This makes the score explainable and more suitable for administrator review than a single unexplained output.

The Android local persistence layer was also completed. Room and SQLite were used to store drivers, vehicles, sessions, safety events, escalation records, score-history records, and synchronization metadata. This supports offline-first operation and protects completed session records from being lost when backend upload is not immediately possible. Repository-supported validation showed that the Android project could build successfully and that the local test suite passed, supporting the correctness of the core client-side implementation.

The project also achieved a working Jetson-side software integration bridge. The Jetson service provides API endpoints and WebSocket telemetry access for the Android client. It exposes health and status information, vehicle-state access, authority-management operations, and live telemetry streaming. This allowed the software system to treat the Jetson as a structured service instead of an unmanaged telemetry source. Although automatic discovery still requires refinement, the API and WebSocket integration boundary was implemented and demonstrated through manual configuration.

Another major achievement was the Spring Boot backend-admin system. The backend provides authenticated administrator access, protected routes, completed-session upload ingestion, driver management, vehicle management, session review, driver analytics, fleet analytics, dashboard metrics, settings, and profile workflows. The backend was implemented using a layered architecture with controllers, services, repositories, mappers, validation logic, security components, persistence entities, and view models. This structure supports maintainability and separates business rules, database access, security, and presentation logic.

The completed-session upload API was a key integration achievement. The backend accepts only authenticated API-client uploads, validates uploaded session structures, checks driver and vehicle references, handles duplicate session cases, stores session summaries, persists child artifacts, records ingestion issues where applicable, and updates aggregate driver values. This completed the connection between Android-produced scored sessions and backend historical storage.

The administrator web interface was completed as the human review layer of the system. It allows administrators to log in securely, manage drivers and vehicles, review completed sessions, inspect session details, examine score history, view events and escalations, and monitor fleet-level analytics. This achievement is important because the project does not stop at producing a score on the Android device; it makes the score persistent, reviewable, and useful for fleet administration.

Backend analytics were also implemented. The system calculates average final scores, total distance, daily score trends, event-type counts, escalation-type counts, session status counts, validity counts, invalid-session rates, warning-session rates, completed-session rates, active-driver share, and busy-vehicle share. These analytics convert stored session records into useful administrative summaries.

The project also achieved server deployment and monitoring support. The backend was deployed as a running service, and monitoring visibility was added through Spring Boot Actuator, Micrometer and Prometheus support, Prometheus, node exporter, and Grafana dashboards. This provides operational awareness for the deployed backend environment and supports troubleshooting beyond local development.

The most important overall achievement is the completion of the end-to-end integration. The AI pipeline on the Jetson Orin Nano produces reliable driver-state indicators from the driver-facing camera. The Android client, Jetson service, backend server, admin web interface, database, and monitoring stack were connected into one coherent workflow. The system receives telemetry, performs local scoring, stores completed sessions, uploads historical records, protects administrative access, presents analytics, and exposes monitoring information. This demonstrates that the combined AI and software subsystems achieved the core capstone objective of transforming live camera input and vehicle telemetry into explainable, persistent, and administratively reviewable driver-reliability records.

Over the course of this project, the Mechatronics subsystem delivered a complete, bench-validated vehicle control platform and the shared compute environment that the AI and Software departments built on top of. The key achievements are summarised below.

Real-time vehicle control firmware. A non-blocking ESP32 firmware architecture was implemented that simultaneously drives a 60 A brushless ESC and steering servo from a Bluetooth Xbox Series X controller, samples an IMU and Hall-effect speed sensor, and transmits structured telemetry, all without a single blocking delay() call in the main loop, ensuring controller responsiveness was never traded off against sensor acquisition or telemetry output.

Safety-first control logic. A safe-start gate, a proportional and instantaneous brake distinct from the normal ramped acceleration path, and a controller-disconnect fail-safe were all implemented and validated on the bench before the vehicle was ever driven under its own power. The brake implementation in particular went through multiple bench-tested iterations before reaching a version that correctly handled real driving sequences rather than only the simplified case it was first tested against.

Sensor integration with empirically verified, not assumed, behaviour. The MPU6050 IMU was integrated with gyroscope offset calibration and accelerometer resting-offset correction, with axis mapping confirmed through direct physical testing rather than inferred from the sensor's described mounting orientation. The US1881 Hall-effect sensor was integrated for interrupt-driven wheel-speed measurement, with a spurious-triggering fault diagnosed and corrected through hardware pull-up resistance during bench testing.

A complete, schema-compliant telemetry pipeline. All sensor and control data was packaged into a JSON telemetry contract agreed jointly with the Software department and transmitted at 20 Hz over

USB serial, validated against roughly 12,000 consecutive packets with zero parse errors during sustained bench testing.

A custom-fabricated physical platform. A chassis was modelled in SolidWorks, assembled in FreeCAD, and fabricated via FDM 3D printing in PLA+ and TPU, with dedicated and vibration-dampened mounts for the Jetson, ESP32, IMU, Hall sensor, motor/ESC, and battery, completed in parallel with firmware development through a deliberately decoupled, bench-first methodology.

A simulated and bench-validated power architecture. The vehicle's power protection circuit, covering reverse-polarity protection, EMI filtering, and a regulated supply to the Jetson, was designed, simulated in LTspice across multiple fault scenarios, and validated on hardware, including working around local component-sourcing constraints through validated substitute parts.

Jetson Orin Nano platform bring-up. Beyond the vehicle electronics, the subsystem flashed and configured the Jetson Orin Nano, including JetPack installation, camera pipeline validation, Jetson-compatible Python/AI environment provisioning, and automatic startup configuration for the telemetry server, delivering a ready-to-use compute platform to the AI and Software departments ahead of their own development work.

Transparent documentation of remaining limitations. Where hardware constraints could not be fully resolved within the project timeline, namely the absence of a true active brake, the steering-induced ground noise coupling, and yaw drift inherent to a magnetometer-less IMU, these were characterised, diagnosed to root cause, and documented with identified fixes, rather than left unexamined or implied to be resolved.

9.2 Lessons Learned

One of the main technical lessons learned from the AI subsystem is that driver monitoring should not depend on a single model or a single signal. Drowsiness detection is more reliable when geometric methods, deep learning, and temporal logic are combined. Eye Aspect Ratio provides an interpretable signal, while the CNN eye-state classifier adds visual classification support. Temporal persistence is necessary because normal blinking or short movements should not trigger safety events.

Another lesson is that emotion recognition must be used carefully. Facial expressions can be affected by lighting, face angle, individual differences, and neutral facial behavior. Because of this, the emotion model was not used as a direct psychological judgment tool. Instead, the system simplified the eight-class emotion output into sad, angry, and neutral. This made the output easier to interpret and reduced the risk of assigning too much meaning to uncertain expressions.

Additionally, it was agreed upon among the team that simulation environments like CARLA are extremely useful for early validation of perception and control logic. YOLOPv2 lane detection and lane deviation concepts were first tested in simulation, which reduced risk before deploying on physical hardware.

Another important lesson is that pretrained perception models are not always sufficient for calibrated real-world measurement tasks. Although YOLOPv2 worked well for lane detection, it was not perfect for accurate deviation scoring on a controlled physical track. This led to the development of a custom HSV-based estimator.

A further lesson was that deploying deep learning models on embedded hardware introduces significant compatibility challenges. The TensorRT conversion process required multiple graph-level

modifications, including removing unsupported ONNX operations and adjusting input resolution constraints.

Finally, integration work showed that AI outputs must be simplified into stable, structured signals. Complex model outputs are not directly usable by software systems unless they are converted into consistent telemetry fields such as lane deviation, speed, and obstacle indicators.

The project also showed the importance of face detection as a reliability gate. If the driver's face is not detected correctly, the following modules may produce unreliable outputs. For this reason, the system avoids forcing drowsiness, emotion, or head-pose predictions when no valid face is available.

From an AI integration perspective, the team learned that AI outputs should be simple, structured, and easy for the software team to consume. Instead of sending raw video or complex model internals, the AI subsystem provides processed indicators. This makes the system more modular and supports privacy-aware integration.

These AI-side lessons connected directly to broader software integration lessons. The most important was that a driver-monitoring system should not be treated as one large application. The final system required Android live scoring, Jetson API and WebSocket telemetry access, backend upload ingestion, MySQL persistence, administrator web review, analytics, deployment, and Grafana monitoring. Each part needed a clear responsibility. When responsibilities were separated properly, the system became easier to understand, test, debug, and explain.

A major lesson was the importance of defining integration contracts early. The Android client, Jetson-side service, and backend server all depend on agreed field names, units, ranges, and meanings. Values such as speed, lane deviation, timestamp, drowsiness score, and emotional-risk value must be interpreted consistently. If a value is treated differently by two components, scoring and analytics can become unreliable. This showed that software integration is not only about connecting APIs; it is also about agreeing on the meaning of the data being exchanged. This mirrors the AI-side lesson that defining the exact communication contract earlier would have improved integration efficiency across departments.

Another lesson was that offline-first design is important for mobile systems connected to physical prototypes. The Android client cannot depend on backend connectivity during every active driving session. The decision to perform scoring locally and store completed sessions using Room and SQLite made the application more resilient. This also showed the value of local persistence, synchronization metadata, and retry-capable upload workflows.

The project also showed the value of explainable scoring. A final driver reliability score is not enough unless the system can show how that score was produced. Separating the score into continuous penalties, event penalties, and escalation penalties made the result easier to justify. Storing score-history points, safety events, and escalation records also made the backend review more meaningful. This lesson is important because systems that affect driver evaluation should not rely on unexplained outputs.

A further technical lesson was that backend systems need more than data storage. The Spring Boot backend had to handle authentication, authorization, upload validation, duplicate-session handling, transactional persistence, driver and vehicle management, session review, analytics, and administrator pages. The project showed that backend design must consider security, data integrity, usability, and long-term maintainability from the beginning.

The Jetson integration also provided an important lesson. A local telemetry source should be exposed through a controlled software interface instead of being treated as an unmanaged script or raw stream. The Jetson API and WebSocket bridge made it possible for the Android client to check health,

vehicle state, authority status, and telemetry availability through defined endpoints. This made the integration more stable and easier to troubleshoot.

Deployment and monitoring taught another practical lesson. A backend that works locally is not automatically reliable as a deployed service. Server configuration, service status, database connectivity, logs, process failures, and runtime metrics all matter. Adding Prometheus, node exporter, Actuator and Micrometer support, and Grafana provided useful operational visibility. This showed that monitoring is part of the software system, not only an optional addition.

From a project management perspective, the project showed that scope can expand quickly when a prototype becomes a complete system. The original work was centered mainly on the Android thick client and scoring model, but the final software scope included backend administration, upload APIs, Jetson integration, deployment, monitoring, and system-level integration. This made it necessary to prioritize core workflows first: scoring, persistence, upload, backend storage, admin review, and integration. Teamwork also required clear ownership. Work became more manageable when each component had a defined owner and when responsibilities were separated by module.

Inter-departmental work showed the importance of regular integration checkpoints. AI and Mechatronics outputs are useful to the Software Department only when they are available through stable, structured telemetry. If integration is delayed until the end, problems with field meanings, timing, connectivity, or value ranges become harder to fix. Future projects should schedule earlier and more frequent integration tests between Jetson telemetry, Android scoring, backend upload, admin review, and monitoring.

Another lesson was that testing must be layered. Unit tests are useful for scoring and backend logic, but they cannot fully replace live integration testing. Repository-supported tests helped validate important software behaviour, but full confidence requires longer real-device testing with the Android phone, Jetson service, backend server, and prototype vehicle operating together. The project therefore demonstrated the difference between code-level validation and operational validation.

Overall, the main lessons learned span both subsystems. On the AI side, reliable driver-state detection requires combining multiple signals, applying temporal logic, and keeping outputs simple and structured. On the software side, successful integration depends on clear contracts, layered architecture, local resilience, explainable logic, secure backend design, and continuous validation. Together, these lessons reflect how mobile software, backend systems, local telemetry services, AI perception, web administration, deployment, and monitoring must be designed as one coherent engineering system rather than as isolated components.

9.3 Future Work and Recommendations

Future work should focus on improving quantitative evaluation, robustness, integration reliability, and real-world deployment readiness across both the AI, mechatronics and software subsystems.

On the AI side, future testing should use a larger and more diverse dataset covering different drivers, lighting conditions, camera angles, glasses, face shapes, and facial expressions. Although the drowsiness and emotion models achieved approximately 92% and 85% accuracy respectively, these figures should be validated further under realistic in-cabin conditions. The face detection and head-pose modules should also be evaluated with more detailed quantitative metrics, including detection rate, false-positive rate, head-pose error, and performance under difficult lighting or occlusion conditions, as these modules were validated functionally in the current work rather than through full quantitative benchmarking.

The drowsiness module can be improved by testing additional thresholds and adapting them to different users. Some drivers may naturally have smaller eye openings or different blinking patterns, so a calibration step could improve reliability. The system may also benefit from infrared camera

support for night-time operation. The emotion module can be improved by fine-tuning on driver-specific or in-cabin facial-expression data. Since the operational output uses only sad, angry, and neutral, future work could train a smaller model specifically for these three categories instead of mapping an eight-class model afterward. The head-pose module can be improved by calibrating the camera position more carefully, since yaw, pitch, and roll depend on camera angle, and a calibration procedure would help make distraction thresholds more accurate across different vehicle setups.

Future work on the Road Monitoring subsystem should focus on improving the accuracy and robustness of lane deviation estimation across both simulated and real environments.

The HSV-based estimator could also be enhanced by adaptive thresholding and automatic recalibration for different lighting conditions and camera positions. This would improve generalisation across different environments beyond the current fixed track setup.

On the deployment side, future work could simplify the TensorRT conversion pipeline by using more stable ONNX export configurations or newer TensorRT-compatible model architectures to avoid graph-level manual fixes. Adding a GPS feature in the project would also be something possible in the future.

Finally, deeper integration between CARLA simulation and real-world Jetson deployment could be developed so that simulation parameters directly mirror physical track calibration values, enabling faster testing and tuning cycles.

For AI-to-software integration, future work should define the communication contract at the start of development. This contract should include field names, data types, update rate, timestamp format, missing-face behavior, and error-handling rules. Defining this early would reduce integration delays between departments and prevent ambiguity in how AI outputs are interpreted by the Android scoring and backend systems.

On the software side, the first recommendation is to perform longer end-to-end integration testing with the Android client, Jetson service, backend server, administrator web interface, database, and Grafana monitoring running together. Longer live sessions are needed to observe telemetry stability, WebSocket reconnection behaviour, authority handling, upload retry behaviour, backend persistence, and dashboard consistency over time. Future testing should include repeated session starts and stops, network interruptions, backend downtime, Jetson restart cases, duplicate upload attempts, and recovery after failed uploads.

A second recommendation is to complete and refine automatic Jetson discovery. Manual configuration of the Jetson base URL demonstrated the API and WebSocket communication path, but a complete user experience should not require manual entry of the Jetson address. The Android client should automatically detect the correct vehicle network, identify the Jetson host, construct the HTTP base URL and WebSocket URL, verify the Jetson health endpoint, and store the selected vehicle configuration.

A third recommendation is to strengthen the Jetson-side API and WebSocket service. The Jetson service should include more detailed health reporting, clearer telemetry status messages, structured error responses, improved heartbeat handling, and more robust recovery when telemetry sources temporarily fail. The authority workflow should also be tested under repeated claim, cancel, heartbeat, and release operations.

A fourth recommendation is to improve Android synchronization reliability. The Android client should maintain a clear upload queue for completed sessions, with retry policies, upload status labels, last-attempt timestamps, and clear error messages. Failed uploads should remain visible, and retry behaviour should avoid creating duplicate backend records.

A fifth recommendation is to expand Android field testing. Future tests should examine battery usage, UI responsiveness during live telemetry, local database growth, behaviour after app restart, behaviour after phone network changes, and performance during long sessions. This is especially important because driver-monitoring software must remain stable during continuous operation.

A sixth recommendation is to improve backend production hardening. The backend should enforce HTTPS, use stronger secret management, move credentials and sensitive configuration out of source code, and apply stricter deployment profiles. Database migrations should be managed using a formal migration tool, and backup and recovery procedures should be defined. The backend should also include clearer service restart policies and tested recovery steps after server reboot or application failure.

A seventh recommendation is to strengthen backend security. Future work should add role-specific permissions, access logs, login-attempt limits, stronger password policy enforcement, session timeout rules, and audit trails for important administrator actions. Since the system stores driver-related safety records, security should be treated as a central requirement rather than a final add-on.

An eighth recommendation is to improve backend analytics and reporting. Future work could add date-range filtering, per-driver comparison, vehicle-level comparison, exportable reports, event heatmaps, score trend summaries, risk-category breakdowns, and administrator notes on sessions. The administrator web interface should also include clearer session-detail pages, better chart readability, stronger filtering, search, sorting, pagination, and more direct links between drivers, vehicles, sessions, events, escalations, and score history.

A ninth recommendation is to expand Grafana and monitoring support. Future work should add alerts for backend downtime, high memory use, high CPU use, database connection problems, failed upload spikes, abnormal request rates, and repeated service restarts. Grafana dashboards should be organized into separate views for server health, backend application metrics, database-related behaviour, and upload activity.

A tenth recommendation is to improve data retention and privacy controls. The system should define how long session records, score-history points, ingestion issues, logs, and monitoring data are kept. This is important because driver-related records can become sensitive over time. Future work should also continue avoiding unnecessary storage of raw facial video or unrelated personal data.

An eleventh recommendation is to improve scoring calibration through controlled testing. The Driver Reliability Score is explainable and deterministic, but its thresholds and penalty weights should be validated through more test scenarios comparing score outputs across normal driving, harsh braking, sustained fatigue indicators, lane-deviation events, emotional-risk indicators, and combined risk patterns.

A twelfth recommendation is to preserve the backend responsibility boundary. Future developers should avoid moving live scoring to the backend unless the architecture is intentionally redesigned. The Android client has the ordered live telemetry stream, while the backend receives completed historical records. If backend recomputation is introduced later, it should be clearly versioned and should not silently overwrite historical Android-produced scores.

A final recommendation is to treat the project as a foundation for a production-style system rather than a finished commercial product. The main architecture is in place across both subsystems: AI perception on the Jetson Orin Nano, Android local scoring, Jetson telemetry access, backend ingestion, admin review, persistence, analytics, and monitoring. The next stage should focus on reliability, usability, security, field validation, and operational hardening. Future work should also create a formal shared integration contract defining each telemetry field, unit, range, update frequency, nullability, failure behaviour, upload payload structure, backend response structure, Jetson API endpoints, WebSocket message format, authority-state transitions, and error codes. With these improvements,

the system could become a robust fleet driver monitoring platform suitable for extended testing and eventual real-world evaluation.

References

[1] A. Madane, A. Singh, S. Fargade, and A. Dongare, "Real-time driver drowsiness detection using deep learning and computer vision techniques," *Int. J. Sci. Res. Sci. Eng. Technol.*, vol. 12, no. 3, pp. 222–229, May 2025.

Link: https://doi.org/10.32628/IJSRSET2512335

[2] R. Florez-Zela, F. Palomino-Quispe, A. B. Alvarez, R. J. Coaquira-Castillo, and J. C. Herrera-Levano, "A real-time embedded system for driver drowsiness detection based on visual analysis of the eyes and mouth using convolutional neural network and mouth aspect ratio," *Sensors*, vol. 24, no. 19, p. 6261, Sep. 2024.

Link: https://doi.org/10.3390/s24196261

[3] Z. Wang, X. Huang, and Z. Hu, "Attention-based LiDAR–camera fusion for 3D object detection in autonomous driving," *World Electr. Veh. J.*, vol. 16, no. 6, p. 306, Jun. 2025.

Link: https://doi.org/10.3390/wevj16060306

[4] W.-H. Lee and C.-Y. Chang, "Assessing driving risk level: Harnessing deep learning hybrid model with intercity bus naturalistic driving data," *IEEE Access*, vol. 13, pp. 2450–2462, Feb. 2025.

Link: https://doi.org/10.1109/ACCESS.2025.3538693

[5] K. Sarvajcz, L. Ari, and J. Sütő, "AI on the Road: NVIDIA Jetson Orin Nano-Powered Computer Vision-Based System for Real-Time Pedestrian and Priority Sign Detection," *Applied Sciences*, vol. 14, no. 4, p. 1440, 2024.

Link: https://doi.org/10.3390/app14041440

[6]I. García Daza, R. Izquierdo Gonzalo, L. M. Martinez, O. Benderius, and D. Fernández Llorca, "Sim-to-real transfer and reality gap modeling in model predictive control for autonomous driving," *Applied Intelligence*, vol. 53, no. 10, pp. 12719–12735, 2023.

Link: https://doi.org/10.1007/s10489-022-04148-1

[7] P. Mishra and P. Sehgal, "Novel Approach for Object Detection in Embedded Systems Using YOLO," *Lex Localis - Journal of Local Self-Government*, vol. 22, no. 2, pp. 483-499, 2024. Link: https://lex-localis.org/index.php/LexLocalis/article/download/801932/2310/24269

[8] M. H. A. Baharuddin, N. Darlis, and S. Sulaiman, "Modification of RC Car Design for Autonomous Vehicle Application," *Progress in Engineering Application and Technology*, vol. 6, no. 1, pp. 603-613, 2025.

Link:https://publisher.uthm.edu.my/periodicals/index.php/peat/article/download/19394/6103/114778

[9] A. Abdulmaksoud and R. Ahmed, "Transformer-Based Sensor Fusion for Autonomous Vehicles: A Comprehensive Review," *IEEE Access*, vol. 13, pp. 41823-41845, 2025.

Link: https://doi.org/10.1109/ACCESS.2025.3545032

[10] V. Shankar, "Embedded Systems for Edge AI: Hardware, Software, and Design Methodologies," *ResearchGate*, 2025.

Link**:** https://doi.org/10.13140/RG.2.2.29378.64962

[11] O. Abdullah and J. Adelusi, "Edge Computing vs. Cloud Computing: A Comparative Study of Performance, Scalability, and Latency," *ResearchGate*, Feb. 2025.

Link:https://www.researchgate.net/publication/388956556_Edge_Computing_vs_Cloud_Computing_A_Comparative_Study_of_Performance_Scalability_and_Latency

[12] A. K. Pallikonda, V. K. Bandarapalli, and V. Aruna, "Enhancing performance and reducing latency in autonomous systems through edge computing for real-time data processing," *Mechatron. Intell. Transp. Syst.*, vol. 4, no. 3, pp. 154–165, 2025.

Link: https://www.acadlore.com/article/MITS/2025_4_3/mits040305

[13] T. Bai, D. Shao, Y. He, Q. Yang, Y. Feng, and S. Fu, "Privacy-Preserving Driver Monitoring on the Edges: Transformer-Based Processing of Secret Shares from Video Streams," *ResearchGate*, Dec. 2025.

Link: https://www.researchgate.net/publication/398495868_Privacy-Preserving_Driver_Monitoring_on_the_Edges_Transformer-Based_Processing_of_Secret_Shares_from_Video_Streams

[14] S. Kim, J. Park, Y. Jeong, and S. E. Lee, "Intelligent Monitoring System with Privacy Preservation Based on Edge AI," *Sensors (Basel)*, vol. 23, no. 18, Sep. 2023.

Link: https://pmc.ncbi.nlm.nih.gov/articles/PMC10536670/.

[15] A. Kuznietsov, B. Gyevnar, C. Wang, S. Peters, and S. V. Albrecht, "Explainable AI for Safe and Trustworthy Autonomous Driving," *arXiv preprint arXiv:2402.10086*, July 2024.

Link: https://arxiv.org/html/2402.10086v2.

[16] G. De Gasperis and S. D. Facchini, "A Comparative Study of Rule-Based and Data-Driven Approaches in Autonomous Vehicle Decision Making," *arXiv preprint arXiv:2509.15848*, Sep. 2025.

Link: https://arxiv.org/html/2509.15848v1.

[17] S. H. Pothineni, "Offline-First Mobile Architecture: Enhancing Usability and Resilience in Mobile Systems," *Journal of Artificial Intelligence General Science (JAIGS)*, vol. 7, no. 1, pp. 320–326, Aug. 2025.

Link: https://www.researchgate.net/publication/393910615_Offline-First_Mobile_Architecture_Enhancing_Usability_and_Resilience_in_Mobile_Systems.

APPENDIX A: Source Code and Repository

Phase 1 Android repository: https://github.com/muhammed110411/fleet-driver-monitoring-system

Phase 2 backend repository: https://github.com/muhammed110411/BACKEND-SERVER-PHASE2

Phase 1 key source areas: app/src/main/java/com/example/capstone/app, core, domain, data, network, orchestration, ui, mock, and debug.

Phase 1 key classes include ScoringEngine.kt, ContinuousRiskCalculator.kt, EventRiskCalculator.kt, EscalationRiskCalculator.kt, SessionState.kt, AppDatabase.kt, TelemetryParser.kt, TelemetryMapper.kt, WebSocketManager.kt, JetsonControlApiService.kt, BackendSessionApiService.kt, BackendAuthApi.kt, BackendDriverApiService.kt, SyncManager.kt, MainActivity.kt, and the ViewModel/screen packages.

Phase 2 key source areas: src/main/java/com/example/phase2/domain, persistence, application, api, web, security, config, exception, and util.

Phase 2 key classes include Phase2Application.java, SecurityConfig.java, ApiClientAuthFilter.java, AdminSessionAuthFilter.java, SessionUploadApiController.java, SessionUploadService.java, UploadStructuralValidator.java, UploadSoftValidationAnalyzer.java, DriverLifecycleService.java, VehicleManagementService.java, SessionQueryService.java, AnalyticsQueryService.java, AdminPageController.java, DriverPageController.java, VehiclePageController.java, SessionPageController.java, and the JPA entity/repository packages.

Representative scoring pseudocode:

```
processTelemetry(sessionId, state, packet)
continuousPenalty = continuousRiskCalculator.calculatePenalty(packet, state)
events = eventDetector.detectEvents(sessionId, packet, state)
escalations = escalationDetector.detectEscalations(sessionId, packet, events, state)
currentScore = maxScore - totalPenalties
return ScoringStepResult(currentScore, totals, events, escalations)
```

Representative integration flow:

```
adminCreatesDriver(driverData)
backend stores driver account and account status
driver logs in from Android while internet-connected
app scans for Jetson vehicle access point
app reads vehicle state
if vehicle is AVAILABLE:
app requests PENDING reservation
if driver confirms before expiry:
app sends CLAIM
vehicle becomes BUSY
app processes telemetry locally
app refreshes lease through heartbeat/health checks
when session ends:
app finalizes local session artifacts
app uploads completed session to Phase 2
backend validates, persists, and exposes session to admin website
```

AI road monitoring subsystem Core Files:

1. demo_trt.py: main TensorRT lane deviation pipeline.
2. ai_inference.py : software integration bridge.
3. start_yolo.sh: Jetson autostart script.
4. build_engine.py: TensorRT engine generation.
5. demo_imx.py: The same logic as demo_trt.py but with PyTorch fallback pipeline.

APPENDIX B: User Manual / Installation Guide

Phase 1 Android build and run steps:

1. Clone or open the fleet-driver-monitoring-system repository in Android Studio.
2. Ensure the required Android SDK, Gradle, Kotlin plugin, Android Gradle Plugin, and JDK versions are installed.
3. Open the project root containing settings.gradle.kts and the app module.
4. Let Gradle sync the version catalog and app dependencies.
5. Build the debug APK using .\gradlew.bat assembleDebug on Windows or ./gradlew assembleDebug on Unix-like systems.
6. Run local unit tests using .\gradlew.bat testDebugUnitTest or ./gradlew testDebugUnitTest.
7. Install the debug APK on an emulator or Android device.
8. Configure backend and Jetson endpoint values before real integration testing. Prototype defaults should not be treated as production secrets.
9. Log in using a backend-created driver account. Driver self-signup should not be used as the final driver-creation authority.
10. For live-session behavior, connect the app to the expected Jetson access point, telemetry WebSocket, and Jetson-control API host.

Phase 2 backend build, run, and access steps:

1. Clone or open the BACKEND-SERVER-PHASE2 repository in an IDE or terminal with Java 21 available.
2. Configure MySQL connection values in the appropriate Spring profile properties.
3. Build and test the backend with the Gradle wrapper.
4. Start the Spring Boot application using the selected local/dev/prod profile.
5. Access the admin website through the deployed website link: [ADMIN WEBSITE].
6. Log in as an administrator through the protected admin login page: username: ops-admin / password: Password1.
7. Create driver accounts through the admin-side driver management flow.
8. Use authenticated API-client credentials for completed-session upload testing.
9. Use the admin pages or protected admin APIs to review drivers, vehicles, sessions, dashboard data, and analytics.
10. Access the Grafana monitoring dashboard through [GRAFANA], subject to the configured server access and authentication policy.

APPENDIX C: Test Data and Additional Results

Phase 1 build command executed: .\gradlew.bat assembleDebug. Result: BUILD SUCCESSFUL in 18 seconds.

Phase 1 local tests covered scoring, session state, telemetry mapping, Jetson mapper/control/discovery, synchronization, backend DTO/mappers/auth API, credential validation, BCrypt password hashing, and localization.

Phase 1 login credential policy validation result: Pass.

Phase 2 testing covered admin authentication, protected admin pages/APIs, driver and vehicle management, completed-session ingestion, persistence of session child records, dashboard pages, session detail review, global analytics, and per-driver analytics.

APPENDIX D: Meeting Minutes and Project Logs

[Include key meeting minutes, project logs, and communication records documenting the project coordination activities.]

APPENDIX E: Poster / Presentation Slides

[Include the project poster and/or final presentation slides.]